

Introduction to
Simultaneous
Localization and Mapping I:
Bayesian Estimation

First Edition, 2024

Hyeonwoo Yu
Department of Intelligent Robotics,
Sungkyunkwan University

To Yunjeong

Contents

1	Introduction	1
2	Probability and Estimation	3
2.1	Basic Concepts in Probability	3
2.2	Multivariate Gaussian and Conditional Probability	6
3	Bayesian Estimation and Measurement	11
3.1	Maximum Likelihood Estimation	11
3.2	Bayesian Mean Square Error	14
4	Markov Chain and Bayesian Filtering	19
4.1	Sequential Measurement and Posterior	20
4.2	Markov Chain and Dynamic Model	23
4.3	Bayesian Linear Filter	29
5	Nonlinear Filter	35
5.1	Extended Kalman Filter	36
5.2	Particle Filter	42
6	Filter-based SLAM	49
6.1	Measurement and Landmark	49
6.2	EKF SLAM	51
6.3	Fast SLAM	57
7	Graph Optimization	65
7.1	Pose Graph and Maximum A Posteriori	65
7.2	Gauss-Newton Method and Graph Optimization	68
8	Graph SLAM	71
8.1	Graph Optimization and Gauss-Newton Method	71
8.2	Graph Optimization and Levenberg-Marquardt	74
8.3	Jacobians for $SE2$	76
8.4	Jacobians for $SE3$	80

1

Introduction

“There is no spoon”

– Neo, *The Matrix* (1999)

Suppose you are standing alone in the environment, and only have eyesights. How do you define where you at? The location is relative, therefore you can define your location by determining the locations of the surroundings existing in the environment. You may look around, see what exist and determine their locations related to your location. Here, you can simply set your present location as the origin of this environment for now. If you want to explore the environment, things become more challenging. When you move, apparently your location has been changed, and you can naturally feel the changes through your observations. Even further, you can also determine your changed location related to the locations of the surroundings. Here the problem occurs. In the viewpoint of yourself, when you move, the locations of the surroundings have also been changed because their locations are determined related to your location. Your location is determined by the locations of the surroundings observed by yourself, thus the locations of the surroundings should be known; the locations of the surroundings are determined by your location, thus your location should be known. In the real life, unfortunately both are unknown and you only have your own observations which are the relation between your location and the surroundings' locations.

Exactly same issue happens in Robotics. Suppose you have a mobile robot explores in the environment, and obtain sequential observations. Since the robot's location and the surroundings' locations are both unknown, the robot should perform simultaneous robot localization and the surroundings' localization from sequential observations. The localization of the surrounding environment can be seen as drawing a map, called mapping of the environment. So in robotics simultaneous localization and mapping, SLAM is the basic essence of the mobile robot.

The localization of the Various sensors equipped to a mobile robot work as your eyesight. Similar to your observations of the surroundings, a mobile

robot gets sensor information of the surroundings, called landmark. Any form of the sensor information can be used as landmarks, such as raw sensor values or geometric features like lines or edges computed from the sensor information, or more high-level semantic features.

When the environment is previously explored and known, the prior map can be given, and the global robot location also can be given from the external system. However, even in this well-known and well-set environment SLAM should be performed to 1) obtain the local map for place matching between the global map and the robot observations, 2) overcome the uncertainty of the global localization system and obtain the precise local localization, and 3) handle the dynamic environment.

In this textbook, the main concept of SLAM will be discussed.

2

Probability and Estimation

2.1 Basic Concepts in Probability

In this section, we briefly discuss the probabilistic essences for the robotics. Humans can experience the world by feeling various senses. Similarly, in the real-world scenarios, the mobile robots equipped sensors measure various quantities in real-time in order to experience the world. These quantities can be anything that can represent the robots such as the locations of the robots or the surrounding features. Unfortunately, every sensors do have noise and always have the uncertainties on their measurements, thus all the measurements should be considered as the random variable which follows the certain probability distributions according to the probabilistic modeling of sensor data. Probabilistic inference is therefore necessary in robotics in order to estimate the values of quantities from the noisy measurements, under the specific laws of probability.

Let $\mathbf{x}$ be the random variable of any quantities such as the robot location. Suppose $\mathbf{x}$ follows a probability distribution $p(\mathbf{x})$. Let $\mathbf{x}$ be the specific state or values might take on, such as the specific states of the robot turn on/off, or specific coordinates of the current robot location. In the case of the discrete random variable like turn on/off of the robot, we have the probability

$$p(\mathbf{x}) = p(\mathbf{x} = \mathbf{x}) \geq 0 \quad (2.1)$$

to represent that the random variable $\mathbf{x}$ has the value of $\mathbf{x}$. From the axiom of probability, the sum of the probability over the all possible values is one as the following:

$$\sum_{\mathbf{x}} p(\mathbf{x} = \mathbf{x}) = 1. \quad (2.2)$$

For simplicity, from now we use $p(\mathbf{x})$ instead of writing $p(\mathbf{x} = \mathbf{x})$.

The major quantities we will handle in this book, such as the measurement of the robot location, are represented in continuous random various,

because they exist in continuous spaces. These continuous random variables follow the specific probability density function (PDF), defined by their own probabilistic models which reflect the characteristics of the variables. In the case of the measurement of the sensor, for example, the measurement value always poses a noise due to the nature of the sensor. This noise model is used as the measurement model, which defines the probabilistic model and the PDF of the measurement. A common PDF is Gaussian distribution with mean μ and variance σ^2 given as the following:

$$p(\mathbf{x}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \frac{(\mathbf{x} - \mu)^2}{\sigma^2}\right) \\ \stackrel{\text{def}}{=} \mathcal{N}(\mu, \sigma^2). \quad (2.3)$$

In the aspect of the measurement example, μ can be regarded as the true quantity measured by the sensor, σ as the sensor uncertainty, and $\mathbf{x}$ as the value obtained from the measurement.

Since the quantities such as the sensor measurement contain multiple values in usual, the random variable $\mathbf{x}$ can take multi-dimensional variables. In the case of the multi-dimensional case, multivariate Gaussian distribution is given as the following:

$$p(\mathbf{x}) = \frac{1}{\det(2\pi\mathbf{\Sigma})^{\frac{1}{2}}} \exp\left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \mathbf{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})\right) \\ \stackrel{\text{def}}{=} \mathcal{N}(\boldsymbol{\mu}, \mathbf{\Sigma}), \quad (2.4)$$

where $\mathbf{x}$ is the multivariate Gaussian random variable, $\boldsymbol{\mu}$ is the mean vector and $\mathbf{\Sigma}$ a symmetric, positive semidefinite matrix called covariance matrix.

Same as the discrete probability distribution, a PDF of continuous random variable integrates to 1:

$$\int_{\mathbf{x}} p(\mathbf{x}) d\mathbf{x} = 1. \quad (2.5)$$

Similar to the case of sensor measurement, in this book we assume all the continuous quantities can be represented as the continuous random variable; that is, all the continuous quantities are measurable, and possess probability densities.

Joint Distribution and Marginalization

Suppose we have two random variables $\mathbf{x}$ and $\mathbf{y}$. These random variables can be jointly defined for the measurements from multiple sensors of one quantity such as a robot location, or two different quantities such as each

components of a robot location on two-dimensional coordinates. Here the *joint distribution* of $\mathbf{x}$ and $\mathbf{y}$ is given as:

$$p(\mathbf{x}, \mathbf{y}) = p(\mathbf{x} = \mathbf{x} \text{ and } \mathbf{y} = \mathbf{y}). \quad (2.6)$$

By integrating the joint distribution over the one random variable, say over $\mathbf{x}$, we have:

$$\int_{\mathbf{x}} p(\mathbf{x}, \mathbf{y}) d\mathbf{x} = p(\mathbf{y}). \quad (2.7)$$

Here we call this integration as *marginalization*, and $p(\mathbf{y})$ is the *marginal distribution*.

Independence

If $\mathbf{x}$ and $\mathbf{y}$ are independent, the joint distribution can be expressed as:

$$p(\mathbf{x}, \mathbf{y}) = p(\mathbf{x}) p(\mathbf{y}). \quad (2.8)$$

In usual, random variables involve information of other random variables. Suppose we observe that one of the sensor value $\mathbf{y}$ is determined as $\mathbf{y}$, and we want to figure out the probability that the other sensor value $\mathbf{x}$ is detected as $\mathbf{x}$ conditioned on our previous observation. In this case we denote the *conditional probability* as:

$$p(\mathbf{x}|\mathbf{y}) = p(\mathbf{x} = \mathbf{x}|\mathbf{y} = \mathbf{y}). \quad (2.9)$$

Using the marginal distribution and the joint distribution, the conditional probability is defined as:

$$p(\mathbf{x}|\mathbf{y}) \stackrel{\text{def}}{=} \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{y})} \quad (2.10)$$

When $p(\mathbf{y})$ is nonzero and $\mathbf{x}$ and $\mathbf{y}$ are independent, substituting Eqn. (2.8) into Eqn. (2.10) we have:

$$p(\mathbf{x}|\mathbf{y}) = \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{y})} = \frac{p(\mathbf{x}) p(\mathbf{y})}{p(\mathbf{y})} = p(\mathbf{x}). \quad (2.11)$$

Therefore, when the random variables are independent their conditional distributions are not *conditioned* to other random variables, thereby the conditional distribution is equal to the marginal distribution.

Expectation

The random variable $\mathbf{x}$ is generated by its probability $p(\mathbf{x})$. Therefore, when we compute the mean of the variable $\mathbf{x}$ it is necessary to consider its probability, not by simple summation and division by the number of variables,

but by weighting the value $\mathbf{x}$. This weighted summation can be achieved as the following:

$$\mathbb{E}(\mathbf{x}) = \int_{\mathbf{x}} \mathbf{x} \cdot p(\mathbf{x}) d\mathbf{x} \quad (2.12)$$

where $\mathbb{E}(\mathbf{x})$ is the *expectation* (mean). In general, when we want to compute the expectation of the value derived from the function f , then the expectation can be expressed as:

$$\mathbb{E}(\mathbf{x}) = \int_{\mathbf{x}} f(\mathbf{x}) p(\mathbf{x}) d\mathbf{x}. \quad (2.13)$$

2.2 Multivariate Gaussian and Conditional Probability

In the realm of Simultaneous Localization and Mapping (SLAM), the synergy between multivariate Gaussian distributions and conditional probability plays a pivotal role in enhancing our understanding of the environment. At the core of this integration lies the recognition that valuable information about the robot's position and surroundings is not solely confined to the direct measurements obtained from a single sensor. Instead, the intricate nature of the environment often necessitates the consideration of additional data sources, such as observations from other sensors.

Conditional probability becomes a necessity in this context as it enables us to quantify the likelihood of a certain measurement given the information gleaned from other sensors. Essentially, it allows us to refine our estimation by incorporating the influence of related observations, providing a more comprehensive and accurate representation of the true state of the environment. This chapter delves into the fundamental principles of multivariate Gaussian distributions and conditional probability, unraveling their significance in harnessing diverse sources of information for a more robust and nuanced SLAM framework.

Suppose we have two Gaussian random variables $\mathbf{x}$ and $\mathbf{y}$ that are jointly Gaussian, where $\mathbf{x}$ is $k \times 1$ and $\mathbf{y}$ is $l \times 1$, with mean vector $[\mathbb{E}(\mathbf{x})^T \mathbb{E}(\mathbf{y})^T]^T$ and partitioned covariance matrix

$$\mathbf{C} = \begin{bmatrix} \mathbf{C}_{xx} & \mathbf{C}_{xy} \\ \mathbf{C}_{yx} & \mathbf{C}_{yy} \end{bmatrix} = \begin{bmatrix} k \times k & k \times l \\ l \times k & l \times l \end{bmatrix} \quad (2.14)$$

thus we have

$$p(\mathbf{x}, \mathbf{y}) = \frac{1}{\sqrt{(2\pi)^{k+l} \det \mathbf{C}}} \exp \left(-\frac{1}{2} \begin{bmatrix} \mathbf{x} - \mathbb{E}(\mathbf{x}) \\ \mathbf{y} - \mathbb{E}(\mathbf{y}) \end{bmatrix}^T \mathbf{C}^{-1} \begin{bmatrix} \mathbf{x} - \mathbb{E}(\mathbf{x}) \\ \mathbf{y} - \mathbb{E}(\mathbf{y}) \end{bmatrix} \right), \quad (2.15)$$

2.2. MULTIVARIATE GAUSSIAN AND CONDITIONAL PROBABILITY 7

then the conditional PDF $p(\mathbf{y}|\mathbf{x})$ is also Gaussian represented as:

$$\mathbf{y}|\mathbf{x} \sim \mathcal{N}(\mathbb{E}(\mathbf{y}|\mathbf{x}), \mathbf{C}_{y|x}) \quad (2.16)$$

where

$$\mathbb{E}(\mathbf{y}|\mathbf{x}) = \mathbb{E}(\mathbf{y}) + \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}(\mathbf{x} - \mathbb{E}(\mathbf{x})) \quad (2.17)$$

$$\mathbf{C}_{y|x} = \mathbf{C}_{yy} - \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}\mathbf{C}_{yx}. \quad (2.18)$$

In the case of Bivariate Gaussian random variable $[x, y]^T$, the covariance $\mathbf{C}$ can be simplified to:

$$\mathbf{C} = \begin{bmatrix} \text{var}(x) & \text{cov}(x, y) \\ \text{cov}(y, x) & \text{var}(y) \end{bmatrix} \quad (2.19)$$

so that

$$\mathbb{E}(y|x) = \mathbb{E}(y) + \frac{\text{cov}(x, y)}{\text{var}(x)}(x - \mathbb{E}(x)) \quad (2.20)$$

$$\text{var}(y|x) = \text{var}(y) - \frac{\text{cov}^2(x, y)}{\text{var}(x)} \quad (2.21)$$

$$= \text{var}(y) \left(1 - \frac{\text{cov}^2(x, y)}{\text{var}(x)\text{var}(y)} \right) \quad (2.22)$$

$$= \text{var}(y)(1 - \rho), \quad (2.23)$$

where ρ is Correlation coefficient.

Proof of Conditional Gaussian

We can easily compute the conditional mean and the conditional covariance by directly applying the definition of the conditional probability. Since we have the joint PDF $p(\mathbf{x}, \mathbf{y})$, the conditional PDF $p(\mathbf{y}|\mathbf{x})$ can be represented as:

$$\begin{aligned} p(\mathbf{y}|\mathbf{x}) &= \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})} \\ &= \frac{\frac{1}{\sqrt{(2\pi)^{k+l} \det \mathbf{C}}} \exp \left(-\frac{1}{2} \begin{bmatrix} \mathbf{x} - \mathbb{E}(\mathbf{x}) \\ \mathbf{y} - \mathbb{E}(\mathbf{y}) \end{bmatrix}^T \mathbf{C}^{-1} \begin{bmatrix} \mathbf{x} - \mathbb{E}(\mathbf{x}) \\ \mathbf{y} - \mathbb{E}(\mathbf{y}) \end{bmatrix} \right)}{\frac{1}{\sqrt{(2\pi)^k \det \mathbf{C}_{xx}}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mathbb{E}(\mathbf{x}))^T \mathbf{C}_{xx}^{-1} (\mathbf{x} - \mathbb{E}(\mathbf{x})) \right)}. \end{aligned} \quad (2.24)$$

To calculate the term $\frac{\det \mathbf{C}}{\det \mathbf{C}_{xx}}$, we use the Schur complement. Suppose the matrices $\mathbf{A}_{11}$ and $\mathbf{A}_{22}$ are invertible and let $\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix}$. Then the

Schur complement is defined as follows:

$$\begin{bmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{A}_{21}\mathbf{A}_{11}^{-1} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \begin{bmatrix} \mathbf{I} & -\mathbf{A}_{11}^{-1}\mathbf{A}_{12} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_{22} - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12} \end{bmatrix}. \quad (2.25)$$

Taking determinant to both side we have:

$$\det \left(\begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \right) = \det \mathbf{A}_{11} \det \left(\mathbf{A}_{22} - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12} \right). \quad (2.26)$$

Substitute $\mathbf{A}$ with $\mathbf{C}$, we have:

$$\det \mathbf{C} = \det \mathbf{C}_{xx} \det \left(\mathbf{C}_{yy} - \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}\mathbf{C}_{yx} \right) \quad (2.27)$$

so that

$$\frac{\det \mathbf{C}}{\det \mathbf{C}_{xx}} = \det \left(\mathbf{C}_{yy} - \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}\mathbf{C}_{yx} \right). \quad (2.28)$$

Substituting Eqn. (2.28) to Eqn. (2.24) we have:

$$p(\mathbf{y}|\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^l \det \left(\mathbf{C}_{yy} - \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}\mathbf{C}_{yx} \right)}} \exp \left(-\frac{1}{2}Q \right), \quad (2.29)$$

where

$$Q = \begin{bmatrix} \mathbf{x} - \mathbb{E}(\mathbf{x}) \\ \mathbf{y} - \mathbb{E}(\mathbf{y}) \end{bmatrix}^T \mathbf{C}^{-1} \begin{bmatrix} \mathbf{x} - \mathbb{E}(\mathbf{x}) \\ \mathbf{y} - \mathbb{E}(\mathbf{y}) \end{bmatrix} - (\mathbf{x} - \mathbb{E}(\mathbf{x}))^T \mathbf{C}_{xx}^{-1} (\mathbf{x} - \mathbb{E}(\mathbf{x})). \quad (2.30)$$

From Eqn. (2.28), we can also have $\mathbf{C}^{-1}$ as the following:

$$\mathbf{C}^{-1} = \begin{bmatrix} \mathbf{I} & -\mathbf{C}_{xx}^{-1}\mathbf{C}_{xy} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{C}_{xx}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}^{-1} \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{C}_{yx}\mathbf{C}_{xx}^{-1} & \mathbf{I} \end{bmatrix} \quad (2.31)$$

where $\mathbf{B} = \mathbf{C}_{yy} - \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}\mathbf{C}_{yx}$. Let $\bar{\mathbf{x}} = \mathbf{x} - \mathbb{E}(\mathbf{x})$ and $\bar{\mathbf{y}} = \mathbf{y} - \mathbb{E}(\mathbf{y})$. Substituting Eqn. (2.31) into Eqn. (2.30) we have:

$$\begin{aligned} Q &= \begin{bmatrix} \bar{\mathbf{x}} \\ \bar{\mathbf{y}} \end{bmatrix}^T \begin{bmatrix} \mathbf{I} & -\mathbf{C}_{xx}^{-1}\mathbf{C}_{xy} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{C}_{xx}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}^{-1} \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{C}_{yx}\mathbf{C}_{xx}^{-1} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \bar{\mathbf{x}} \\ \bar{\mathbf{y}} \end{bmatrix} - \bar{\mathbf{x}}^T \mathbf{C}_{xx}^{-1} \bar{\mathbf{x}} \\ &= \left(\bar{\mathbf{y}} - \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}\bar{\mathbf{x}} \right)^T \mathbf{B}^{-1} \left(\bar{\mathbf{y}} - \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}\bar{\mathbf{x}} \right) \\ &= \left[\mathbf{y} - \left(\mathbb{E}(\mathbf{y}) + \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}(\mathbf{x} - \mathbb{E}(\mathbf{x})) \right) \right]^T \\ &\quad \left[\mathbf{C}_{yy} - \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}\mathbf{C}_{yx} \right]^{-1} \left[\mathbf{y} - \left(\mathbb{E}(\mathbf{y}) + \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}(\mathbf{x} - \mathbb{E}(\mathbf{x})) \right) \right]. \quad (2.32) \end{aligned}$$

2.2. MULTIVARIATE GAUSSIAN AND CONDITIONAL PROBABILITY9

Substituting Eqn. (2.32) into Eqn. (2.29), we finally have:

Conditional Probability of Gaussian:

$$\mathbf{y}|\mathbf{x} \sim \mathcal{N}\left(\mathbb{E}(\mathbf{y}|\mathbf{x}), \mathbf{C}_{\mathbf{y}|\mathbf{x}}\right) \quad (2.33)$$

where

$$\mathbb{E}(\mathbf{y}|\mathbf{x}) = \mathbb{E}(\mathbf{y}) + \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}(\mathbf{x} - \mathbb{E}(\mathbf{x})) \quad (2.34)$$

$$\mathbf{C}_{\mathbf{y}|\mathbf{x}} = \mathbf{C}_{yy} - \mathbf{C}_{yx}\mathbf{C}_{xx}^{-1}\mathbf{C}_{yx}. \quad (2.35)$$

The most important thing here is that the conditional distribution of two Gaussian random variables is still represented as a Gaussian distribution. We will utilize this fact for the remaining chapters.

3

Bayesian Estimation and Measurement

Consider the robot is about to explore the environment. In order to understand the surrounding environment, first the robot should obtain the sensor information about the environment, for example distances between the robot and the surrounding structures. The sensor information, in this case, is a measurement of the true distance. The problem lies here. In the realm of Robotics, our sensors serve as imperfect observers, providing us with measurements that are, in essence, true essence of the environment involving noises. This “true essence” represents the genuine state of the surrounding environments or events, unblemished by the inherent inaccuracies of the sensors. Our goal is to uncover the genuine nature concealed within the noisy sensor measurements, and achieve the true essence of the surroundings for the robot perception and further robot applications. Simply, our goal is to estimate the true value from the measurements.

In order to discern and achieve this authentic landscape, we turn to the probabilistic estimation theory. Apparently, it is impossible to know the correct noise model of the certain sensors. However this is not the end. We empirically know that the various noise models in nature approximately follow the known distributions such as Gaussian distribution, thereby it is possible to approximate and make sense of the true reality that lies beneath the surface of our sensor measurements. Here Bayesian estimation becomes our guiding light from now.

3.1 Maximum Likelihood Estimation

Suppose the robot is equipped with the time-of-flight (ToF) sensor and is in a stationary state. Consider that the robot is measuring the distance to the structure placed in front. Here, let $x[n]$ be the measurement obtained

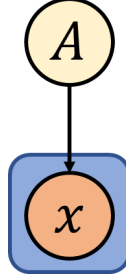


Figure 3.1: Graph of the true value A and the measurement x . The measurement model can be represented as the Graph structure.

from the sensor in time n , and let A be the true distance between the sensor and the structure. Since the measurement of the sensor inevitably involves noise, we cannot say that the measurement $x[n]$ is the true distance A , but we can approximately find the value of A in probabilistic manner.

For this estimation, first we need to assume the noise model (or measurement model) of the sensor with known distributions. In usual, Gaussian distribution plays a main role as the measurement model. Assume that $w[n]$ is the random variable representing the noise occurred in the n^{th} measurement. Let us say the robot obtains N times measurements, then we can express the relation between $x[n]$ and A as the following:

$$x[n] = A + w[n], \quad n = 0, 1, \dots, N-1 \quad (3.1)$$

where

$$w[n] \sim \mathcal{N}(0, \sigma^2). \quad (3.2)$$

Here σ^2 reflects the uncertainty of the sensor measurement. In this example let us assume that σ^2 is known, which can be obtained empirically in practice. The relation between A and x are depicted as a graph in Fig. 3.1.

Since $w[n]$ is a Gaussian random variable and A is a constant, $x[n]$ is also a Gaussian random variable with the following probability distribution:

$$p(x[n]) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(x[n] - A)^2}{2\sigma^2} \right], \quad (3.3)$$

which is the *likelihood* of the random variable $x[n]$. In this equation, apparently A is a constant hence not a probabilistic random variable. However, in usual we express A as if a random variable for convenience, because when the value of A is determined, we can say the random variable $x[n]$ follows the above probability distribution which is determined by A . In other words,

the distribution of $x[n]$ is conditionally determined by A . In this context, we can also rewrite this likelihood as if the conditional probability:

$$p(x[n]; A) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(x[n] - A)^2}{2\sigma^2} \right]. \quad (3.4)$$

Apparently, in order to represent the conditional probability the left-hand side of the above equation should be $p(x[n] | A)$. However, we need to note that the likelihood is not a conditional probability in usual because A in our case is a constant, not a random variable. Therefore we simply use a semicolon to express the relation between $x[n]$ and A in the likelihood.

The likelihood involves the noise model of the measurement, thus we can utilize the likelihood as the measurement model (or observation model). Since the robot measures the distance N times, under the assumption that all the measurements are independent to each other we can represent the likelihood of N measurements as the following:

$$\begin{aligned} p(\mathbf{x}; A) &= p(x[0], \dots, x[N-1]; A) \\ &= \prod_{n=0}^{N-1} \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2\sigma^2} (x[n] - A)^2 \right] \\ &= \frac{1}{(2\pi\sigma^2)^{\frac{N}{2}}} \exp \left[-\frac{1}{2\sigma^2} \sum_{n=0}^{N-1} (x[n] - A)^2 \right] \end{aligned} \quad (3.5)$$

where $\mathbf{x} = \{x[0], \dots, x[N-1]\}$. To find the optimal parameter of A , the first approach we can consider is Maximum Likelihood Estimation (MLE). Suppose we have optimal parameter $\hat{A}$ for the true distance. Since we assume that the measurements $\mathbf{x}$ is sampled from the likelihood, this likelihood has the maximum probability with the optimal parameter $\hat{A}$ thus we have the following expression:

$$\hat{A} = \underset{A}{\operatorname{argmax}} p(\mathbf{x}; A) \quad (3.6)$$

Therefore, the optimal parameter $\hat{A}$ can be calculated by taking the derivation of the likelihood. For convenience, in usual we use log-likelihood for Gaussian distribution, since Gaussian involves quadratic form which is convex and the logarithm is monotonically increasing function so that the optimal point remains the same while in logarithm form. Therefore we have:

$$\hat{A} = \underset{A}{\operatorname{argmax}} \log p(\mathbf{x}; A) \quad (3.7)$$

where

$$\begin{aligned}\log p(\mathbf{x}; A) &= -\frac{N}{2} \log 2\pi\sigma^2 - \frac{1}{2\sigma^2} \sum_{n=0}^{N-1} (x[n] - A)^2 \\ &= \text{const} - \frac{1}{2\sigma^2} \sum_{n=0}^{N-1} (x[n] - A)^2.\end{aligned}\quad (3.8)$$

Note that in the case of Gaussian, we have quadratic form inside of the exponential form when taking logarithm of Gaussian. This will play an important role in further. By taking the partial derivative of the log-likelihood, we have the following:

$$\frac{\partial \log p(\mathbf{x}; A)}{\partial A} = \frac{1}{\sigma^2} \sum_{n=0}^{N-1} (x[n] - A). \quad (3.9)$$

Set to zero we finally have:

$$\hat{A} = \frac{1}{N} \sum_{n=0}^{N-1} x[n]. \quad (3.10)$$

Therefore, for the measurements of the constant quantities such as the distance between the static robot and the surrounding structure, we can conclude that the optimal parameters can be determined by taking average of the multiple measurement. Note that the noise model (or observation model) here is assumed to be Gaussian.

3.2 Bayesian Mean Square Error

Estimating the true value of the target quantities always poses a great challenge, due to the measurement noise or model uncertainty. In the previous section we handled this uncertainty by assuming the measurement model as Gaussian and by applying MLE. In this section we handle this noise by simply minimizing the error and compare the result to the MLE case. Assume that the robot at a standstill measures the distance to the static object and obtain the measurement $x[n]$ in time n , and let A be the true distance. Due to the noises from the measurement model, the measurement $x[n]$ and the true distance A always pose a difference, which is the error that we need to minimize. To compute the error, we define square error criterion J as the cost function as the following:

$$J(A) = \sum_{n=0}^{N-1} (x[n] - A)^2. \quad (3.11)$$

Apparently, the optimal solution $\hat{A}$ of this quadratic form can be given as:

$$\hat{A} = \frac{1}{N} \sum_{n=0}^{N-1} x[n], \quad (3.12)$$

which is the same result of the optimal solution from MLE.

Although the square error enables us to attain the same solution in Gaussian scenarios, it falls short in providing insights beyond measurement models expressed solely in terms of likelihood.

In robotics, there are instances where various information can be anticipated in advance through pre-existing knowledge of the system model. For example, having prior information about the surrounding environment allows for a rough anticipation of the positions of observed objects. Additionally, knowing the control input provided to the robot enables the prediction of the robot's anticipated location before movement. From this, predictions can be made regarding how far an observed object will be from the robot compared to its position before movement.

To illustrate, consider having prior knowledge about the approximate distance of an object placed in front of a robot. Let the true distance from the robot to the object be denoted as A , and the observed distance from the measurement be denoted as x . Since likelihood only represents the measurement model, given A , it mimics the probability density function (PDF) for the observed value x . Therefore, there is no avenue to incorporate our existing knowledge about the object's position. Here, A is simply a constant parameter.

However, with prior knowledge about the approximate range of distances to the object, we now possess probabilistic information about A , treating it as a random variable. For instance, if we have prior knowledge that A lies between a and b , the PDF for A can be expressed as follows:

$$p(A) = \begin{cases} b - a, & \text{if } a < A < b \\ 0 & \text{else} \end{cases}. \quad (3.13)$$

Considering such prior knowledge allows for compensating noisy observations or incorrect system models. For instance, when x is observed to be greater than or equal to b , since the probability of the true distance A being greater than or equal to b is zero, measures such as ignoring the square error in this case can be taken.

Now, let's apply the prior knowledge we have about the parameter A to define the square error. As A is also a random variable, we can consider not just the likelihood but the joint probability density function (PDF) of $\mathbf{x}$ and A , denoted as $p(\mathbf{x}, A)$. Let's denote the optimal value for the distance

as $\hat{A}$. The Bayesian mean square error ($Bmse$) is given by the following expression:

$$\begin{aligned} Bmse(\hat{A}) &= \mathbb{E}(A - \hat{A})^2 \\ &= \int \int (A - \hat{A})^2 p(\mathbf{x}, A) d\mathbf{x} dA. \end{aligned} \quad (3.14)$$

Note that this $Bmse$ can be considered as the weighted error summation, different from the traditional square error criterion J . Here each error $(A - \hat{A})^2$ has its corresponding weight $p(\mathbf{x}, A)$ so that the probability of the target error now can be reflected.

Since $p(\mathbf{x}, A) = p(A|\mathbf{x})p(\mathbf{x})$, we have:

$$Bmse(\hat{A}) = \int \left(\int (A - \hat{A})^2 p(A|\mathbf{x}) dA \right) p(\mathbf{x}) d\mathbf{x}. \quad (3.15)$$

Our goal is to have the optimal value $\hat{A}$ from the redefined error criterion $Bmse$. From the axiom of probability $p(\mathbf{x}) \geq 0$, therefore $\hat{A}$ can be obtained by minimizing the integral of A . Taking the derivation we have:

$$\begin{aligned} \frac{\partial}{\partial \hat{A}} \int (A - \hat{A})^2 p(A|\mathbf{x}) dA &= \int \frac{\partial}{\partial \hat{A}} (A - \hat{A})^2 p(A|\mathbf{x}) dA \\ &= \int -2(A - \hat{A}) p(A|\mathbf{x}) dA \\ &= -2 \int A p(A|\mathbf{x}) dA + 2\hat{A} \underbrace{\int p(A|\mathbf{x}) dA}_{=1}. \end{aligned} \quad (3.16)$$

Set to zero we finally have:

$$\begin{aligned} \hat{A} &= \int A p(A|\mathbf{x}) dA \\ &= \mathbb{E}(A|\mathbf{x}). \end{aligned} \quad (3.17)$$

In the end, the mean becomes the optimal solution for $Bmse$. That is, for arbitrary parameter A , the minimum mean square error estimator considering the joint PDF is given by the mean of $p(A|\mathbf{x})$, denoted as $\mu_{A|\mathbf{x}}$. Then what is $p(A|\mathbf{x})$? How do we find $p(A|\mathbf{x})$?

Bayes' Rule

$p(A|\mathbf{x})$ in Eqn. (3.17) is the posterior, the probability of A given $\mathbf{x}$, which can be represented using the *likelihood* and the *prior* by the following Bayes'

rule:

$$\begin{aligned}
 \underbrace{p(A|\mathbf{x})}_{\text{posterior}} &= \frac{p(\mathbf{x}|A)p(A)}{p(\mathbf{x})} && \text{(Bayes' Rule)} \\
 &= \frac{p(\mathbf{x}|A)p(A)}{\int p(\mathbf{x}, A) dA} && \text{(marginalization)} \\
 &= \frac{p(\mathbf{x}|A)p(A)}{\int p(\mathbf{x}|A)p(A) dA} && \text{(Def. of Conditional Dist.)} \\
 &\propto \underbrace{p(\mathbf{x}|A)}_{\text{likelihood}} \underbrace{p(A)}_{\text{prior}} && (p(\mathbf{x}) \text{ is Constant}). \tag{3.18}
 \end{aligned}$$

Note that $\mathbf{x}$ are measurements. In other words, $\mathbf{x}$ denote the observations of the true value A , which is the parameter of our probability. At this point of time, the observations $\mathbf{x}$ are the determined variables observed in the past, even if those variables are sampled from the probability distribution. Therefore $\mathbf{x}$ are the known variables thus we say $p(\mathbf{x})$ in Eqn. (3.18) becomes constant. In Bayesian estimation, $\mathbf{x}$ are also called as *evidence*, since the observations $\mathbf{x}$ play a key role of estimating the desired parameter A .

In general, directly obtaining, assuming or formulating the posterior is challenging, compared to the cases of the likelihood (measurement model), or the prior (range of the parameter). Similarly, considering $p(\mathbf{x})$ is more challenging, because this distribution is the *generativemodel*. Therefore, in order to computing the posterior, simply from the likelihood and the prior, we will utilize Eqn. (3.18) in the rest of the chapters in this book.

Consequently, finding the optimal solution of Bayesian mean square error is equal to the solution of MAP, and the solution is determined as the mean of the posterior distribution. In summary, for Bayesian estimation we follow:

- Find the likelihood, mostly representing the observation model
- Assume the reasonable prior
- Compute the posterior from the likelihood and the prior by Bayes' rule
- Use the mean of the posterior as the optimal solution of the Bayesian estimation

4

Markov Chain and Bayesian Filtering

In the context of embodied AI, where robots engage in real-time interactions with their environment, the intricacies of dynamic systems come to the forefront. Unlike computer vision and simple AI, embodied AI involves robots that not only process information but also move and operate in real-time scenarios. This paradigm shift necessitates a deeper exploration of the role of filters, such as the Kalman filter, within the dynamics of robotics, especially considering the Markov assumption and Markov chain.

Let's explore this perspective with a scenario involving a mobile robot exploring an environment. The robot's location is measured through sensors, and its dynamic model and measurement model are crucial components in this process. Consider a mobile robot exploring an environment, measuring its location through sensors. The robot's position is obtained either by directly measuring its distance to specific objects within the environment or by external observations using sensors. Regardless of the method, let's assume we have a measurement of the robot's position.

Given that the robot operates in real-time, continuously measuring its surroundings at the system's frame rate, the dynamic nature of its motion introduces a time-dependent parameter to the estimation problem. Unlike scenarios where a stationary robot's position or the location of a static structure is measured, the key difference lies in the fact that the quantity to be estimated continually changes over time. In the case of a mobile robot, this evolving parameter is influenced by the robot's motion model. For instance, let's assume we know the robot's position and motion model at time steps 1 to t . Intuitively, for sufficiently short time intervals, the robot's position at time $t + 1$ would be determined by its position and motion model at times 1 to t . Graphically representing this relationship yields a node and edge graph. However, in the context of a real-time, continuously operating mobile robot, the accumulation of the robot's sequence over time results in a significant

computational drawback. To mitigate this, we introduce the Markov assumption to the mobile robot's sequence. Simply put, in a dynamic system, the state at time t is assumed to depend only on the state at time $t-1$, making the system Markovian. This assumption implies that, for a sufficiently short time interval, knowing the robot's position and motion model at time $t-1$ allows us to estimate its position at time t . This recursive application extends to states at times $t-1$ and $t-2$ and is graphically represented as a Markov chain.

The Markov assumption, applied to the mobile robot's sequence, alleviates the computational burden, enabling more efficient state estimation. This sequential calculation can be obtained through recursive calculations, both in real-time and in a post-processing manner after acquiring all the data. This narrative sheds light on the significance of the Markov assumption and Markov chain within the embodied AI framework, emphasizing the role of filters like the Kalman filter in addressing the challenges posed by real-time, dynamic robotic systems.

4.1 Sequential Measurement and Posterior

Recall that the Posterior can be represented with the prior and the likelihood as the following:

$$\begin{aligned} p(A|\mathbf{x}) &= \frac{p(\mathbf{x}|A)p(A)}{p(\mathbf{x})} \\ &= \frac{p(\mathbf{x}|A)p(A)}{\int p(\mathbf{x}|A)p(A)dA}. \end{aligned} \quad (4.1)$$

From now, we let z be the value of true location, instead of A . Since we have the series of observations $\mathbf{x} = \{x_0, \dots, x_t\}$, we can rewrite Eqn. (4.1) as follows:

$$\begin{aligned} p(z|\mathbf{x}) &= p(z|x_0, \dots, x_t) \\ &= \frac{p(x_t|z, x_0, \dots, x_{t-1})p(z|x_0, \dots, x_{t-1})}{\int p(x_t|z, x_0, \dots, x_{t-1})p(z|x_0, \dots, x_{t-1})dz}. \end{aligned} \quad (4.2)$$

For simplicity, from now let $x_{0:t-1} = x_0, \dots, x_{t-1}$. Since the measurement x_t is generated solely from z as shown in Fig. 4.1, we have:

$$p(x_t|z, x_0, \dots, x_{t-1}) = p(x_t|z). \quad (4.3)$$

Then we can rewrite Eqn. (4.2) as:

$$p(z|x_{0:t}) = \frac{p(x_t|z)p(z|x_{0:t-1})}{\int p(x_t|z)p(z|x_{0:t-1})dz} \quad (4.4)$$

$$\propto \underbrace{p(x_t|z)}_{\text{likelihood}} \underbrace{p(z|x_{0:t-1})}_{\text{prior}}. \quad (4.5)$$

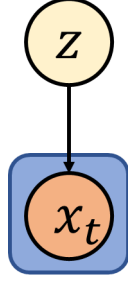


Figure 4.1: Graph of the true value z and the measurement x_t . Note that the measurement x_t is solely depend on z .

Thus, if we have *likelihood* and *prior*, then we can directly compute *posterior* by taking multiplication and integration in Eqn. (4.4). Here, the likelihood

$$p(x_t|z) \sim \mathcal{N}(z, \mathbf{R}) \quad (4.6)$$

denotes the measurement model in usual, and it can equivalently be represented as:

$$x_t = z + w, \quad \text{where } w \sim \mathcal{N}(\mathbf{0}, \mathbf{R}). \quad (4.7)$$

w denotes the measurement noise.

Note that the *prior* in Eqn. (4.5) can also be seen as the *posterior* in $(t-1)^{th}$ step:

$$p(z|x_{0:t-1}) \sim \mathcal{N}(\mu_{z|x_{t-1}}, \Sigma_{z|x_{t-1}}) \quad (4.8)$$

In other words, in Eqn. (4.5) we can use the previously computed *posterior* in $(t-1)^{th}$ time step as the *prior* in t^{th} time step. Therefore, if we obtain a new measurement (*likelihood*), then using the previous *posterior* (as the current *prior*) we can compute our desired *posterior*.

Consequently, assuming that we already know the posterior $p(z|x_{0:t-1})$ derived from the previous step, the target posterior $p(z|x_{0:t})$ can be directly computed from Eqn. (4.4). However, rather than computing multiplication and integration, it is way more easier to consider the joint PDF $p(z, x_t|x_{0:t-1})$ for finding the posterior $p(z|x_{0:t})$.

Consider the joint probability of two random variables z, x_t :

$$p(z, x_t|x_{0:t-1}) \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \quad (4.9)$$

where

$$\boldsymbol{\mu} = \begin{bmatrix} \mu_z \\ \mu_{x_t} \end{bmatrix}, \quad \boldsymbol{\Sigma} = \begin{bmatrix} \Sigma_{z,z} & \Sigma_{z,x_t} \\ \Sigma_{x_t,z} & \Sigma_{x_t,x_t} \end{bmatrix}. \quad (4.10)$$

The conditional PDF of this joint PDF is

$$p(z|x_t, x_{0:t-1}) = p(z|x_{0:t}), \quad (4.11)$$

which is the *posterior* that we need to find.

Recall that from the Conditional Gaussian PDF formula in Eqn. (2.34) and Eqn. (2.35). For the arbitrary random variables $\mathbf{Z}$ and $\mathbf{X}$ as an example, we have:

$$\mathbf{Z}|\mathbf{X} \sim \mathcal{N}(\mathbb{E}(\mathbf{Z}|\mathbf{X}), \mathbf{C}_{\mathbf{Z}|\mathbf{X}}) \quad (4.12)$$

where

$$\mathbb{E}(\mathbf{Z}|\mathbf{X}) = \mathbb{E}(\mathbf{Z}) + \mathbf{C}_{\mathbf{Z}\mathbf{X}}\mathbf{C}_{\mathbf{X}\mathbf{X}}^{-1}(\mathbf{X} - \mathbb{E}(\mathbf{X})) \quad (4.13)$$

$$\mathbf{C}_{\mathbf{Z}|\mathbf{X}} = \mathbf{C}_{\mathbf{Z}\mathbf{Z}} - \mathbf{C}_{\mathbf{Z}\mathbf{X}}\mathbf{C}_{\mathbf{X}\mathbf{X}}^{-1}\mathbf{C}_{\mathbf{X}\mathbf{Z}}. \quad (4.14)$$

In our case, $\mathbf{X} = x_t$ and $\mathbf{Z} = z$. Therefore, by following those equations, the conditional PDF (=posterior) $p(z|x_t, x_{0:t-1}) = p(z|x_{0:t}) \sim \mathcal{N}(\mu_{z|x_t}, \Sigma_{z|x_t})$ can be given as:

$$\mu_{z|x_t} = \mu_z + \Sigma_{z,x_t}\Sigma_{x_t,x_t}^{-1}(x_t - \mu_{x_t}) \quad (4.15)$$

$$\Sigma_{z|x_t} = \Sigma_{z,z} - \Sigma_{z,x_t}\Sigma_{x_t,x_t}^{-1}\Sigma_{x_t,z}. \quad (4.16)$$

Note that we do not need direct computation as Eqn. (4.4) no more, since we can obtain the *posterior* from Eqn. (4.15) and Eqn. (4.16), which can be computed by matrix operation.

Now, let us compute Eqn. (4.15) and Eqn. (4.16). Apparently, μ_z and $\Sigma_{z,z}$ are the parameters of the previous posterior $p(z|x_{0:t-1})$, we simply put:

$$\mu_z = \mu_{z|x_{t-1}} \quad (4.17)$$

$$\Sigma_{z,z} = \Sigma_{z|x_{t-1}}. \quad (4.18)$$

For the other parameters, we have:

$$\begin{aligned}\mu_{x_t} &= \mathbb{E}(x_t | x_{0:t-1}) \\ &= \mathbb{E}(z + w | x_{0:t-1}) \\ &= \mathbb{E}(z | x_{0:t-1}) = \mu_{z|x_{t-1}}\end{aligned}\tag{4.19}$$

$$\begin{aligned}\Sigma_{x_t, x_t} &= \mathbb{E}\left[(x_t - \mu_{x_t})^T (x_t - \mu_{x_t}) | x_{0:t-1}\right] \\ &= \mathbb{E}\left[(z + w - \mu_{x_t})^T (z + w - \mu_{x_t}) | x_{0:t-1}\right] \\ &= \mathbb{E}\left[\left(z + w - \mu_{z|x_{t-1}}\right)^T \left(z + w - \mu_{z|x_{t-1}}\right) | x_{0:t-1}\right] \\ &= \Sigma_{z|x_{t-1}} + \mathbf{R}\end{aligned}\tag{4.20}$$

$$\begin{aligned}\Sigma_{z, x_t} &= \mathbb{E}\left[(z - \mu_z)^T (x_t - \mu_{x_t}) | x_{0:t-1}\right] \\ &= \mathbb{E}\left[(z - \mu_z)^T (z + w - \mu_{x_t}) | x_{0:t-1}\right] \\ &= \mathbb{E}\left[\left(z - \mu_{z|x_{t-1}}\right)^T \left(z + w - \mu_{z|x_{t-1}}\right) | x_{0:t-1}\right] \\ &= \Sigma_{z|x_{t-1}}.\end{aligned}\tag{4.21}$$

Using these parameters, we can rewrite Eqn. (4.15) and Eqn. (4.16) for the updated posterior $p(z | x_{0:t})$ as:

$$\mu_{z|x_t} = \mu_{z|x_{t-1}} + \Sigma_{z|x_{t-1}}^T \left(\Sigma_{z|x_{t-1}} + \mathbf{R}\right)^{-1} (x_t - \mu_{z|x_{t-1}})\tag{4.22}$$

$$\Sigma_{z|x_t} = \Sigma_{z|x_{t-1}} - \Sigma_{z|x_{t-1}} \left(\Sigma_{z|x_{t-1}} + \mathbf{R}\right)^{-1} \Sigma_{z|x_{t-1}}.\tag{4.23}$$

Consequently, when the additional measurement x_t is added, we can update the previous posterior $p(z | x_{0:t-1})$ to $p(z | x_{0:t})$ simply by updating the parameters.

4.2 Markov Chain and Dynamic Model

Up until now, we have only considered scenarios where the robot is in a static state, without accounting for cases involving robot mobility or considering the dynamics of the robot. The basic essential of the mobile robot is that the robot moves around, and now it is time that we should handle the dynamics of the robot as well. Recall that the posterior for the measurement model is given as the following:

$$p(z | x_{0:t}) = \frac{p(x_t | z) p(z | x_{0:t-1})}{\int p(x_t | z) p(z | x_{0:t-1}) dz}.$$

Since we assume that the robot has no dynamics, apparently the robot's location z remains constant according to t . Now suppose the robot moves

around and measures its location at the same time. In this case the posterior for t 'th measurement can be rewritten as:

$$\begin{aligned} \underbrace{p(z_t|x_{0:t})}_{\text{posterior}} &= \frac{p(x_t|z_t)p(z_t|x_{0:t-1})}{\int p(x_t|z_t)p(z_t|x_{0:t-1})dz} \\ &= \frac{1}{Z} \underbrace{p(x_t|z_t)}_{\text{likelihood}} \underbrace{p(z_t|x_{0:t-1})}_{\text{prior}} \end{aligned}$$

where Z is a normalizing constant. Similar to the case of the robot without any dynamics, our goal is to estimate the t^{th} posterior $p(z_t|x_{0:t})$ from $t-1^{\text{th}}$ posterior $p(z_{t-1}|x_{0:t-1})$ and the measurement x_t . In order to achieve this, in the above equation first we need the likelihood

$$p(x_t|z_t) \sim \mathcal{N}(z_t, \mathbf{R}) \quad (4.24)$$

that can be rewritten as:

$$x_t = z_t + w \quad (4.25)$$

where $w \sim \mathcal{N}(0, \mathbf{R})$ is the measurement noise. Similarly, the prior is expressed as a Gaussian:

$$p(z_t|x_{0:t-1}) \sim \mathcal{N}(\mu_{z_t|x_{t-1}}, \Sigma_{z_t|x_{t-1}}). \quad (4.26)$$

Fortunately, the likelihood represents the measurement model depending on the sensor noise w , thereby we assume that it does not depend on the time step t . Compared to the case in previous chapter, the only difference is the prior $p(z_t|x_{0:t-1})$, which was previously known from the previous step. However, the only thing that we have now from the previous step is $t-1^{\text{th}}$ posterior

$$p(z_{t-1}|x_{0:t-1}) \sim \mathcal{N}(\mu_{z_{t-1}|x_{t-1}}, \Sigma_{z_{t-1}|x_{t-1}}), \quad (4.27)$$

which is different from $p(z_t|x_{0:t-1})$, because z_t is now time-varying. Therefore, we cannot directly use the previously computed *posterior* as the current *prior*. We need more step to figure out the prior $p(z_t|x_{0:t-1})$ from $t-1^{\text{th}}$ posterior $p(z_{t-1}|x_{0:t-1})$.

In order to find the relation between $p(z_t|x_{0:t-1})$ and $p(z_{t-1}|x_{0:t-1})$, we can rewrite $p(z_t|x_{0:t-1})$ as the following:

$$p(z_t|x_{0:t-1}) = \int p(z_t|z_{t-1}, x_{0:t-1}) \underbrace{p(z_{t-1}|x_{0:t-1})}_{\text{posterior from } t-1^{\text{th}} \text{ step}} dz_{t-1}. \quad (4.28)$$

Note that $p(z_{t-1}|x_{0:t-1})$ is the *posterior* computed from the previous step, thus we now can use our previously computed *posterior* here. Then how

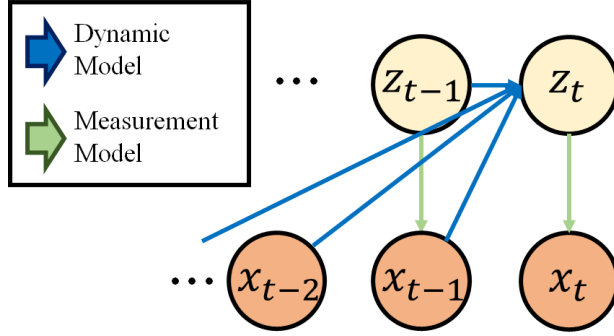


Figure 4.2: Graph of the true value z and the measurement x . Since the dynamic model involves the all measurements, the graph cannot be represented in simple structure.

about $p(z_t|z_{t-1}, x_{0:t-1})$?

Dynamic Model

In Eqn. (4.28), $p(z_t|z_{t-1}, x_{0:t-1})$ is called as a *dynamic model*. This PDF describes the location z_t at t^{th} time step according to the previous location z_{t-1} and the observations $x_{0:t-1}$. This relation is depicted in Fig. 4.2. Unfortunately, this relation is somewhat complex, thus it is challenging to derive the simple dynamic model. To simplify our dynamic model, we adopt *Markov assumption*. That is, the current state of robot is assumed to be determined only by its previous state. In other words, the current robot state is determined not by measurements, but by the previous robot state. The example of this Markov assumption is displayed in Fig. 4.3. Note that now the graph is represented as a chain, thus we call it as Markov Chain. Under this Markov assumption the dynamic model $p(z_t|z_{t-1}, x_{0:t-1})$ can be simplified to $p(z_t|z_{t-1})$, therefore we finally have:

$$p(z_t|x_{0:t-1}) = \int \underbrace{p(z_t|z_{t-1})}_{\text{dynamic model}} \underbrace{p(z_{t-1}|x_{0:t-1})}_{\text{posterior from } t-1^{th} \text{ step}} dz_{t-1}. \quad (4.29)$$

Assume the dynamic model $p(z_t|z_{t-1})$ follows Gaussian distribution, then the dynamic model can equivalently be represented as:

$$z_t = z_{t-1} + v \quad (4.30)$$

where $v \sim \mathcal{N}(0, Q)$ is the noise of the dynamic model. Let's refer to it as the dynamic noise. This noise represents the uncertainty of our dynamic model, which apparently have gap between the real-world scenario.

The dynamic model presented above is exceedingly simplistic, representing a scenario where the robot's location z_t remains unchanged at time t ,

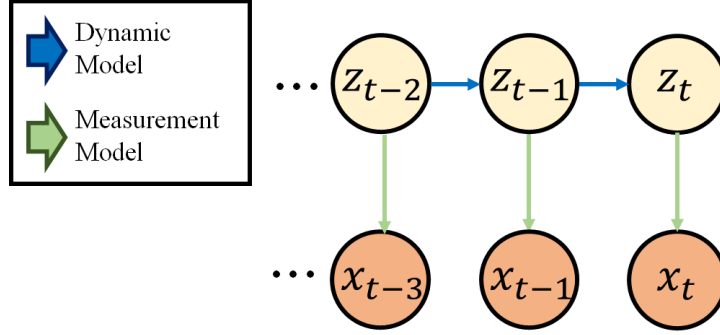


Figure 4.3: Markov chain of z and x . Since z_t is determined only by the previous robot state z_{t-1} , the dynamic model can be defined in simple form.

with only observation noise v varying over time. In other words, this dynamic model depicts a state where the robot is stationary. Now, let us explore how we can represent the dynamic model for a mobile robot with movement. Since a mobile robot moves based on the robot's control input u_{t-1} given at time $t-1$, the robot's dynamic model can be expressed as u_{t-1} . The control input for a robot can be provided through various means. For example, in a scenario where a person directly manipulates the robot, it can be defined as the manipulation input. In the case of a robot engaged in autonomous navigation, the control input is generated based on the robot's own path planning algorithm. For a mobile robot equipped with multiple sensors, in addition to sensors for environmental perception, the robot's movement can be inferred from measurements of the location obtained from robust sensors capable of recording the robot's trajectory. This type of measurement can also be utilized as odometry.

Odometry, in its literal sense, refers to a method of measuring the position of a moving object by recording the number of rotations through encoders and measuring the inclination using an device such as Inertial Measurement Unit (IMU). Incorporating this dynamic model u , the graph model of a mobile robot with a Markov chain can be expressed as follows. Let us designate both the control input and odometry as u from now on. Assume that at time $t-1$, the odometry u_{t-1} is given to the mobile robot. Then we can expect that within a very short period of time, this odometry remains approximately same during that time interval. That is, we can assume that the robot has a constant velocity model for its movement. To simplify the problem, let us assume that u_{t-1} is expressed in the same coordinate system as the one representing the robot's location. Then, for time t , our new dynamic model is expressed as follows:

$$z_t = z_{t-1} + u_{t-1} + v. \quad (4.31)$$

This dynamic model is also called as *prediction model*, since we expect the

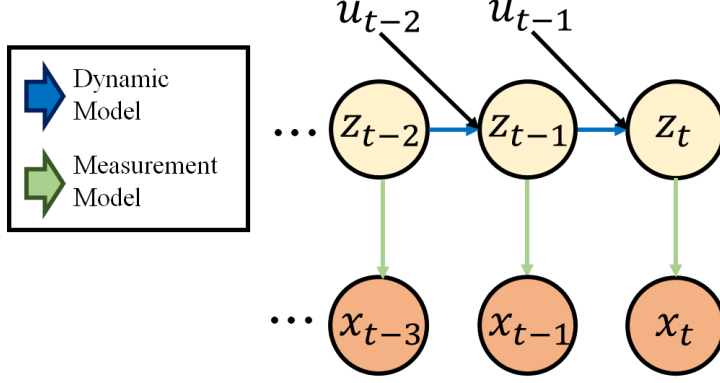


Figure 4.4: Markov chain involving the control input u . z_t is determined by the previous robot state z_{t-1} and the control input u_{t-1} .

robot location z_t is determined by the previous location z_{t-1} and our control input u_{t-1} . The graphical model including the control input u is displayed in Fig. 4.4.

Using the dynamic model, instead of computing the integral in Eqn. (4.29), now we can find prior $p(z_t|x_{0:t-1}) \sim \mathcal{N}(\mu_{z_t|x_{t-1}}, \Sigma_{z_t|x_{t-1}})$ by directly computing the parameters (mean and covariance) as follows:

$$\begin{aligned} \mu_{z_t|x_{t-1}} &= \mathbb{E}(z_t|x_{0:t-1}) \\ &= \mathbb{E}(z_{t-1} + u_{t-1} + v|x_{0:t-1}) \\ &= \mu_{z_{t-1}|x_{t-1}} + u_{t-1} \end{aligned} \quad (4.32)$$

$$\begin{aligned} \Sigma_{z_t|x_{t-1}} &= \mathbb{E}\left[(z_t - \mu_{z_t|x_{t-1}})(z_t - \mu_{z_t|x_{t-1}})^T | x_{0:t-1}\right] \\ &= \mathbb{E}\left[(z_{t-1} + v - \mu_{z_{t-1}|x_{t-1}})(z_{t-1} + v - \mu_{z_{t-1}|x_{t-1}})^T | x_{0:t-1}\right] \\ &= \mathbb{E}\left[(z_{t-1} - \mu_{z_{t-1}|x_{t-1}} + v)(z_{t-1} - \mu_{z_{t-1}|x_{t-1}} + v)^T | x_{0:t-1}\right] \\ &= \Sigma_{z_{t-1}|x_{t-1}} + \mathbf{Q}. \end{aligned} \quad (4.33)$$

Note that we do not compute Eqn. (4.29), but compute the parameters of PDF, since it is much easier. The rest part remains same. By using the prior and the likelihood $p(x_t|z_t)$, we can perform the measurement update and finally have:

$$\mu_{z_t|x_t} = \mu_{z_t|x_{t-1}} + \Sigma_{z_t|x_{t-1}}^T (\Sigma_{z_t|x_{t-1}} + \mathbf{R})^{-1} (x_t - \mu_{z_t|x_{t-1}}) \quad (4.34)$$

$$\Sigma_{z_t|x_t} = \Sigma_{z_t|x_{t-1}} - \Sigma_{z_t|x_{t-1}} (\Sigma_{z_t|x_{t-1}} + \mathbf{R})^{-1} \Sigma_{z_t|x_{t-1}}. \quad (4.35)$$

Consequently, the sequential measurement with the dynamic model can be summarized as follows. For the robot location z_t , assume we have observations $x_{0:t}$ and the odometry $u_{0:t-1}$. our goal is to achieve the posterior of t^{th} step:

$$\begin{aligned} \underbrace{p(z_t|x_{0:t}, u_{0:t-1})}_{\text{posterior}} &= \frac{p(x_t|z_t, u_{0:t-1}) p(z_t|x_{0:t-1}, u_{0:t-1})}{\int p(x_N|z_N, u_{0:t-1}) p(z_t|x_{0:t-1}, u_{0:t-1}) dz} \\ &= \frac{1}{Z} p(x_t|z_t, u_{0:t-1}) p(z_t|x_{0:t-1}, u_{0:t-1}) \\ &= \frac{1}{Z} \underbrace{p(x_t|z_t)}_{\text{likelihood (measurement model)}} \underbrace{p(z_t|x_{0:t-1}, u_{0:t-1})}_{\text{prior}}. \end{aligned}$$

where Z is a normalizing constant. Note that the relationships between each variable can be easily derived from the graph model. For the prior, we have the following step:

$$\underbrace{p(z_t|x_{0:t-1}, u_{0:t-1})}_{\text{prior}} = \int \underbrace{p(z_t|z_{t-1}, u_{t-1})}_{\text{dynamic model}} \underbrace{p(z_{t-1}|x_{0:t-1}, u_{0:t-2})}_{\text{posterior from } t-1} dz_{t-1}.$$

For each model we have:

- Dynamic model:

$$z_t = z_{t-1} + u_{t-1} + v$$

- Measurement model:

$$x_t = z_t + w$$

where $v \sim \mathcal{N}(0, \mathbf{Q})$ and $w \sim \mathcal{N}(0, \mathbf{R})$ are the dynamic noise and the measurement noise. From the above models, we have

- Dynamic update:

$$\begin{aligned} \mu_{z_t|x_{t-1}} &= \mu_{z_{t-1}|x_{t-1}} + u_{t-1} \\ \Sigma_{z_t|x_{t-1}} &= \Sigma_{z_{t-1}|x_{t-1}} + \mathbf{Q} \end{aligned}$$

- Measurement update:

$$\begin{aligned} \mu_{z_t|x_t} &= \mu_{z_t|x_{t-1}} + K_t (x_t - \mu_{z_t|x_{t-1}}) \\ \Sigma_{z_t|x_t} &= \Sigma_{z_t|x_{t-1}} - K_t \Sigma_{z_t|x_{t-1}} \end{aligned}$$

where

$$K_t = \Sigma_{z_t|x_{t-1}} (\Sigma_{z_t|x_{t-1}} + \mathbf{R})^{-1}.$$

Since we already know that the optimal solution of Bmse is equivalent to the mean value of the posterior, $\mu_{z_t|x_{t-1}}$ is directly utilized as the optimal robot location. By computing other parameters, we can sequentially estimate the optimal solution of the robot location while the robot collects measurement values.

4.3 Bayesian Linear Filter

Until now, we have assumed a simple dynamic model and measurement model. For instance, we assumed that the odometry u could be easily represented in the robot's coordinates. Unfortunately, in real-world scenarios, models are nonlinear and extremely complex. For example, let's consider that the robot's location z we aim to estimate is represented in a 3D Cartesian coordinate system. While it would be ideal for the robot's odometry u to also be expressed in Cartesian coordinates, in many cases, the robot's odometry is unfortunately represented in terms of linear velocity and angular acceleration. To represent this in Cartesian coordinates, nonlinear functions, such as trigonometric functions, need to be employed. Similarly, in the case of the measurement model, the sensor observations x for the robot's location z are often not expressed in the same Cartesian coordinates. Instead, they may be represented in terms of distance and angle. As a result, the relationships between various variables are no longer expressed as linear relationships but instead take on nonlinear forms.

Suppose the robot follows a complex dynamic model which can be represented by the function g . Similarly, assume the sensor model is given as the function h . With the Gaussian noise $v \sim \mathcal{N}(0, \mathbf{Q})$ and $w \sim \mathcal{N}(0, \mathbf{R})$, for the robot location z and the measurement x we have:

- Dynamic model:

$$z_t = g(z_{t-1}, u_{t-1}) + \mathbf{G}v$$

- Measurement model:

$$x_t = h(z_t) + w$$

where $v \sim \mathcal{N}(0, \mathbf{Q})$ and $w \sim \mathcal{N}(0, \mathbf{R})$ are the dynamic noise and the measurement noise.

These are equivalently represented as:

- Dynamic model:

$$p(z_t|x_{t-1}, u_{t-1}) = \mathcal{N}(g(z_{t-1}, u_{t-1}), \mathbf{G}\mathbf{Q}\mathbf{G}^T)$$

- Measurement model:

$$p(x_t|z_t) = \mathcal{N}(h(z_t), \mathbf{R}).$$

Unlike the previous simple example, in the Dynamic model even the random variable z_{t-1} is Gaussian we cannot tell z_t is also a Gaussian random variable, because g is nonlinear. Using the same logic, in the measurement model we cannot conclusively assert that x_t is also Gaussian, even if z_t is a Gaussian random variable.

To simplify our discussion, assume that we have the approximated *linear* models represented as follows:

- Dynamic model:

$$\begin{aligned} z_t &= g(z_{t-1}, u_{t-1}) + \mathbf{G}v \\ &\simeq \mathbf{A}z_{t-1} + \mathbf{B}u_{t-1} + \mathbf{G}v \end{aligned}$$

- Measurement model:

$$\begin{aligned} x_t &= h(z_t) + w \\ &\simeq \mathbf{C}z_t + w \end{aligned}$$

First, for the dynamic update, instead of calculating the integral in Eqn. (4.29), we can find the parameters (mean and covariance) of prior $p(z_t|x_{0:t-1}, u_{0:t-1}) \sim \mathcal{N}(\mu_{z_t|x_{t-1}}, \Sigma_{z_t|x_{t-1}})$ as follows:

$$\begin{aligned} \mu_{z_t|x_{t-1}} &= \mathbb{E}(z_t|x_{0:t-1}, u_{0:t-1}) \\ &= \mathbb{E}(\mathbf{A}z_{t-1} + \mathbf{B}u_{t-1} + \mathbf{G}v|x_{0:t-1}, u_{0:t-1}) \\ &= \mathbf{A}\mu_{z_{t-1}|x_{t-1}} + \mathbf{B}u_{t-1} \end{aligned} \tag{4.36}$$

$$\begin{aligned} \Sigma_{z_t|x_{t-1}} &= \mathbb{E}\left[\left(z_t - \mu_{z_t|x_{t-1}}\right)\left(z_t - \mu_{z_t|x_{t-1}}\right)^T | x_{0:t-1}, u_{0:t-1}\right] \\ &= \mathbb{E}\left[\left(\mathbf{A}z_{t-1} + \mathbf{G}v - \mathbf{A}\mu_{z_{t-1}|x_{t-1}}\right)\left(\mathbf{A}z_{t-1} + \mathbf{G}v - \mathbf{A}\mu_{z_{t-1}|x_{t-1}}\right)^T | \cdot\right] \\ &= \mathbb{E}\left[\left(\mathbf{A}z_{t-1} - \mathbf{A}\mu_{z_{t-1}|x_{t-1}} + \mathbf{G}v\right)\left(\mathbf{A}z_{t-1} - \mathbf{A}\mu_{z_{t-1}|x_{t-1}} + \mathbf{G}v\right)^T | \cdot\right] \\ &= \mathbf{A}\Sigma_{z_{t-1}|x_{t-1}}\mathbf{A}^T + \mathbf{G}\mathbf{Q}\mathbf{G}^T. \end{aligned} \tag{4.37}$$

For the measurement update, we compute the mean and covariance of the posterior $p(z_t|x_{0:t}, u_{0:t-1}) \sim \mathcal{N}(\mu_{z_t|x_t}, \Sigma_{z_t|x_t})$ by using the computed prior $p(z_t|x_{0:t-1}, u_{0:t-1})$ as follows:

$$\begin{aligned} \mu_{z_t|x_t} &= \mu_{z_t|x_{t-1}} + \Sigma_{z_t,x_t}\Sigma_{x_t,x_t}^{-1}(x_t - \mu_{x_t}) \\ \Sigma_{z_t|x_t} &= \Sigma_{z_t|x_{t-1}} - \Sigma_{z_t,x_t}\Sigma_{x_t,x_t}^{-1}\Sigma_{x_t,z_t}. \end{aligned} \tag{4.38}$$

Here, the parameters can be computed as follows:

$$\begin{aligned}
\mu_{x_t} &= \mathbb{E}(x_t | x_{0:t-1}) \\
&= \mathbb{E}(Cz_t + w | x_{0:t-1}) \\
&= \mathbb{E}(Cz_t | x_{0:t-1}) = C\mu_{z_t|x_{t-1}}
\end{aligned} \tag{4.39}$$

$$\begin{aligned}
\Sigma_{x_t, x_t} &= \mathbb{E}[(x_t - \mu_{x_t})(x_t - \mu_{x_t})^T | x_{0:t-1}] \\
&= \mathbb{E}\left[\left(Cz_t + w - C\mu_{z_t|x_{t-1}}\right)\left(Cz_t + w - C\mu_{z_t|x_{t-1}}\right)^T | x_{0:t-1}\right] \\
&= C\Sigma_{z_t|x_{t-1}}C^T + R
\end{aligned} \tag{4.40}$$

$$\begin{aligned}
\Sigma_{z_t, x_t} &= \mathbb{E}\left[\left(z_t - \mu_{z_t|x_{t-1}}\right)(x_t - \mu_{x_t})^T | x_{0:t-1}\right] \\
&= \mathbb{E}\left[\left(z_t - \mu_{z_t|x_{t-1}}\right)\left(Cz_t + w - C\mu_{z_t|x_{t-1}}\right)^T | x_{0:t-1}\right] \\
&= \Sigma_{z_t|x_{t-1}}C^T.
\end{aligned} \tag{4.41}$$

In summary, for the linear models the update process can be represented as follows:

- Dynamic update:

$$\begin{aligned}
\mu_{z_t|x_{t-1}} &= A\mu_{z_{t-1}|x_{t-1}} + Bu_{t-1} \\
\Sigma_{z_t|x_{t-1}} &= A\Sigma_{z_{t-1}|x_{t-1}}A^T + GQG^T
\end{aligned}$$

- Measurement update:

$$\begin{aligned}
\mu_{z_t|x_t} &= \mu_{z_t|x_{t-1}} + K_t(x_t - C\mu_{z_t|x_{t-1}}) \\
\Sigma_{z_t|x_t} &= \Sigma_{z_t|x_{t-1}} - K_tC\Sigma_{z_t|x_{t-1}}
\end{aligned}$$

where

$$K_t = \Sigma_{z_t|x_{t-1}}C^T(C\Sigma_{z_t|x_{t-1}}C^T + R)^{-1}.$$

The algorithm of this linear Bayesian Filter is shown in Algorithm. 1. In summary, to derive filters for dynamic and measurement models that actually follow nonlinear systems, we made several assumptions. Firstly, we assumed linear models, and secondly, we assumed Gaussian noise. Based on these two robust assumptions, the filtering technique is known as the Kalman Filter, a type of Bayesian linear filter.

Even though this filter assumes linear models, it proves useful for modeling simple robot dynamics. Let's explore how we can represent various types of dynamic models as linear models through the following examples.

Algorithm 1 Linear Kalman Filter

Require: $x_t, u_{t-1}, \mu_{z_{t-1}|x_{t-1}}, \Sigma_{z_{t-1}|x_{t-1}}$ *[Dynamic Update]*

1: $\mu_{z_t|x_{t-1}} = \mathbf{A}\mu_{z_{t-1}|x_{t-1}} + \mathbf{B}u_{t-1}$

2: $\Sigma_{z_t|x_{t-1}} = \mathbf{A}\Sigma_{z_{t-1}|x_{t-1}}\mathbf{A}^T + \mathbf{G}\mathbf{Q}\mathbf{G}^T$

[Measurement Update]

3: $\mathbf{K}_t = \Sigma_{z_t|x_{t-1}}\mathbf{C}^T (\mathbf{C}\Sigma_{z_t|x_{t-1}}\mathbf{C}^T + \mathbf{R})^{-1}$

4: $\mu_{z_t|x_t} = \mu_{z_t|x_{t-1}} + \mathbf{K}_t (x_t - \mathbf{C}\mu_{z_t|x_{t-1}})$

5: $\Sigma_{z_t|x_t} = \Sigma_{z_t|x_{t-1}} - \mathbf{K}_t\mathbf{C}\Sigma_{z_t|x_{t-1}}$

6: **return** $\mu_{z_t|x_t}, \Sigma_{z_t|x_t}$

Example: Constant velocity model

Let's consider a scenario where a mobile robot is navigating a 2-dimensional plane. In this case, let's denote the robot's state at time t as z_t . Up until now, we've referred to only the robot's location $[z^1, z^2]^T$ as the state. However, in reality, there are no constraints on what constitutes the state. Therefore, this time, let's include not only the robot's location but also its velocity $[\dot{z}^1, \dot{z}^2]^T$ in the state. This can be expressed as follows:

$$z_t = [z_t^1, \dot{z}_t^1, z_t^2, \dot{z}_t^2]^T.$$

Here, for the dynamic model of the robot, we can make a very simple assumption. Although the robot's velocity $[\dot{z}^1, \dot{z}^2]^T$ may vary with time t , for very short time intervals Δt , we can assume constant velocity motion. Then, for the robot's location $[z^1, z^2]^T$, it can be expressed as follows:

$$\begin{bmatrix} z_t^1 \\ z_t^2 \end{bmatrix} = \begin{bmatrix} z_{t-1}^1 \\ z_{t-1}^2 \end{bmatrix} + \begin{bmatrix} \dot{z}_{t-1}^1 \\ \dot{z}_{t-1}^2 \end{bmatrix} \Delta t.$$

Since we assume the constant velocity model, with the model noise w we have:

$$\begin{bmatrix} \dot{z}_t^1 \\ \dot{z}_t^2 \end{bmatrix} = \begin{bmatrix} \dot{z}_{t-1}^1 \\ \dot{z}_{t-1}^2 \end{bmatrix} + \begin{bmatrix} w^1 \\ w^2 \end{bmatrix}.$$

The dynamic model presented above can be expressed collectively as follows:

$$z_t = \mathbf{A}z_{t-1} + \mathbf{G}w$$

where

$$\mathbf{A} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{G} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Assume that we obtain the measurement $x = [x^1 x^2]^T$ for the robot location $[z^1, z^2]^T$, and we can write the measurement model as:

$$x_t = \mathbf{C}z_t + v$$

where v is a measurement noise and

$$\mathbf{C} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

5

Nonlinear Filter

In the realm of robotics and SLAM, the role of nonlinear filters is paramount in overcoming the inherent challenges posed by dynamic and uncertain environments. These filters, such as the Extended Kalman Filter (EKF), Particle Filter (PF), and Unscented Kalman Filter (UKF), take center stage in the robotic landscape, each contributing uniquely to the complex interplay of localization, mapping, and dynamic state estimation.

Within the context of robotics, the Extended Kalman Filter (EKF) serves as a stalwart ally. In the dynamic world of robotic motion, the EKF facilitates accurate state estimation by incorporating nonlinearities. Its iterative linearization process allows robots to navigate through challenging terrains, maintaining precise localization even amidst the unpredictable nature of real-world scenarios.

Shifting our focus to the Particle Filter (PF), its role in robotics becomes pronounced, especially in the arena of SLAM. Here, robots are not merely navigating through space; they are actively mapping unknown environments. The PF, with its ability to adaptively represent belief states through discrete particles, proves invaluable. In SLAM applications, where uncertainties abound, the PF dynamically adjusts its representation, ensuring robust performance in complex, non-Gaussian environments.

In the ever-evolving landscape of robotics and SLAM, the Unscented Kalman Filter (UKF) emerges as a crucial player. Avoiding the pitfalls of linearization, the UKF excels in capturing the intricacies of nonlinear systems. In the context of SLAM, where uncertainties stem not only from robot motion but also from mapping unknown features, the UKF provides a reliable and efficient solution, contributing to enhanced localization and mapping accuracy.

In essence, nonlinear filters are not mere algorithms; they are indispensable tools that empower robotic systems to navigate and interact with the world intelligently. By addressing the challenges of nonlinearity and uncertainty, these filters elevate the capabilities of robots, paving the way for

advancements in autonomous navigation, mapping, and exploration within dynamic and complex environments. In this chapter, we discuss the two famous nonlinear filters: EKF and PF as the main examples.

5.1 Extended Kalman Filter

Recall that the goal of our Bayesian estimation is to find the mean of the posterior. The posterior is given by the measurements and the prior as the following:

$$\begin{aligned}
 \underbrace{p(z_t|x_{0:t}, u_{0:t-1})}_{\text{posterior of } t^{\text{th}} \text{ step}} &= \frac{p(x_t|z_t, u_{0:t-1}) p(z_t|x_{0:t-1}, u_{0:t-1})}{\int p(x_t|z_t, u_{0:t-1}) p(z_t|x_{0:t-1}, u_{0:t-1}) dz_t} \quad (5.1) \\
 &= \underbrace{\frac{1}{Z}}_{\text{const}} p(x_t|z_t, u_{0:t-1}) p(z_t|x_{0:t-1}, u_{0:t-1}) \\
 &\propto p(x_t|z_t, u_{0:t-1}) p(z_t|x_{0:t-1}, u_{0:t-1}) \\
 &= \underbrace{p(x_t|z_t)}_{\text{likelihood (measurement model)}} \underbrace{p(z_t|x_{0:t-1}, u_{0:t-1})}_{\text{prior}}.
 \end{aligned}$$

Here, the prior is given as the following:

$$\underbrace{p(z_t|x_{0:t-1}, u_{0:t-1})}_{\text{prior}} = \int \underbrace{p(z_t|z_{t-1}, u_{t-1})}_{\text{dynamic model}} \underbrace{p(z_{t-1}|x_{0:t-1}, u_{0:t-2})}_{\text{posterior from } t-1} dz_{t-1}. \quad (5.2)$$

To compute the prior and the posterior of t^{th} step sequentially, we first assume the following two models:

- Dynamic model:

$$z_t = g(z_{t-1}, u_{t-1}) + \mathbf{G}v$$

- Measurement model:

$$x_t = h(z_t) + w$$

where $v \sim \mathcal{N}(0, \mathbf{Q})$ and $w \sim \mathcal{N}(0, \mathbf{R})$ are the dynamic noise and the measurement noise. In the Kalman filter, we simplified the problem by assuming that the functions g and h are linear functions. However, if the robot system is nonlinear, such assumptions will lead to suboptimal estimation results.

So, how do we address nonlinearity? The key challenge in this problem is that when the system is nonlinear, meaning g and h are nonlinear functions, we cannot leverage the powerful advantage that Gaussian random variables remain Gaussian when combined linearly.

This means that we can no longer assert that the likelihood and prior PDFs needed to compute the posterior follow Gaussian distributions. Consequently, the posterior, represented as the product of these two PDFs, cannot

be considered Gaussian. Therefore, the only way to retain the advantages of Gaussian random variables is to approximate the nonlinear functions linearly.

To approximate nonlinear functions linearly, we can apply Taylor expansion. When using up to the second-order terms in Taylor expansion, the nonlinear functions representing the dynamic model, g , and the measurement model, h , can be approximated as follows:

$$g(z_{t-1}, u_{t-1}) \approx g(\mu_{t-1}, u_{t-1}) + \frac{\partial g(\mu_{t-1}, u_{t-1})}{\partial z_{t-1}} (z_{t-1} - \mu_{t-1}) \quad (5.3)$$

$$h(z_t) \approx h(\tilde{\mu}_t) + \frac{\partial h(\tilde{\mu}_t)}{\partial z_t} (z_t - \tilde{\mu}_t). \quad (5.4)$$

Intuitively, the state of the robot at time t is determined by applying the robot's dynamic model to the state at time $t-1$. Therefore, we set $\mu_{t-1} = \mu_{z_{t-1}|x_{t-1}}$. Similarly, the measurement model for the robot's state will be obtained from the state influenced by the robot's dynamic model. Therefore, we define $\tilde{\mu}_t = \mu_{z_t|x_{t-1}}$. Since the derivatives of the nonlinear functions g and h are expressed as Jacobian matrices $\mathcal{G}$ and $\mathcal{H}$, respectively, we can approximate g and h as follows:

$$g(z_{t-1}, u_{t-1}) \approx g(\mu_{z_{t-1}|x_{t-1}}, u_{t-1}) + \mathcal{G}_t (z_{t-1} - \mu_{z_{t-1}|x_{t-1}}), \quad (5.5)$$

$$h(z_t) \approx h(\mu_{z_t|x_{t-1}}) + \mathcal{H}_t (z_t - \mu_{z_t|x_{t-1}}), \quad (5.6)$$

where

$$\mathcal{G}_t = \left. \frac{\partial g(z_{t-1}, u_{t-1})}{\partial z_{t-1}} \right|_{z_{t-1} = \mu_{z_{t-1}|x_{t-1}}}$$

$$\mathcal{H}_t = \left. \frac{\partial h(z_t)}{\partial z_t} \right|_{z_t = \mu_{z_t|x_{t-1}}}$$

Using the approximated model, we have:

- Dynamic model:

$$z_t \approx g(\mu_{z_{t-1}|x_{t-1}}, u_{t-1}) + \mathcal{G}_t (z_{t-1} - \mu_{z_{t-1}|x_{t-1}}) + \mathbf{G}v$$

- Measurement model:

$$x_t \approx h(\mu_{z_t|x_{t-1}}) + \mathcal{H}_t (z_t - \mu_{z_t|x_{t-1}}) + w.$$

Following this, the derivation process for obtaining the posterior is the same as the derivation process for the Kalman filter for a linear system. First, for the dynamic update, we find the parameters of prior $p(z_t|x_{0:t-1}, u_{0:t-1}) \sim$

$\mathcal{N}(\mu_{z_t|x_{t-1}}, \Sigma_{z_t|x_{t-1}})$ as follows:

$$\begin{aligned}
\mu_{z_t|x_{t-1}} &= \mathbb{E}(z_t|x_{0:t-1}, u_{0:t-1}) \\
&\approx \mathbb{E}\left(g(\mu_{z_{t-1}|x_{t-1}}, u_{t-1}) + \mathcal{G}_t(z_{t-1} - \mu_{z_{t-1}|x_{t-1}}) + \mathbf{G}v|x_{0:t-1}, u_{0:t-1}\right) \\
&= g(\mu_{z_{t-1}|x_{t-1}}, u_{t-1}) + \mathcal{G}_t(\cancel{\mu_{z_{t-1}|x_{t-1}}} - \cancel{\mu_{z_{t-1}|x_{t-1}}}) \\
&= g(\mu_{z_{t-1}|x_{t-1}}, u_{t-1})
\end{aligned} \tag{5.7}$$

$$\begin{aligned}
\Sigma_{z_t|x_{t-1}} &= \mathbb{E}\left[\left(z_t - \mu_{z_t|x_{t-1}}\right)\left(z_t - \mu_{z_t|x_{t-1}}\right)^T | x_{0:t-1}, u_{0:t-1}\right] \\
&\approx \mathbb{E}\left[\left(g(\cancel{\mu_{z_{t-1}|x_{t-1}}}, u_{t-1}) + \mathcal{G}_t(z_{t-1} - \mu_{z_{t-1}|x_{t-1}}) + \mathbf{G}v - g(\cancel{\mu_{z_{t-1}|x_{t-1}}}, u_{t-1})\right)(\cdot)^T | \cdot\right] \\
&= \mathbb{E}\left[\left(\mathcal{G}_t(z_{t-1} - \mu_{z_{t-1}|x_{t-1}}) + \mathbf{G}v\right)\left(\mathcal{G}_t(z_{t-1} - \mu_{z_{t-1}|x_{t-1}}) + \mathbf{G}v\right)^T | \cdot\right] \\
&= \mathcal{G}_t \Sigma_{z_{t-1}|x_{t-1}} \mathcal{G}_t^T + \mathbf{G} \mathbf{Q} \mathbf{G}^T.
\end{aligned} \tag{5.8}$$

For the measurement update, we compute the mean and covariance of the posterior $p(z_t|x_{0:t}, u_{0:t-1}) \sim \mathcal{N}(\mu_{z_t|x_t}, \Sigma_{z_t|x_t})$ by using the computed prior $p(z_t|x_{0:t-1}, u_{0:t-1})$ as follows:

$$\begin{aligned}
\mu_{z_t|x_t} &= \mu_{z_t|x_{t-1}} + \Sigma_{z_t, x_t} \Sigma_{x_t, x_t}^{-1} (x_t - \mu_{x_t}) \\
\Sigma_{z_t|x_t} &= \Sigma_{z_t|x_{t-1}} - \Sigma_{z_t, x_t} \Sigma_{x_t, x_t}^{-1} \Sigma_{x_t, z_t},
\end{aligned} \tag{5.9}$$

where

$$\begin{aligned}
\mu_{x_t} &= \mathbb{E}(x_t|x_{0:t-1}) \\
&\approx \mathbb{E}\left(h(\mu_{z_t|x_{t-1}}) + \mathcal{H}_t(z_t - \mu_{z_t|x_{t-1}}) + w\right) \\
&= h(\mu_{z_t|x_{t-1}}) + \mathcal{H}_t(\cancel{\mu_{z_t|x_{t-1}}} - \cancel{\mu_{z_t|x_{t-1}}}) = h(\mu_{z_t|x_{t-1}})
\end{aligned} \tag{5.10}$$

$$\begin{aligned}
\Sigma_{x_t, x_t} &= \mathbb{E}\left[(x_t - \mu_{x_t})(x_t - \mu_{x_t})^T | x_{0:t-1}\right] \\
&\approx \mathbb{E}\left[\left(h(\cancel{\mu_{z_t|x_{t-1}}}) + \mathcal{H}_t(z_t - \mu_{z_t|x_{t-1}}) + w - h(\cancel{\mu_{z_t|x_{t-1}}})\right)(\cdot)^T | \cdot\right] \\
&= \mathbb{E}\left[\left(\mathcal{H}_t(z_t - \mu_{z_t|x_{t-1}}) + w\right)\left(\mathcal{H}_t(z_t - \mu_{z_t|x_{t-1}}) + w\right)^T | x_{0:t-1}\right] \\
&= \mathcal{H}_t \Sigma_{z_t|x_{t-1}} \mathcal{H}_t^T + \mathbf{R}
\end{aligned} \tag{5.11}$$

$$\begin{aligned}
\Sigma_{z_t, x_t} &= \mathbb{E}\left[(z_t - \mu_{z_t|x_{t-1}})(x_t - \mu_{x_t})^T | x_{0:t-1}\right] \\
&\approx \mathbb{E}\left[\left(z_t - \mu_{z_t|x_{t-1}}\right)\left(h(\cancel{\mu_{z_t|x_{t-1}}}) + \mathcal{H}_t(z_t - \mu_{z_t|x_{t-1}}) + w - h(\cancel{\mu_{z_t|x_{t-1}}})\right)^T | \cdot\right] \\
&= \mathbb{E}\left[\left(z_t - \mu_{z_t|x_{t-1}}\right)\left(\mathcal{H}_t(z_t - \mu_{z_t|x_{t-1}}) + w\right)^T | x_{0:t-1}\right] \\
&= \Sigma_{z_t|x_{t-1}} \mathcal{H}_t^T.
\end{aligned} \tag{5.12}$$

In summary, for the EKF filter the update process can be represented as follows:

- Dynamic update:

$$\begin{aligned}\mu_{z_t|x_{t-1}} &= g\left(\mu_{z_{t-1}|x_{t-1}}, u_{t-1}\right) \\ \Sigma_{z_t|x_{t-1}} &= \mathbf{G}_t \Sigma_{z_{t-1}|x_{t-1}} \mathbf{G}_t^T + \mathbf{G} \mathbf{Q} \mathbf{G}^T\end{aligned}$$

- Measurement update:

$$\begin{aligned}\mu_{z_t|x_t} &= \mu_{z_t|x_{t-1}} + K_t \left(x_t - h\left(\mu_{z_t|x_{t-1}}\right) \right) \\ \Sigma_{z_t|x_t} &= \Sigma_{z_t|x_{t-1}} - K_t \mathbf{H}_t \Sigma_{z_t|x_{t-1}}\end{aligned}$$

where

$$K_t = \Sigma_{z_t|x_{t-1}} \mathbf{H}_t^T \left(\mathbf{H}_t \Sigma_{z_t|x_{t-1}} \mathbf{H}_t^T + \mathbf{R} \right)^{-1}.$$

Let us examine how the EKF is practically utilized for robot location esti-

Algorithm 2 Extended Kalman Filter

Require: $x_t, u_{t-1}, \mu_{z_{t-1}|x_{t-1}}, \Sigma_{z_{t-1}|x_{t-1}}$

[*Dynamic Update*]

- 1: $\mu_{z_t|x_{t-1}} = g\left(\mu_{z_{t-1}|x_{t-1}}, u_{t-1}\right)$
- 2: $\Sigma_{z_t|x_{t-1}} = \mathbf{G}_t \Sigma_{z_{t-1}|x_{t-1}} \mathbf{G}_t^T + \mathbf{G} \mathbf{Q} \mathbf{G}^T$

[*Measurement Update*]

- 3: $K_t = \Sigma_{z_t|x_{t-1}} \mathbf{H}_t^T \left(\mathbf{H}_t \Sigma_{z_t|x_{t-1}} \mathbf{H}_t^T + \mathbf{R} \right)^{-1}$
 - 4: $\mu_{z_t|x_t} = \mu_{z_t|x_{t-1}} + K_t \left(x_t - h\left(\mu_{z_t|x_{t-1}}\right) \right)$
 - 5: $\Sigma_{z_t|x_t} = \Sigma_{z_t|x_{t-1}} - K_t \mathbf{H}_t \Sigma_{z_t|x_{t-1}}$
 - 6: **return** $\mu_{z_t|x_t}, \Sigma_{z_t|x_t}$
-

mation through the following example.

Example I: Constant velocity model

Suppose that the mobile robot explores on the 2D plane in the constant linear and angular velocity. At time t , The robot state z_t is represented as:

$$z = \left[z_t^x, z_t^y, z_t^\theta, z_t^v \right]^T \quad (5.13)$$

where $[z_t^x, z_t^y, z_t^\theta]^T$ denotes the robot pose, and z^v represents the linear velocity along the robot trajectory. Assume that the robot location is directly measured as $x_t = [x_t^x, x_t^y]$. In this case, by assuming the constant velocity model during Δt , the dynamic model is given as:

$$\begin{aligned} \begin{bmatrix} z_t^x \\ z_t^y \\ z_t^\theta \\ z_t^v \end{bmatrix} &= \underbrace{\begin{bmatrix} z_{t-1}^x \\ z_{t-1}^y \\ z_{t-1}^\theta \\ z_{t-1}^v \end{bmatrix} + \begin{bmatrix} z^v \cos(z_{t-1}^\theta) \Delta t \\ z^v \sin(z_{t-1}^\theta) \Delta t \\ 0 \\ 0 \end{bmatrix}}_g + \begin{bmatrix} 0 \\ 0 \\ v^\theta \\ v^v \end{bmatrix} \\ &= \underbrace{\begin{bmatrix} z_{t-1}^x \\ z_{t-1}^y \\ z_{t-1}^\theta \\ z_{t-1}^v \end{bmatrix} + \begin{bmatrix} z^v \cos(z_{t-1}^\theta) \Delta t \\ z^v \sin(z_{t-1}^\theta) \Delta t \\ 0 \\ 0 \end{bmatrix}}_g + \mathbf{G} \underbrace{\begin{bmatrix} v^\theta \\ v^v \end{bmatrix}}_{\text{model noise}} \end{aligned}$$

where

$$\mathbf{G} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

and

$$v = \begin{bmatrix} v^\theta \\ v^v \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}). \quad (5.14)$$

To approximate the nonlinear dynamic model, we need to compute the Jacobian $\mathbf{G}$ as the following:

$$\begin{aligned} \mathbf{G} &= \begin{bmatrix} \frac{\partial z_t^x}{\partial z_{t-1}^x} & \frac{\partial z_t^x}{\partial z_{t-1}^y} & \frac{\partial z_t^x}{\partial z_{t-1}^\theta} & \frac{\partial z_t^x}{\partial z_{t-1}^v} \\ \frac{\partial z_t^y}{\partial z_{t-1}^x} & \frac{\partial z_t^y}{\partial z_{t-1}^y} & \frac{\partial z_t^y}{\partial z_{t-1}^\theta} & \frac{\partial z_t^y}{\partial z_{t-1}^v} \\ \frac{\partial z_t^\theta}{\partial z_{t-1}^x} & \frac{\partial z_t^\theta}{\partial z_{t-1}^y} & \frac{\partial z_t^\theta}{\partial z_{t-1}^\theta} & \frac{\partial z_t^\theta}{\partial z_{t-1}^v} \\ \frac{\partial z_t^v}{\partial z_{t-1}^x} & \frac{\partial z_t^v}{\partial z_{t-1}^y} & \frac{\partial z_t^v}{\partial z_{t-1}^\theta} & \frac{\partial z_t^v}{\partial z_{t-1}^v} \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & -z^v \sin(z_{t-1}^\theta) \Delta t & \cos(z_{t-1}^\theta) \Delta t \\ 0 & 1 & z^v \cos(z_{t-1}^\theta) \Delta t & \sin(z_{t-1}^\theta) \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \end{aligned}$$

Using $\mu_{z_{t-1}|x_{t-1}}$ from the $t-1^{th}$ step, we have:

$$\mathbf{g}_t = \begin{bmatrix} 1 & 0 & -z^v \sin\left(\mu_{z_{t-1}|x_{t-1}}^\theta\right) \Delta t & \cos\left(\mu_{z_{t-1}|x_{t-1}}^\theta\right) \Delta t \\ 0 & 1 & z^v \cos\left(\mu_{z_{t-1}|x_{t-1}}^\theta\right) \Delta t & \sin\left(\mu_{z_{t-1}|x_{t-1}}^\theta\right) \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Since we assume the robot location is directly measured as x_t , we have the following linear measurement model:

$$x_t = \mathbf{C}z_t + w$$

where

$$\mathbf{C} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

Example II: Odometry and Dynamic Model

Assume that we have the set of the odometry $\mathbf{u} = [u_0, u_1, \dots, u_{T-1}]^T$ for time $0 : T-1$. Let $u_t = [u_t^v, u_t^w]$ where u_t^v and u_t^w are the linear velocity and the angular velocity of the robot at time t . Since the robot velocity is given as the form of the odometry, we can dismiss the velocity from the robot state z_t . In this case, the Dynamic model is given as:

$$\begin{aligned} \begin{bmatrix} z_t^x \\ z_t^y \\ z_t^\theta \end{bmatrix} &= \underbrace{\begin{bmatrix} z_{t-1}^x \\ z_{t-1}^y \\ z_{t-1}^\theta \end{bmatrix} + \begin{bmatrix} u_t^v \cos\left(z_{t-1}^\theta + u_t^w \Delta t\right) \Delta t \\ u_t^v \sin\left(z_{t-1}^\theta + u_t^w \Delta t\right) \Delta t \\ 0 \end{bmatrix}}_g + \begin{bmatrix} 0 \\ 0 \\ v \end{bmatrix} \\ &= \underbrace{\begin{bmatrix} z_{t-1}^x \\ z_{t-1}^y \\ z_{t-1}^\theta \end{bmatrix} + \begin{bmatrix} u_t^v \cos\left(z_{t-1}^\theta + u_t^w \Delta t\right) \Delta t \\ u_t^v \sin\left(z_{t-1}^\theta + u_t^w \Delta t\right) \Delta t \\ 0 \end{bmatrix}}_g + \mathbf{G} \underbrace{v}_{\text{model noise}}. \end{aligned}$$

where $\mathbf{G} = [0, 0, 1]^T$ and $v \sim \mathcal{N}(0, Q)$. In this case, Jacobian at time t $\mathbf{g}_t$ is given as the following:

$$\mathbf{g}_t = \begin{bmatrix} 1 & 0 & -u_t^v \sin\left(\mu_{z_{t-1}|x_{t-1}}^\theta + u_t^w \Delta t\right) \Delta t \\ 0 & 1 & u_t^v \cos\left(\mu_{z_{t-1}|x_{t-1}}^\theta + u_t^w \Delta t\right) \Delta t \\ 0 & 0 & 1 \end{bmatrix}.$$

Thus we can update the covariance by the following step:

$$\Sigma_{z_t|x_{t-1}} = \mathbf{g}_t \Sigma_{z_{t-1}|x_{t-1}} \mathbf{g}_t^T + \mathbf{G} Q \mathbf{G}^T.$$

Example III: Odometry as a Random Variable

Until now we simply assume the model noise is simply given as, for example $w \sim \mathcal{N}(0, Q)$. However the Dynamic model noise can be modeled by treating the odometry as a random variable:

$$\begin{aligned} u_t &= \begin{bmatrix} u_t^v \\ u_t^w \end{bmatrix} + \mathbf{v} \\ &= \mu_{u_t} + \mathbf{v} \end{aligned} \quad (5.15)$$

where $\mathbf{v} = [v^0, v^1]^T \sim \mathcal{N}(\mathbf{0}, \mathbf{M})$. The Dynamic model then can be approximated as the following:

$$z_t \approx g(\mu_{z_{t-1}|x_{t-1}}, u_t) + \mathcal{G}_t(z_{t-1} - \mu_{z_{t-1}|x_{t-1}}) + \mathbf{v}_t(u_t - \mu_{u_t})$$

where

$$\mathbf{v}_t = \left. \frac{\partial g(z_{t-1}, u_t)}{\partial u_t} \right|_{z_{t-1} = \mu_{z_{t-1}|x_{t-1}}}$$

Thus the covariance update is modified as the following:

$$\Sigma_{z_t|x_{t-1}} = \mathcal{G}_t \Sigma_{z_{t-1}|x_{t-1}} \mathcal{G}_t^T + \mathbf{v}_t \mathbf{M} \mathbf{v}_t^T.$$

5.2 Particle Filter

Same as the EKF case, note that the goal of the Bayesian estimation is to find the posterior, since the mean of the posterior is the optimal solution of the Bayesian estimation.

Again, consider the way we compute the posterior. The posterior is given by the measurement model and the dynamic model as the following:

$$\underbrace{p(z_t|x_{0:t}, u_{0:t-1})}_{\text{posterior of } t^{th} \text{ step}} \propto \underbrace{p(x_t|z_t)}_{\text{likelihood (measurement model)}} \underbrace{p(z_t|x_{0:t-1}, u_{0:t-1})}_{\text{prior}}$$

where

$$\underbrace{p(z_t|x_{0:t-1}, u_{0:t-1})}_{\text{prior}} = \int \underbrace{p(z_t|z_{t-1}, u_{t-1})}_{\text{dynamic model}} \underbrace{p(z_{t-1}|x_{0:t-1}, u_{0:t-2})}_{\text{posterior from } t-1^{th} \text{ step}} dz_{t-1}.$$

We assume that each model follows a Gaussian distribution as:

- Dynamic model:

$$z_t = g(z_{t-1}, u_{t-1}) + \mathbf{G}v$$

- Measurement model:

$$x_t = h(z_t) + w$$

where $v \sim \mathcal{N}(0, \mathbf{Q})$ and $w \sim \mathcal{N}(0, \mathbf{R})$ are the dynamic noise and the measurement noise.

In EKF, to approximate the random variables as Gaussian random variables, we linearize the nonlinear function g and h using Jacobian. By doing so, z_t in Dynamic model is represented as the linear combination of two Gaussian random variables z_{t-1} and v . Similarly, x_t in Measurement model can be seen as the linear combination of z_t and w . Since the linear combination of the Gaussian random variables is also a Gaussian random variable, we can easily utilize z_t in Dynamic model and x_t in Measurement model as Gaussian random variables as well.

However, linearizing the nonlinear function introduces errors compared to the actual model, resulting in inaccurate estimations. Is there a way to perform Bayesian estimation without approximating the nonlinear model? Using sampling techniques can solve this problem. The key here is that the optimal solution in Bayesian estimation is not the posterior distribution but simply the mean of the posterior. Therefore, importance sampling techniques can be introduced to estimate this mean.

Importance Sampling

Our goal is to compute the mean, which is equivalent to the expected over the given distribution. Suppose we want to compute the expected value $\mathbb{E}$ of arbitrary nonlinear function f over any distribution p as the following:

$$\mathbb{E}[f] = \int f(z) p(z) dz \quad (5.16)$$

Apparently, we can directly compute the above expectation when z follows a Gaussian distribution and f is a linear function. In case of f being a nonlinear function, it is almost impossible to compute the integration (For example, consider the case that $f(z) = \arctan z$ and p is a Gaussian distribution). Nevertheless, note that it is easy to compute the single value of $f(z)p(z)$ for any z . In order to utilize this fact, let us rewrite Eqn. (5.16) as follows:

$$\begin{aligned} \mathbb{E}[f] &= \int f(z) \frac{p(z)}{q(z)} q(z) dz \\ &= \mathbb{E}_{z \sim q} \left[\frac{p(z)}{q(z)} f(z) \right] \\ &\simeq \frac{1}{N} \sum_{n=1}^N \frac{p(z^{(n)})}{q(z^{(n)})} f(z^{(n)}), \end{aligned} \quad (5.17)$$

where q is a known distribution, and $z^{(n)}$ is a particle sampled from $q(z)$. Since Eqn. (5.17) holds for any nonzero q , we can set q as any known and tractable distribution that we can easily handle. Therefore, we can approximately have the expectation of f , even the integration is infeasible.

Dynamic Update

Now let us apply the importance sampling to compute the mean of the posterior. First we need to compute the prior distribution, which is equivalent to Dynamic update. This also can be resolved by the importance sampling as follows:

$$\begin{aligned}
& \underbrace{p(z_t | x_{0:t-1}, u_{0:t-1})}_{\text{prior}} \\
&= \int \underbrace{p(z_t | z_{t-1}, u_{t-1})}_{\text{dynamic model}} \underbrace{p(z_{t-1} | x_{0:t-1}, u_{0:t-2})}_{\text{posterior from } t-1} dz_{t-1} \\
& \text{(by Bayes' rule,)} \\
&= \int p(z_t | z_{t-1}, u_{t-1}) \frac{p(x_{t-1} | z_{t-1}) p(z_{t-1} | x_{0:t-2}, u_{0:t-2})}{\int p(x_{t-1} | z_{t-1}) p(z_{t-1} | x_{0:t-2}, u_{0:t-2}) dz_{t-1}} dz_{t-1} \\
&= \int \underbrace{p(z_t | z_{t-1}, u_{t-1})}_f \underbrace{\frac{p(x_{t-1} | z_{t-1})}{\int p(x_{t-1} | z_{t-1}) p(z_{t-1} | x_{0:t-2}, u_{0:t-2}) dz_{t-1}}}_{N \cdot w_{t-1}} \underbrace{p(z_{t-1} | x_{0:t-2}, u_{0:t-2})}_{q, \text{ prior at } t-1} dz_{t-1} \\
&= \mathbb{E}_{z \sim q} [w_{t-1} \cdot p(z_t | z_{t-1}, u_{t-1})] \\
&\simeq \sum_n^N w_{t-1}^{(n)} p(z_t | z_{t-1}^{(n)}, u_{t-1})
\end{aligned}$$

where N is the number of samples and

$$z_{t-1}^{(n)} \sim \underbrace{p(z_{t-1} | x_{0:t-2}, u_{0:t-1})}_{q, \text{ prior at } t-1}, \quad w_{t-1}^{(n)} = \frac{p(x_{t-1} | z_{t-1}^{(n)})}{\sum_n^N p(x_{t-1} | z_{t-1}^{(n)})}. \quad (5.18)$$

Consequently, when performing Dynamic Update at time t , first sample particles $z_{t-1}^{(n)}$ from prior at $t-1$. Using those samples and weights, compute the prior at time t . Now we represent the prior as the mixture distribution, thus we can utilize it for the next step.

Measurement Update

Our final goal is to perform measurement update in order to compute the mean of the posterior. By directly applying the sampling-based approach,

the mean value can be represented as the expectation form as:

$$\begin{aligned}
\mathbb{E}[h(z_t)] &= \mu_{z_t|x_t} \\
&= \int h(z_t) p(z_t|x_{0:t}, u_{0:t-1}) dz_t \\
&= \int h(z_t) \frac{p(x_t|z_t) p(z_t|x_{0:t-1}, u_{0:t-1})}{\int p(x_t|z_t, u_{0:t-1}) p(z_t|x_{0:t-1}, u_{0:t-1}) dz_t} dz_t \\
&= \int h(z_t) \frac{p(x_t|z_t)}{\underbrace{\int p(x_t|z_t) p(z_t|x_{0:t-1}, u_{0:t-1}) dz_t}_{N \cdot w_t}} \underbrace{p(z_t|x_{0:t-1}, u_{0:t-1})}_{\text{prior obtained from Dynamic Update}} dz_t \\
&= \mathbb{E}_z[w_t \cdot h(z_t)] \\
&\simeq \sum_n^N w_t^{(n)} h(z_t^{(n)})
\end{aligned}$$

where N is the number of the samples and

$$z_t^{(n)} \sim \underbrace{p(z_t|x_{0:t-1}, u_{0:t-1})}_{\text{prior obtained from Dynamic Update}}, \quad w_t^{(n)} = \frac{p(x_t|z_t^{(n)})}{\sum_n^N p(x_t|z_t^{(n)})}. \quad (5.19)$$

Thus, for Measurement Update first we sample a particle $z_t^{(n)}$ from prior obtained from Dynamic Update, and compute the weight $w_t^{(n)}$ by using the measurement model. Then we compute the weighted sum of h to obtain the mean value $\mathbb{E}[h(z_t)] = \mu_{z_t|x_t}$.

In summary, at time step t , we can perform Dynamic Update and the Measurement Update of the Particle Filter as:

- **Dynamic Update**

1. Sample particles $z_{t-1}^{(n)}$ from prior at $t-1$: $z_{t-1}^{(n)} \sim p(z_{t-1}|x_{0:t-2}, u_{0:t-2})$
2. Compute weights $w_{t-1}^{(n)} \propto p(x_{t-1}|z_{t-1}^{(n)})$
3. Update prior $p(z_t|x_{0:t-1}, u_{0:t-1}) \simeq \sum_n w_{t-1}^{(n)} p(z_t|z_{t-1}^{(n)}, u_{t-1})$

- **Measurement Update**

1. Sample particles $z_t^{(n)}$ from updated prior: $z_t^{(n)} \sim p(z_t|x_{0:t-1}, u_{0:t-1})$
2. Compute weights $w_t^{(n)} \propto p(x_t|z_t^{(n)})$
3. Compute mean $\mathbb{E}[h(z_t)] = \mu_{z_t|x_t} \simeq \sum_n w_t^{(n)} h(z_t^{(n)})$

Note that steps 1-2 of Dynamic Update at time step t was already obtained from steps 1-2 of Measurement Update at time step $t-1$. Therefore we can

Algorithm 3 Particle Filter

Require: $x_t, u_t, \mathcal{Z}_{t-1} = \{z_{t-1}^{(n)}\}_{n=0,\dots,N}, \mathcal{W}_{t-1} = \{w_{t-1}^{(n)}\}_{n=0,\dots,N}$

- 1: $\mathcal{Z}_t = \{\}$ $\mathcal{W}_t = \{\}$
- 2: **for** $n = 0$ to N **do**
- 3: sample $z_t^{(n)} \sim p(z_t|x_{0:t-1}, u_{0:t-1}) \simeq \sum_n w_{t-1}^{(n)} p(z_t|z_{t-1}^{(n)}, u_{t-1})$
- 4: $w_t^{(n)} \propto p(x_t|z_t^{(n)})$
- 5: $\mathcal{Z}_t = \mathcal{Z}_t \cup \{z_t^{(n)}\}$
- 6: $\mathcal{W}_t = \mathcal{W}_t \cup \{w_t^{(n)}\}$
- 7: **end for**
- 8: **return** $\mathcal{Z}_t, \mathcal{W}_t$

simplify the sequence as follows:

Particle Filter:• **Dynamic Update**

1. Update prior $p(z_t|x_{0:t-1}, u_{0:t-1}) \simeq \sum_n w_{t-1}^{(n)} p(z_t|z_{t-1}^{(n)}, u_{t-1})$

• **Measurement Update**

1. Sample particles $z_t^{(n)}$ from updated prior: $z_t^{(n)} \sim p(z_t|x_{0:t-1}, u_{0:t-1})$
2. Compute weights $w_t^{(n)} \propto p(x_t|z_t^{(n)})$
3. Compute mean $\mathbb{E}[h(z_t)] = \mu_{z_t|x_t} \simeq \sum_n w_t^{(n)} h(z_t^{(n)})$
4. Save $z_t^{(n)}$ and $w_t^{(n)}$ for the next step

This particle filter process can be represented in the Algorithm 3

In order to perform step 1 in Dynamic Update it is necessary to compute prior $p(z_t|x_{0:t-1}, u_{0:t-1}) \simeq \sum_n w_{t-1}^{(n)} p(z_t|z_{t-1}^{(n)}, u_{t-1})$. This step can also be replaced by another sampling scheme, thus the sequence of the Particle Filter can be expressed as the followings:

Particle Filter (simplified 1):• **Dynamic Update**

1. Draw $z_{t-1}^{(n)}$ with probability $\propto w_{t-1}^{(n)}$ for time t
2. Set prior as dynamic model: $p(z_t|x_{0:t-1}, u_{0:t-1}) \simeq p(z_t|z_{t-1}, u_{t-1})$

• **Measurement Update**

1. Sample a particle $z_t^{(n)}$ from each prior(= dynamic model)
 $: z_t^{(n)} \sim p(z_t|z_{t-1}^{(n)}, u_{t-1})$

Algorithm 4 Particle Filter (simplified 2)

Require: $x_t, u_t, \mathcal{Z}_{t-1} = \{z_{t-1}^{(n)}\}_{n=0,\dots,N}$

- 1: **for** $n = 0$ to N **do**
- 2: sample $z_t^{(n)} \sim p(z_t|z_{t-1}^{(n)}, u_{t-1})$
- 3: $w'^{(n)} \propto p(x_t|z_t^{(n)})$
- 4: **end for**
- 5: $\mathcal{Z}_t = \{\}$
- 6: **for** $n = 0$ to N **do**
- 7: resample $z_t^{(n)}$ with probability $\propto w'^{(n)}$
- 8: $\mathcal{Z}_t = \mathcal{Z}_t \cup \{z_t^{(n)}\}$
- 9: **end for**
- 10: **return** $\mathcal{Z}_t$

2. Compute weights $w_t^{(n)} \propto p(x_t|z_t^{(n)})$
3. Compute mean $\mathbb{E}[h(z_t)] = \mu_{z_t|x_t} \simeq \sum_n w_t^{(n)} h(z_t^{(n)})$
4. Save $z_t^{(n)}$ and $w_t^{(n)}$ for the next step

Note that, at time step t , step 1 in Dynamic Update utilizes the savings $z_{t-1}^{(n)}$ and $w_{t-1}^{(n)}$ from the Measurement Update in previous time step $t - 1$. Therefore step 1 can be conducted from the previous Measurement Update. Reflecting this fact, the sequence can be further simplified as follows:

Particle Filter (simplified 2):• **Dynamic Update**

1. Set prior as dynamic model: $p(z_t|x_{0:t-1}, u_{0:t-1}) \simeq p(z_t|z_{t-1}, u_{t-1})$

• **Measurement Update**

1. Sample a particle $z_t^{(n)}$ from each prior(= dynamic model)
 $: z_t^{(n)} \sim p(z_t|z_{t-1}^{(n)}, u_{t-1})$,
where $z_{t-1}^{(n)}$ is a particle saved at time $t - 1$
2. Compute weights $w'^{(n)} \propto p(x_t|z_t^{(n)})$
3. Compute mean $\mathbb{E}[h(z_t)] = \mu_{z_t|x_t} \simeq \sum_n w'^{(n)} h(z_t^{(n)})$
4. Draw $z_t^{(n)}$ with probability $\propto w_t^{(n)}$ for time $t + 1$

In this way, there is no longer a need to store the weights $w^{(n)}$ computed from the Measurement Update, while still achieving the same purpose of the particle filter. This simplified particle filter can be represented as in Algorithm 4.

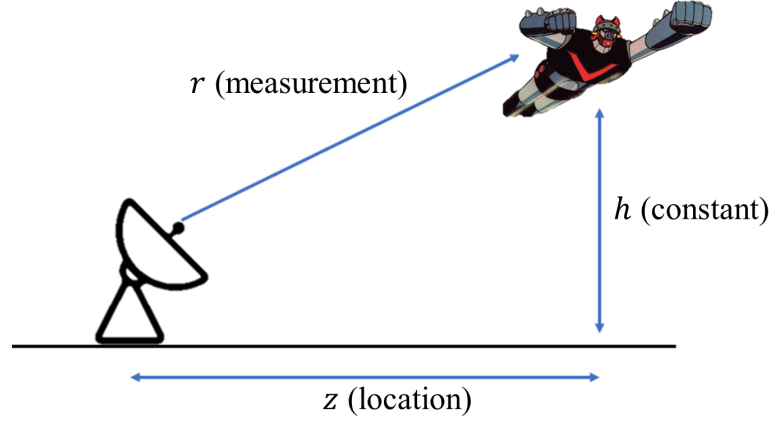


Figure 5.1: Example of flying robot localization

Example I: Constant velocity model

In this example, we aim to estimate the robot's location by measuring the distance to the robot using radar. In this scenario, let us assume the robot is flying at a constant altitude on a 2D plane. Additionally, we assume that the robot is flying at a constant speed along the x -axis. The measured distance from the radar to the robot is denoted as r , and the altitude of the robot is denoted as h . Let

$$\mathbf{z} = \begin{bmatrix} z & \dot{z} & h \end{bmatrix} \quad (5.20)$$

be the robot state. Since the robot velocity $\dot{z}$ and the altitude h is assumed to be constants, the dynamic model is given as:

$$\dot{\mathbf{z}} = \begin{bmatrix} \dot{z} \\ \ddot{z} \\ \dot{z} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} z \\ \dot{z} \\ h \end{bmatrix} + \begin{bmatrix} 0 \\ \omega_1 \\ \omega_2 \end{bmatrix} \quad (5.21)$$

where $\begin{bmatrix} \omega_1 & \omega_2 \end{bmatrix}$ is the model noise. The measurement x is then given as the following:

$$\begin{aligned} x = r &= \sqrt{z^2 + h^2} + \nu \\ &= h(\mathbf{z}) + \nu \end{aligned} \quad (5.22)$$

where ν is the measurement noise.

(Exercise 5-1) write the algorithm table of particle filter using the above system models.

6

Filter-based SLAM

6.1 Measurement and Landmark

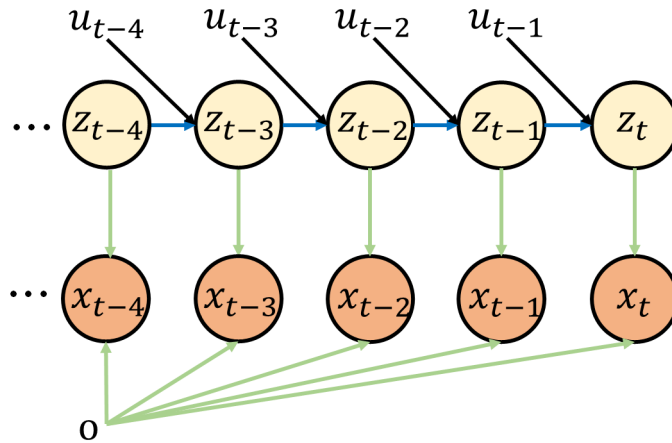


Figure 6.1: Graphical model of robot poses and observation. Every observations are obtained according to the origin $\mathbf{o}$.

So far, x has been referred to as an observation of the robot pose z . Here, x is an observation of the robot pose z with respect to the orientation $\mathbf{o}$, as depicted in Fig. 6.1. Therefore, the obtained x can be considered as a measurement of the robot pose in the absolute coordinate system.

However, obtaining measurements of the robot pose in an absolute coordinate system is challenging unless equipped with a global positioning system.

In general, the robot pose z is obtained by the robot directly measuring its surrounding environment. In other words, the robot pose z is measured with respect to the environment that the robot is exploring. Let's denote the reference points used for localization while navigating in the environment as

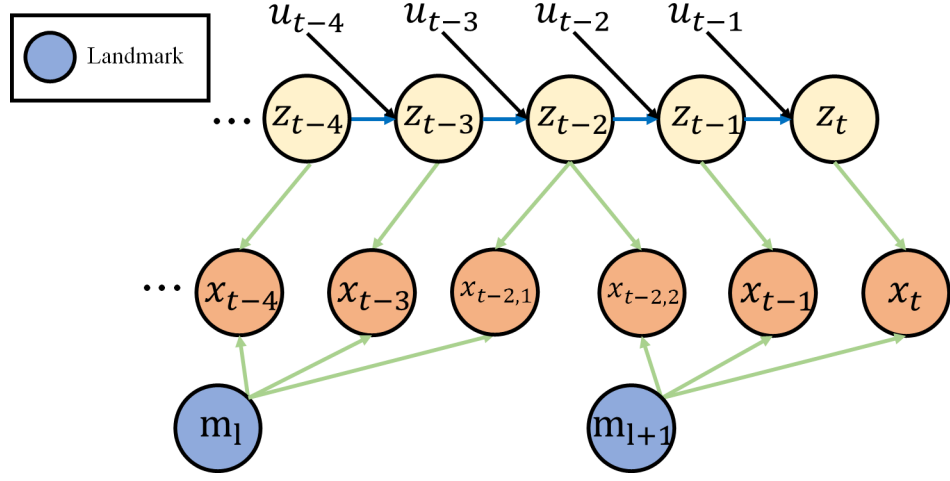


Figure 6.2: Graphical model of robot poses and observation with landmarks. Since the robot measures its location according to the landmarks, each observation x is connected to the landmarks and the robot poses.

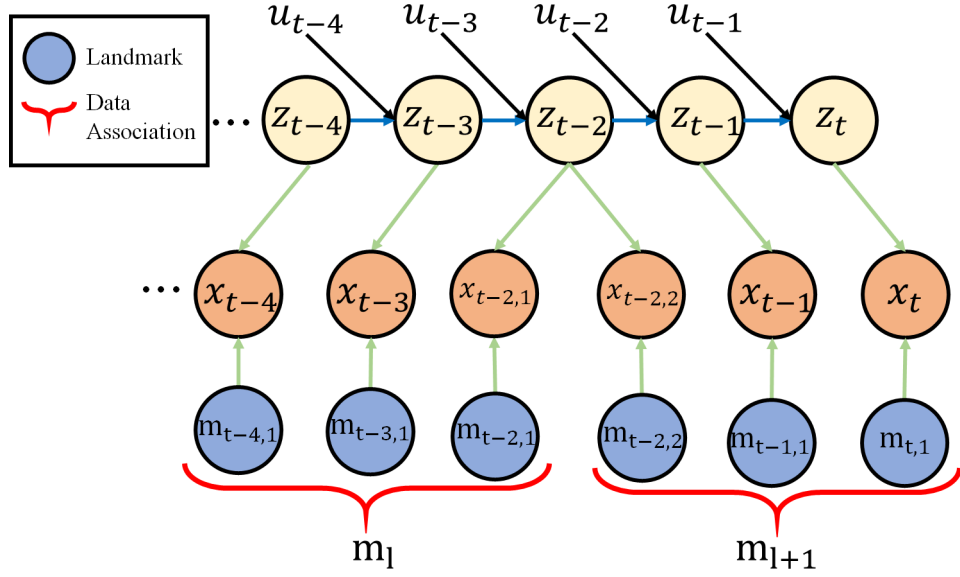


Figure 6.3: Graphical model and data association.

landmarks, as shown in Fig. 6.2. While navigating, the robot measures its relative pose with respect to these landmarks.

In other words, instead of using the origin $\mathbf{o}$ of the absolute coordinate system, the robot's relative pose is observed with respect to the landmarks present in the environment. In this scenario, both the robot's pose z and the

pose of the landmarks m are unknown. Therefore, unlike the localization of the robot pose alone, it is necessary to estimate both z and m . This problem of simultaneously estimating the robot's pose and the poses of all the landmarks that serve as references for observations is known as *Simultaneous Localization and Mapping* (SLAM). Here, *localization* refers to estimating the robot pose, while *mapping* refers to estimating the poses of landmarks.

In order to estimate the robot's relative pose with respect to landmarks, it is necessary to determine which landmark each measurement is referencing. In simpler terms, it is crucial to distinguish whether a landmark observed in the current frame corresponds to the same landmark seen in the previous frame. This task of associating observations with landmarks is known as the *data association* (DA) problem, which is depicted in Fig. 6.3. It is one of the most critical issues in SLAM because incorrect solutions to the data association problem can lead to erroneous pose optimization results.

In this textbook, to simplify the problem, we only consider *SLAM with known data association* or *known correspondence*. This means we assume that we know the number of landmarks present in the environment, and we have prior knowledge of data associations. Afterward, we address the problems of estimating the poses of the robot and landmarks.

6.2 EKF SLAM

Suppose we have the measurement $x_{1:t}$ odometry $u_{1:t}$ and for time $1, \dots, t$. Recall that our goal is to estimate the robot state z_t for time t by estimating the following posterior:

$$\underbrace{p(z_t | x_{0:t}, u_{0:t-1})}_{\text{target posterior of } t^{\text{th}} \text{ step}} \propto \underbrace{p(x_t | z_t)}_{\text{likelihood (measurement model)}} \underbrace{p(z_t | x_{0:t-1}, u_{0:t-1})}_{\text{prior}}.$$

The prior is given as:

$$\underbrace{p(z_t | x_{0:t-1}, u_{0:t-1})}_{\text{prior}} = \int \underbrace{p(z_t | z_{t-1}, u_t)}_{\text{dynamic model}} \underbrace{p(z_{t-1} | x_{0:t-1}, u_{0:t-1})}_{\text{posterior from } t-1} dz_{t-1}.$$

To estimate the target posterior, we assume the following two models:

- Dynamic model:

$$z_t = g(z_{t-1}, u_t) + v$$

- Measurement model:

$$x_t = h(z_t) + w$$

where $v \sim \mathcal{N}(0, \mathbf{Q})$ and $w \sim \mathcal{N}(0, \mathbf{R})$ are the dynamic noise and the measurement noise. Recall that in EKF, we can estimate the target posterior by

update the prior (dynamic model) and the likelihood (measurement model):

- Dynamic update:

$$\begin{aligned}\mu_{z_t|x_{t-1}} &= g\left(\mu_{z_{t-1}|x_{t-1}}, u_t\right) \\ \Sigma_{z_t|x_{t-1}} &= \mathbf{G}_t \Sigma_{z_{t-1}|x_{t-1}} \mathbf{G}_t^T + \mathbf{Q}\end{aligned}$$

- Measurement update:

$$\begin{aligned}\mu_{z_t|x_t} &= \mu_{z_t|x_{t-1}} + K_t \left(x_t - h\left(\mu_{z_t|x_{t-1}}\right) \right) \\ \Sigma_{z_t|x_t} &= \Sigma_{z_t|x_{t-1}} - K_t \mathbf{H}_t \Sigma_{z_t|x_{t-1}}\end{aligned}$$

where

$$K_t = \Sigma_{z_t|x_{t-1}} \mathbf{H}_t^T \left(\mathbf{H}_t \Sigma_{z_t|x_{t-1}} \mathbf{H}_t^T + \mathbf{R} \right)^{-1}.$$

Now Let us consider the SLAM problem: the robot explores 2-dimensional space, and the robot state involves the robot pose $z_t = [z_t^x, z_t^y, z_t^\theta]^T$. Note that in this case, the mean of the state $\mu_{z_t|x_t}$ is a 3×1 vector, and the covariance $\Sigma_{z_t|x_t}$ is a 3×3 matrix, which are represented as:

$$\mu_{z_t|x_t} = \begin{bmatrix} \mu^x \\ \mu^y \\ \mu^\theta \end{bmatrix}, \quad \Sigma_{z_t|x_t} = \begin{bmatrix} \Sigma^{xx} & \Sigma^{xy} & \Sigma^{x\theta} \\ \Sigma^{yx} & \Sigma^{yy} & \Sigma^{y\theta} \\ \Sigma^{\theta x} & \Sigma^{\theta y} & \Sigma^{\theta\theta} \end{bmatrix}.$$

Apparently, $\mathbf{Q}$ and $\mathbf{R}$ are also 3×3 matrices that represent uncertainties for the robot pose. For example, $\mathbf{R}$ represents the noise of the measurement of $x_t = [x_t^x, x_t^y, x_t^\theta]^T$ which is given as:

$$\mathbf{R} = \begin{bmatrix} \mathbf{R}^{xx} & \mathbf{R}^{xy} & \mathbf{R}^{x\theta} \\ \mathbf{R}^{yx} & \mathbf{R}^{yy} & \mathbf{R}^{y\theta} \\ \mathbf{R}^{\theta x} & \mathbf{R}^{\theta y} & \mathbf{R}^{\theta\theta} \end{bmatrix}.$$

Until now, the robot state z only involves the robot pose itself, and the measurement x is also related to the robot pose. In real-world scenarios, however, the robot obtains the measurement x which describes the relation between the robot pose and the poses of the landmarks; for example, the distance between the sensor of the robot and the landmarks in the environment. Therefore, the new state z' should include the poses of the landmarks as well as the robot pose:

$$\begin{aligned}z'_t &= [z_t^T, m_{0:M}^T]^T \\ &= \underbrace{[z_t^x, z_t^y, z_t^\theta]}_{\text{robot}}, \underbrace{[m_0^T, \dots, m_M^T]}_{\text{landmarks}}^T \\ &= \underbrace{[z_t^x, z_t^y, z_t^\theta]}_{\text{robot}}, m_0^x, m_0^y, \dots, m_M^x, m_M^y]^T\end{aligned}$$

where

$$\begin{aligned}\mathbf{m}_{0:M}^T &= [m_0^T, \dots, m_M^T]^T \\ &= [m_0^x, m_0^y, \dots, m_M^x, m_M^y]^T.\end{aligned}\quad (6.1)$$

Here, $m^T = [m^x, m^y]^T$ is the landmark pose. M denotes the total number of landmarks. Note that we assume the total number of landmarks M is previously known, which is a strong assumption compared to real-world scenario. For now, we maintain this assumption to keep our problem simple. With this redefined state z' , the mean $\mu_{z_t|x_t}$ is now a $(3 + 2M) \times 1$ vector, and the covariance $\Sigma_{z_t|x_t}$ is a $(3 + 2M) \times (3 + 2M)$ matrix as follows:

$$\begin{aligned}\mu_{z|x} &= \left[\begin{array}{ccc|ccc} \mu^x & \mu^y & \mu^\theta & \mu_0^{mx} & \mu_0^{my} & \dots & \mu_M^{mx} & \mu_M^{my} \end{array} \right]^T, \\ \Sigma_{z|x} &= \begin{bmatrix} \begin{array}{ccc|ccc} \Sigma^{xx} & \Sigma^{xy} & \Sigma^{x\theta} & \Sigma^{x,m_0^x} & \Sigma^{x,m_0^y} & \dots & \Sigma^{x,m_M^x} & \Sigma^{x,m_M^y} \\ \Sigma^{yx} & \Sigma^{yy} & \Sigma^{y\theta} & \Sigma^{y,m_0^x} & \Sigma^{y,m_0^y} & \dots & \Sigma^{y,m_M^x} & \Sigma^{y,m_M^y} \\ \Sigma^{\theta x} & \Sigma^{\theta y} & \Sigma^{\theta\theta} & \Sigma^{\theta,m_0^x} & \Sigma^{\theta,m_0^y} & \dots & \Sigma^{\theta,m_M^x} & \Sigma^{\theta,m_M^y} \end{array} \\ \begin{array}{ccc|ccc} \Sigma^{m_0^x,x} & \Sigma^{m_0^x,y} & \Sigma^{m_0^x,\theta} & \Sigma^{m_0^x,m_0^x} & \Sigma^{m_0^x,m_0^y} & \dots & \Sigma^{m_0^x,m_M^x} & \Sigma^{m_0^x,m_M^y} \\ \Sigma^{m_0^y,x} & \Sigma^{m_0^y,y} & \Sigma^{m_0^y,\theta} & \Sigma^{m_0^y,m_0^x} & \Sigma^{m_0^y,m_0^y} & \dots & \Sigma^{m_0^y,m_M^x} & \Sigma^{m_0^y,m_M^y} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \Sigma^{m_M^x,x} & \Sigma^{m_M^x,y} & \Sigma^{m_M^x,\theta} & \Sigma^{m_M^x,m_0^x} & \Sigma^{m_M^x,m_0^y} & \dots & \Sigma^{m_M^x,m_M^x} & \Sigma^{m_M^x,m_M^y} \\ \Sigma^{m_M^y,x} & \Sigma^{m_M^y,y} & \Sigma^{m_M^y,\theta} & \Sigma^{m_M^y,m_0^x} & \Sigma^{m_M^y,m_0^y} & \dots & \Sigma^{m_M^y,m_M^x} & \Sigma^{m_M^y,m_M^y} \end{array} \end{bmatrix} \\ &= \begin{bmatrix} \Sigma^{zz} & \Sigma^{zm} \\ \Sigma^{mz} & \Sigma^{mm} \end{bmatrix}.\end{aligned}$$

To make it simple, we omit the subscript t . Compared to EKF case, the only difference is the newly defined state z' that involves the poses of the landmarks, because the actual measurement that the mobile robot can obtain is the relative geometric information between the robot and the landmarks in the environment. The other part remains the same as the EKF. Let us examine how we practically implement the EKF SLAM with the following example.

Example: Robot and Landmark Pose Estimation in 2D Space

Suppose the robot explores in 2D space with the odometry $u_t = [u_t^v \ u_t^w]$, where u^v and u^w are the linear velocity and the angular velocity of the robot at time t . Recall that we need to perform the following two steps for EKF SLAM:

- Dynamic update:

$$\begin{aligned}\mu_{z_t|x_{t-1}} &= g(\mu_{z_{t-1}|x_{t-1}}, u_t) \\ \Sigma_{z_t|x_{t-1}} &= \mathbf{g}_t \Sigma_{z_{t-1}|x_{t-1}} \mathbf{g}_t^T + \mathbf{Q}\end{aligned}$$

- Measurement update:

$$\begin{aligned}\mu_{z_t|x_t} &= \mu_{z_t|x_{t-1}} + K_t \left(x_t - h \left(\mu_{z_t|x_{t-1}} \right) \right) \\ \Sigma_{z_t|x_t} &= \Sigma_{z_t|x_{t-1}} - K_t \mathcal{H}_t \Sigma_{z_t|x_{t-1}}\end{aligned}$$

where

$$K_t = \Sigma_{z_t|x_{t-1}} \mathcal{H}_t^T \left(\mathcal{H}_t \Sigma_{z_t|x_{t-1}} \mathcal{H}_t^T + \mathbf{R} \right)^{-1}.$$

Dynamic Update: Mean

Note that the nonlinear Dynamic model g is defined only for the robot state $z = [z^x \ z^y \ z^\theta]^T$ as the following:

$$\begin{bmatrix} z_t^x \\ z_t^y \\ z_t^\theta \end{bmatrix} = \underbrace{\begin{bmatrix} z_{t-1}^x \\ z_{t-1}^y \\ z_{t-1}^\theta \end{bmatrix}}_g + \begin{bmatrix} u_t^v \cos \left(z_{t-1}^\theta \right) \Delta t \\ u_t^v \sin \left(z_{t-1}^\theta \right) \Delta t \\ 0 \end{bmatrix}.$$

Since we have newly defined state vector

$$z' = \underbrace{[z_t^x, z_t^y, z_t^\theta]}_{z \text{ (robot)}} \underbrace{[m_0^x, m_0^y, m_0^\theta, \dots, m_M^x, m_M^y, m_M^\theta]}_{\mathbf{m} \text{ (landmarks)}}^T$$

we can rewrite the Dynamic model as:

$$z' = \begin{bmatrix} z_t^x \\ z_t^y \\ z_t^\theta \\ \mathbf{m} \end{bmatrix} = \underbrace{\begin{bmatrix} z_{t-1}^x \\ z_{t-1}^y \\ z_{t-1}^\theta \\ \mathbf{m} \end{bmatrix}}_{g'} + \begin{bmatrix} u_t^v \cos \left(z_{t-1}^\theta \right) \Delta t \\ u_t^v \sin \left(z_{t-1}^\theta \right) \Delta t \\ 0 \\ \mathbf{0} \end{bmatrix},$$

where g' is the nonlinear function of the redefined Dynamic model. Consequently, with g' , we can update the mean as the following:

$$\mu_{z_t|x_{t-1}} = g' \left(\mu_{z_{t-1}|x_{t-1}}, u_t \right)$$

Dynamic Update: Covariance

We can directly compute the Jacobian $\mathcal{G}'$ of g' as:

$$\begin{aligned}\mathcal{G}' &= \begin{bmatrix} \frac{\partial z_t}{\partial z_{t-1}} & \frac{\partial z_t}{\partial \mathbf{m}} \\ \frac{\partial \mathbf{m}}{\partial z_{t-1}} & \frac{\partial \mathbf{m}}{\partial \mathbf{m}} \end{bmatrix} \\ &= \begin{bmatrix} \mathcal{G} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}.\end{aligned}$$

Note that for the modified $(3 + 2M) \times (3 + 2M)$ Jacobian $\mathcal{G}'$, we can directly utilize 3×3 Jacobian $\mathcal{G}$ of the robot pose.

With the modified Jacobian $\mathcal{G}'$, we can rewrite the covariance update step:

$$\begin{aligned}\Sigma_{z_t|x_{t-1}} &= \mathcal{G}'_t \Sigma_{z_{t-1}|x_{t-1}} \mathcal{G}'_t^T + Q' \\ &= \begin{bmatrix} \mathcal{G}_t & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \Sigma^{zz} & \Sigma^{zm} \\ \Sigma^{mz} & \Sigma^{mm} \end{bmatrix} \begin{bmatrix} \mathcal{G}_t^T & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \end{bmatrix} + \begin{bmatrix} Q & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \\ &= \begin{bmatrix} \mathcal{G}_t \Sigma^{zz} \mathcal{G}_t^T & \mathcal{G}_t \Sigma^{zm} \\ (\mathcal{G}_t \Sigma^{zm})^T & \Sigma^{mm} \end{bmatrix} + \begin{bmatrix} Q & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}.\end{aligned}$$

Note that in the Dynamic update, mean and covariance of the landmark does not change.

Measurement Update

In general, a range sensor returns the distance (range) and the angle measurement according to the sensor pose. Suppose the robot equips a range sensor and obtain the measurement $x_{k,t}$ of the k^{th} landmark at time t :

$$x_{k,t} = \begin{bmatrix} d_{k,t} \\ \phi_{k,t} \end{bmatrix}.$$

Here, following the above measurement $x_{k,t}$ we can redefine x_t as:

$$x_t = \{x_{k,t}\}_{k=0,\dots,M}. \quad (6.2)$$

Different from our measurements, the state z' represents the robot pose and the landmark poses represented in the Cartesian coordinate. To exploit our measurement, let us consider the relation between z' and $x_{k,t}$. Since the measurement $x_{k,t}$ is relative to the robot, the position of the landmark can be represented as follows:

$$\begin{bmatrix} m_k^x \\ m_k^y \end{bmatrix} = \begin{bmatrix} z_t^x \\ z_t^y \end{bmatrix} + \begin{bmatrix} d_{k,t} \cos(z_t^\theta + \phi_{k,t}) \\ d_{k,t} \sin(z_t^\theta + \phi_{k,t}) \end{bmatrix}.$$

Let $\delta_{k,t} = \begin{bmatrix} \delta_{k,t}^x \\ \delta_{k,t}^y \end{bmatrix} = \begin{bmatrix} m_k^x \\ m_k^y \end{bmatrix} - \begin{bmatrix} z_t^x \\ z_t^y \end{bmatrix}$, then we have:

$$\begin{aligned}\sqrt{\delta_{k,t}^T \delta_{k,t}} &= d_{k,t} \\ \tan(z_t^\theta + \phi_{k,t}) &= \frac{\delta_{k,t}^y}{\delta_{k,t}^x}.\end{aligned}$$

Therefore, the measurement $x_{k,t}$ can be rewritten as:

$$\begin{aligned}
 x_{k,t} &= h(z_t, m_k) = h(z_t^x, z_t^y, z_t^\theta, m_k^x, m_k^y) \\
 &= \underbrace{\begin{bmatrix} \sqrt{\delta_{k,t}^T \delta_{k,t}} \\ \arctan\left(\frac{\delta_{k,t}^y}{\delta_{k,t}^x}\right) - z_t^\theta \end{bmatrix}}_{h \text{ for } k^{th} \text{ landmark}}. \tag{6.3}
 \end{aligned}$$

Since we have M landmarks, suppose that the robot observes M landmarks and obtain M measurements. Then the measurement x_t at time t is given as:

$$x_t = \underbrace{\begin{bmatrix} \sqrt{\delta_{0,t}^T \delta_{0,t}} \\ \arctan\left(\frac{\delta_{0,t}^y}{\delta_{0,t}^x}\right) - z_t^\theta \\ \vdots \\ \sqrt{\delta_{M,t}^T \delta_{M,t}} \\ \arctan\left(\frac{\delta_{M,t}^y}{\delta_{M,t}^x}\right) - z_t^\theta \end{bmatrix}}_h \triangleq \begin{bmatrix} r_{0,t} \\ a_{0,t} \\ \vdots \\ r_{M,t} \\ a_{M,t} \end{bmatrix}.$$

Now we can compute Jacobian $\mathcal{H}$ as the following:

$$\begin{aligned}
 \mathcal{H} &= \begin{bmatrix} \frac{\partial r_0}{\partial z^x} & \frac{\partial r_0}{\partial z^y} & \frac{\partial r_0}{\partial z^\theta} & \frac{\partial r_0}{\partial m_0^x} & \frac{\partial r_0}{\partial m_0^y} & \cdots & \frac{\partial r_0}{\partial m_M^x} & \frac{\partial r_0}{\partial m_M^y} \\ \frac{\partial a_0}{\partial z^x} & \frac{\partial a_0}{\partial z^y} & \frac{\partial a_0}{\partial z^\theta} & \frac{\partial a_0}{\partial m_0^x} & \frac{\partial a_0}{\partial m_0^y} & \cdots & \frac{\partial a_0}{\partial m_M^x} & \frac{\partial a_0}{\partial m_M^y} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \frac{\partial r_M}{\partial z^x} & \frac{\partial r_M}{\partial z^y} & \frac{\partial r_M}{\partial z^\theta} & \frac{\partial r_M}{\partial m_0^x} & \frac{\partial r_M}{\partial m_0^y} & \cdots & \frac{\partial r_M}{\partial m_M^x} & \frac{\partial r_M}{\partial m_M^y} \\ \frac{\partial a_M}{\partial z^x} & \frac{\partial a_M}{\partial z^y} & \frac{\partial a_M}{\partial z^\theta} & \frac{\partial a_M}{\partial m_0^x} & \frac{\partial a_M}{\partial m_0^y} & \cdots & \frac{\partial a_M}{\partial m_M^x} & \frac{\partial a_M}{\partial m_M^y} \end{bmatrix} \\
 &= \begin{bmatrix} -\delta_0^x/r_0 & -\delta_0^y/r_0 & 0 & \delta_0^x/r_0 & \delta_0^y/r_0 & \cdots & 0 & 0 \\ \delta_0^y/r_0^2 & -\delta_0^x/r_0^2 & -1 & -\delta_0^y/r_0^2 & \delta_0^x/r_0^2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ -\delta_M^x/r_M & -\delta_M^y/r_M & 0 & 0 & 0 & \cdots & \delta_M^x/r_M & \delta_M^y/r_M \\ \delta_M^y/r_M^2 & -\delta_M^x/r_M^2 & -1 & 0 & 0 & \cdots & -\delta_M^y/r_M^2 & \delta_M^x/r_M^2 \end{bmatrix}.
 \end{aligned}$$

Note that $\mathcal{H}$ is a $2M \times (3 + 2M)$ matrix. Unfortunately, in real-world scenarios it is challenging to observe all M landmarks while navigating. To

handle this problem, suppose the robot observes some landmarks. For example, let k^{th} landmark is solely observed. In this case, we cannot obtain the measurement values for the other landmarks. We can say that the measurements of the other landmarks are obtained as any value regardless of the other landmarks' true pose, because they are not observed. Considering this aspect we can write this case as the following:

$$\frac{\partial \mathbf{m}_i}{\partial *} = 0, \quad \frac{\partial *}{\partial \mathbf{m}_i} = 0 \quad \forall i \neq k.$$

Thus we can rewrite the Jacobian $\mathbf{H}$ as the following:

$$\mathbf{H} = \begin{bmatrix} 0 & 0 & 0 & 0 & \dots & 0 & 0 & 0 & 0 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \dots \\ 0 & 0 & 0 & 0 & \dots & 0 & 0 & 0 & 0 & \dots \\ \frac{\partial r_k}{\partial z^x} & \frac{\partial r_k}{\partial z^y} & \frac{\partial r_k}{\partial z^\theta} & 0 & \dots & 0 & \frac{\partial r_k}{\partial m_k^x} & \frac{\partial r_k}{\partial m_k^y} & 0 & \dots \\ \frac{\partial a_k}{\partial z^x} & \frac{\partial a_k}{\partial z^y} & \frac{\partial a_k}{\partial z^\theta} & 0 & \dots & 0 & \frac{\partial a_k}{\partial m_k^x} & \frac{\partial a_k}{\partial m_k^y} & 0 & \dots \\ 0 & 0 & 0 & 0 & \dots & 0 & 0 & 0 & 0 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \dots \\ 0 & 0 & 0 & 0 & \dots & 0 & 0 & 0 & 0 & \dots \end{bmatrix}.$$

From this technique, we can selectively consider the measurements of the observed landmarks. Using the Jacobian $\mathbf{H}$ now we can compute the Kalman gain K and update the parameters sequentially by following the rest steps:

$$\begin{aligned} K_t &= \Sigma_{z_t|x_{t-1}} \mathbf{H}_t^T \left(\mathbf{H}_t \Sigma_{z_t|x_{t-1}} \mathbf{H}_t^T + \mathbf{R}' \right)^{-1} \\ \mu_{z_t|x_t} &= \mu_{z_t|x_{t-1}} + K_t \left(x_t - h \left(\mu_{z_t|x_{t-1}} \right) \right) \\ \Sigma_{z_t|x_t} &= \Sigma_{z_t|x_{t-1}} - K_t \mathbf{H}_t \Sigma_{z_t|x_{t-1}}. \end{aligned}$$

Consequently, the only difference between EKF localization and EKF SLAM is the pose. In EKF localization, the pose only considers the robot pose z , but in EKF SLAM the pose involves the robot pose z and the landmark pose m as well, and those two variables are optimized simultaneously.

6.3 Fast SLAM

As we discussed in EKF SLAM, our newly defined posterior is expressed as the following:

$$p(z'_t | x_{0:t}, u_{0:t-1}) = p(z_t, \mathbf{m}_{0:M} | x_{0:t}, u_{0:t-1}) \quad (6.4)$$

where

$$\begin{aligned} \mathbf{m}_{0:M}^T &= [m_o^T, \dots, m_M^T] \\ &= [m_o^x, m_o^y, \dots, m_M^x, m_M^y] \end{aligned} \quad (6.5)$$

is a set of landmark pose. Also note that

$$x_t = \{x_{m,t}\}_{m=1,\dots,M} \quad (6.6)$$

is a set of measurement at time t . $x_{m,t}$ is a measurement for the m^{th} landmark at time t . Similar to the case of EKF SLAM, measurements $x_{0:t}$ is of the relative pose between the robot and the landmarks, and the dynamic model and the measurement models g and h are nonlinear models.

Here, we can also adopt *particle filter* approach for our SLAM problem in order to handle the nonlinear system. However, compared to the robot localization problem, SLAM possesses the high-dimensional system modeling since the pose vector z' involves the robot pose and poses of a number of landmarks as well. To handle such a complex system, the particle filter-based approach requires a large number of particles, exponentially proportional to the system complexity. As a result, it leads to a highly inefficient SLAM algorithm.

To relax this issue, we use particle filter for the robot pose only, and use EKF for the landmarks. For this strategy, firstly let us rewrite Eqn. (6.4) as the following:

$$\begin{aligned} p(z'_t | x_{0:t}, u_{0:t-1}) &= p(z_t, \mathbf{m}_{0:M} | x_{0:t}, u_{0:t-1}) \\ &= p(\mathbf{m}_{0:M} | z_t, x_{0:t}, u_{0:t-1}) p(z_t | x_{0:t}, u_{0:t-1}) \\ &= \underbrace{p(\mathbf{m}_{0:M} | z_t, x_{0:t})}_{\text{posterior of landmarks}} \underbrace{p(z_t | x_{0:t}, u_{0:t-1})}_{\text{posterior of robot localization}}. \end{aligned} \quad (6.7)$$

Assume that the landmarks are conditionally independent given the robot pose z_t , Eqn. (6.7) is expressed as:

$$\begin{aligned} p(z'_t | x_{0:t}, u_{0:t-1}) &= \underbrace{p(z_t | x_{0:t}, u_{0:t-1})}_{\text{posterior of robot localization}} \underbrace{p(\mathbf{m}_{0:M} | z_t, x_{0:t})}_{\text{posterior of landmarks}} \\ &= \underbrace{p(z_t | x_{0:t}, u_{0:t-1})}_{\text{posterior of robot localization}} \underbrace{\prod_m^M p(\mathbf{m}_m | z_t, x_{0:t})}_{\text{posterior of landmarks}}. \end{aligned} \quad (6.8)$$

Eqn. (6.8) is Rao-Blackwellization for SLAM formulation. In this representation, we will adopt particle filter-based approach to the posterior of robot pose, and adopt EKF-based approach to the posterior of landmarks separately. First, let us perform Dynamic Update.

Dynamic Update

For Dynamic Update, we do not need to consider the landmarks since only the robot moves according to our Dynamic model. Therefore, as in the 2nd line of Algorithm. 4, we can perform the following for the robot pose estimation:

- **Dynamic Update**

1. Set prior as dynamic model: $p(z_t|x_{0:t-1}, u_{0:t-1}) \simeq p(z_t|z_{t-1}, u_{t-1})$

Measurement Update

For Measurement Update, both the two posteriors of robot pose and the landmark pose in Eqn. (6.8) should be considered. As in the 2nd line of Algorithm. 4, for the robot pose we can perform the following:

- **Measurement Update**

1. Sample a particle $z_t^{(n)}$ from each prior(= dynamic model)
 $: z_t^{(n)} \sim p(z_t|z_{t-1}^{(n)}, u_{t-1})$,
 where $z_{t-1}^{(n)}$ is a particle saved at time $t - 1$

Meanwhile, how about the measurement update for the landmarks? To reduce the computational burden, as mentioned earlier we can adopt EKF-based approach. In this case, each particle has its landmark observations as shown in Fig. 6.4. In other words, each particle has its own landmarks, and we separately perform EKF for all landmarks. Let the landmark posterior at time t be:

$$p(\mathbf{m}_{m,t}|z_t, x_{0:t}) = \mathcal{N}(\mu_{m,t}, \Sigma_{m,t}). \quad (6.9)$$

Here, $x_{m,t}$ is the measurement for m^{th} landmark measured at time t . Since our measurement model is originally defined as:

$$x_t = h(z_t'), \quad (6.10)$$

thus without loss of generality we can redefine the above measurement model for the m^{th} landmark observed at time t as:

$$x_{m,t} = h(\mathbf{m}_{m,t}, z_t). \quad (6.11)$$

Using Eqn. (6.9) and Eqn. (6.11), similar to EKF-based robot localization we have:

- **Measurement update:**

$$\hat{x}_{m,t} = h(\mu_{m,t-1}, z_t) \quad (6.12)$$

$$\mu_{m,t} = \mu_{m,t-1} + K_{m,t}(x_{m,t} - \hat{x}_{m,t}) \quad (6.13)$$

$$\Sigma_{m,t} = \Sigma_{m,t-1} - K_{m,t}\mathbf{H}_{m,t}\Sigma_{m,t-1} \quad (6.14)$$

where

$$\mathbf{R}'_{m,t} = \mathbf{H}_{m,t}\Sigma_{m,t-1}\mathbf{H}_{m,t}^T + \mathbf{R} \quad (6.15)$$

$$K_{m,t} = \Sigma_{m,t-1}\mathbf{H}_{m,t}^T (\mathbf{R}'_{m,t})^{-1} \quad (6.16)$$

and $\mathbf{H}_{m,t}$ is the Jacobian of $h(\mathbf{m}_{m,t}, z_t)$.

Until now, we have conducted the following steps:

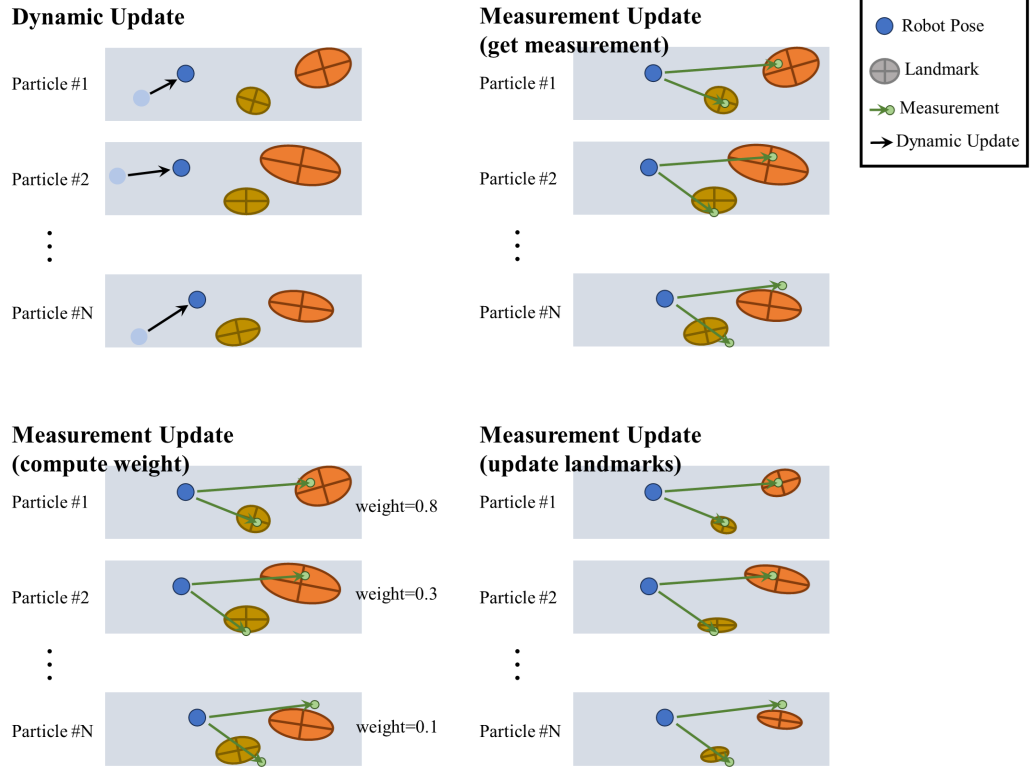


Figure 6.4: Example of the FastSLAM1.0.

- **Dynamic Update**

1. Set prior as dynamic model: $p(z_t|x_{0:t-1}, u_{0:t-1}) \simeq p(z_t|z_{t-1}, u_{t-1})$

- **Measurement Update**

1. Sample a particle $z_t^{(n)}$ from each prior(= dynamic model)
 $z_t^{(n)} \sim p(z_t|z_{t-1}^{(n)}, u_{t-1})$,
 where $z_{t-1}^{(n)}$ is a particle saved at time $t-1$
2. Perform EKF for $p(m_{m,t}|z_t^{(n)}, x_{0:t})$

In order to complete the particle filter-based robot pose estimation, for the next step, we need to compute the weight w and resample $z_t^{(n)}$ according to w for the robot pose. To compute the weight w , let us revisit the measurement update formulation in the particle filter. For the measurement update, we

can compute the mean of the posterior as the followings:

$$\begin{aligned}
& \mathbb{E} [h(\mathbf{m}_{m,t}, z_t)] \\
&= \int h(\mathbf{m}_{m,t}, z_t) p(z_t | x_{0:t}, u_{0:t-1}) dz_t \\
&= \int h(\mathbf{m}_{m,t}, z_t) \underbrace{\frac{p(z_t | x_{0:t}, u_{0:t-1})}{p(z_t | x_{0:t-1}, u_{0:t-1})}}_{\text{prior from Dynamic Update}} \underbrace{p(z_t | x_{0:t-1}, u_{0:t-1})}_{\text{prior from Dynamic Update}} dz_t \\
&= \int h(\mathbf{m}_{m,t}, z_t) \underbrace{w_t}_{\text{weight}} \underbrace{p(z_t | x_{0:t-1}, u_{0:t-1})}_{\text{prior from Dynamic Update}} dz_t. \tag{6.17}
\end{aligned}$$

Here, it is known that the weight w_t is represented as:

$$w_t \propto \prod_{m \in \text{observed}} |2\pi \mathbf{R}'_{m,t}|^{-\frac{1}{2}} \exp \left[- (x_{m,t} - \hat{x}_{m,t})^T (\mathbf{R}'_{m,t})^{-1} (x_{m,t} - \hat{x}_{m,t}) \right]. \tag{6.18}$$

Combine all the process, we finally have:

- **Dynamic Update**

1. Set prior as dynamic model: $p(z_t | x_{0:t-1}, u_{0:t-1}) \simeq p(z_t | z_{t-1}, u_{t-1})$

- **Measurement Update**

1. Sample a particle $z_t^{(n)}$ from each prior(= dynamic model)
 $: z_t^{(n)} \sim p(z_t | z_{t-1}^{(n)}, u_{t-1})$,
where $z_{t-1}^{(n)}$ is a particle saved at time $t-1$
2. Perform EKF for $p(\mathbf{m}_{m,t} | z_t^{(n)}, x_{0:t})$
3. Compute
 $w_t^{(n)} = \prod_m^M |2\pi \mathbf{R}'|^{-1} \exp \left[(x_{m,t} - \hat{x}_{m,t}^{(n)})^T (\mathbf{R}')^{-1} (x_{m,t} - \hat{x}_{m,t}^{(n)}) \right]$
4. Draw $z_t^{(n)}$ with probability $\propto w_t^{(n)}$ for time $t+1$

The algorithm table of Fast SLAM is shown in Algorithm. 5.

Proof of Eqn. (6.18)

From Eqn. (6.17), we have:

$$\begin{aligned}
w_t &= \frac{p(z_t | x_{0:t}, u_{0:t-1})}{p(z_t | x_{0:t-1}, u_{0:t-1})} \\
&= \frac{\eta \cdot p(x_t | z_t, x_{0:t-1}, u_{0:t-1}) \cancel{p(z_t | x_{0:t-1}, u_{0:t-1})}}{\cancel{p(z_t | x_{0:t-1}, u_{0:t-1})}} \\
&= \eta \cdot p(x_t | z_t, x_{0:t-1}, u_{0:t-1}).
\end{aligned}$$

Algorithm 5 Fast SLAM 1.0

Require: $x_t, u_t, \mathcal{Z}_{t-1} = \{z_{t-1}^{(n)}, \mu_{0:M,t-1}^{(n)}, \Sigma_{0:M,t-1}^{(n)}\}_{n=0,\dots,N}$

- 1: **for** $n = 0$ to N **do**
- 2: sample $z_t^{(n)} \sim p(z_t | z_{t-1}^{(n)}, u_{t-1})$
- 3: $w^{(n)} = 1$
- 4: **for** all observed landmarks **do**
- 5: EKF for Landmarks:

$$\hat{x}_{m,t} = h(\mu_{m,t-1}, z_t)$$

$$\mathbf{R}'_{m,t} = \mathbf{H}_{m,t} \Sigma_{m,t-1} \mathbf{H}_{m,t}^T + \mathbf{R}$$

$$\mathbf{K}_{m,t} = \Sigma_{m,t-1} \mathbf{H}_{m,t}^T (\mathbf{R}'_{m,t})^{-1}$$

$$\mu_{m,t} = \mu_{m,t-1} + \mathbf{K}_{m,t} (x_{m,t} - \hat{x}_{m,t})$$

$$\Sigma_{m,t} = \Sigma_{m,t-1} - \mathbf{K}_{m,t} \mathbf{H}_{m,t} \Sigma_{m,t-1}$$
- 6: $w^{(n)} = w^{(n)} \cdot |2\pi \mathbf{R}'_{m,t}|^{-1} \exp \left[- \left(x_{m,t} - \hat{x}_{m,t}^{(n)} \right)^T (\mathbf{R}'_{m,t})^{-1} \left(x_{m,t} - \hat{x}_{m,t}^{(n)} \right) \right]$
- 7: **end for**
- 8: **for** all unobserved landmarks **do**
- 9: $\mu_{t,m}, \Sigma_{t,m} = \mu_{t-1,m}, \Sigma_{t-1,m}$
- 10: **end for**
- 11: **end for**
- 12: $\mathcal{Z}_t = \{\}$
- 13: **for** $n = 0$ to N **do**
- 14: resample $\{z_{t-1}^{(n)}, \mu_{0:M,t-1}^{(n)}, \Sigma_{0:M,t-1}^{(n)}\}$ with probability $\propto w^{(n)}$
- 15: **if** sampled **then**
- 16: $\mathcal{Z}_t = \mathcal{Z}_t \cup \{z_{t-1}^{(n)}, \mu_{0:M,t-1}^{(n)}, \Sigma_{0:M,t-1}^{(n)}\}$
- 17: **end if**
- 18: **end for**
- 19: **return** $\mathcal{Z}_t$

where η is a normalizing constant. Since $x_t = \{x_{m,t}\}_m \quad \forall$ observed m , we have:

$$\begin{aligned}
 w_t &= \eta \cdot p(x_t | z_t, x_{0:t-1}, u_{0:t-1}) \\
 &= \eta \prod_{m \in \text{observed}} p(x_{m,t} | z_t, x_{0:t-1}, u_{0:t-1}).
 \end{aligned} \tag{6.19}$$

Let $w_{m,t} = p(x_m | z_t, x_{0:t-1}, u_{0:t-1})$. Then we have:

$$\begin{aligned}
 w_{m,t} &= p(x_{m,t} | z_t, x_{0:t-1}, u_{0:t-1}) \\
 &= \int p(x_{m,t} | z_t, x_{0:t-1}, \mathbf{m}_m, u_{0:t-1}) p(\mathbf{m}_m | z_t, x_{0:t-1}, u_{0:t-1}) d\mathbf{m}_m \\
 &= \int p(x_{m,t} | z_t, \mathbf{m}_m, u_{0:t-1}) p(\mathbf{m}_m | z_t, x_{0:t-1}, u_{0:t-1}) d\mathbf{m}_m \\
 &= \int p(x_{m,t} | z_t, \mathbf{m}_m) \underbrace{p(\mathbf{m}_m | z_{t-1}, x_{0:t-1})}_{\text{from Markov chain}} d\mathbf{m}_m. \tag{6.20}
 \end{aligned}$$

The result of Eqn. (6.20) shows the similar procedure of the prior computation in Eqn. (5.2). In other words, Eqn. (6.20) can be seen as the update process using the prior. Since we have

$$x_{m,t} = h(\mu_{m,t-1}, z_t) + v \tag{6.21}$$

$$p(\mathbf{m}_m | z_{t-1}, x_{0:t-1}) = \mathcal{N}(\mu_{m,t-1}, \Sigma_{m,t-1}) \tag{6.22}$$

from Eqn. (6.11) and Eqn. (6.9). Then, similar to the solution of Eqn. (5.2) we can directly compute the parameters (mean, covariance) by substituting Eqn. (6.21) into Eqn. (6.20), we have:

$$w_{m,t} = |2\pi \mathbf{R}'_{m,t}|^{-\frac{1}{2}} \exp \left[- (x_{m,t} - \hat{x}_{m,t})^T (\mathbf{R}'_{m,t})^{-1} (x_{m,t} - \hat{x}_{m,t}) \right]. \tag{6.23}$$

Substituting Eqn. (6.23) into Eqn. (6.19), we finally have Eqn. (6.18). ■

7

Graph Optimization

Suppose the robot navigation concludes at final time step T and we perform the filter-based pose estimation. In this case, we have the prior distribution from the estimated posterior at time $T - 1$ and the Dynamic model. We also know the likelihood defined from our Measurement model and currently obtained observation x_T . Therefore, we directly obtain the posterior by direct calculation. Note that this is possible because the whole process is conducted sequentially for every time step $0, \dots, T - 1$ thereby we always know the previously computed posterior at every time step.

The above estimation process, we call it *online* estimation, which solely optimizes the current state z_T and cannot jointly optimizes the previous states $\mathbf{z}_{T-1} = \{z_0, \dots, z_{T-1}\}$. If we can achieve the joint optimization for the entire time steps by considering the whole observations $\mathbf{x}_T = \{x_0, \dots, x_T\}$, then we can further correct the accumulated errors for the whole states $\mathbf{z}_T = \{z_0, \dots, z_T\}$. This process is conducted after obtaining the whole observations $\mathbf{x}_T = \{x_0, \dots, x_T\}$, thus we call this process as *offline* estimation. Then how can we perform this offline joint optimization not for the current state, but for the entire states including the past time steps?

7.1 Pose Graph and Maximum A Posteriori

To achieve the joint optimization for the entire robot states $\mathbf{z}_T = \{z_0, \dots, z_T\}$ by considering every observations $\mathbf{x}_T = \{x_0, \dots, x_T\}$ and odometries (or control inputs) $u_0, \dots, u_{T-1}$, we can compute the joint distribution of these variables. Similar to the filter-based optimization, again we can compute the posterior for computing optimized variables. Here, our newly defined posterior for all time step $0, \dots, T$ is given as the following:

$$p(z_{0:T} | x_{0:T}, u_{0:T-1}). \quad (7.1)$$

Since the graph of our system is given as Fig. 4.4, the posterior in Eqn. (7.1) can be expressed as:

$$\begin{aligned}
& \underbrace{p(z_{0:T}|x_{0:T}, u_{0:T-1})}_{\text{posterior at time } T} \\
& \text{(Bayes' rule)} = \frac{1}{\mathcal{Z}} p(x_T|z_{0:T}, x_{0:T-1}, u_{0:T-1}) p(z_{0:T}|x_{0:T-1}, u_{0:T-1}) \\
& \text{(from graph model)} = \frac{1}{\mathcal{Z}} p(x_T|z_T) p(z_{0:T}|x_{0:T-1}, u_{0:T-1}) \\
& = \frac{1}{\mathcal{Z}} p(x_T|z_T) p(z_{0:T-1}, z_T|x_{0:T-1}, u_{0:T-1}) \\
& \text{(independence)} = \frac{1}{\mathcal{Z}} p(x_T|z_T) p(z_T|z_{0:T-1}, x_{0:T-1}, u_{0:T-1}) p(z_{0:T-1}|x_{0:T-1}, u_{0:T-1}) \\
& \text{(from graph model)} = \frac{1}{\mathcal{Z}} \underbrace{p(x_T|z_T)}_{\text{Measurement model}} \underbrace{p(z_T|z_{T-1}, u_{T-1})}_{\text{Dynamic model}} \underbrace{p(z_{0:T-1}|x_{0:T-1}, u_{0:T-1})}_{\text{posterior at time } T-1} \\
& \hspace{15em} (7.2)
\end{aligned}$$

where $\mathcal{Z}$ is a normalizing constant. In Eqn. (7.2) we have $T-1$ time step posterior, thus we can recursively apply this expansion. Then we finally have:

$$p(z_{0:T}|x_{0:T}, u_{0:T-1}) = \frac{1}{\mathcal{Z}'} p(z_0|x_0) \prod_t^T [p(x_t|z_t) p(z_t|z_{t-1}, u_{t-1})] \quad (7.3)$$

where $\mathcal{Z}'$ is a normalizing constant. Note that $p(z_0|x_0)$ is a constant, since the first post of a mobile robot z_0 is set to be origin. In this aspect the observation x_0 is in fact unnecessary, therefore in practice we only need $1, \dots, T$ time step observations $\{x_1, \dots, x_T\}$.

For the filter-based approach, we use Bayesian mean square error, which is equivalent to compute the posterior. This is possible because in the filter-based setting the previously computed posterior is simply given, therefore the previous states like $z_{T-1} = \{z_0, \dots, z_{T-1}\}$ does not need to be optimized. All that we need to optimize is the final state z_T , and this can be handled by computing the current posterior.

However, now things are changed. We need to optimize all the states $z_T = \{z_0, \dots, z_T\}$ that satisfies the posterior in Eqn. (7.3) simultaneously. How do we achieve this? Recall that at Section III, for estimation we utilize Maximum Likelihood Estimation, which finds optimal value that maximize the likelihood according to the given observations. We can use the similar strategies since we now have the posterior and the observations $x_T = \{x_0, \dots, x_T\}$ and the odometries $u_0, \dots, u_{T-1}$. In other words, we can find the optimal states that maximize our posterior according to our obtained observations and the odometries. This process is called *Maximum A Posteriori* (MAP). Compared to MLE, MAP can achieve the optimal solution that satisfies both the likelihood and the prior.

To perform MAP, similar to the case of MLE let us take logarithm for Eqn. (7.3). Then we have:

$$\begin{aligned} \log p(z_{0:T}|x_{0:T}, u_{0:T-1}) &= \underbrace{-\log \mathcal{Z}' + \log p(z_0|x_0)}_{\text{const.}} \\ &+ \sum_t^T [\log p(x_t|z_t) + \log p(z_t|z_{t-1}, u_{t-1})]. \end{aligned} \quad (7.4)$$

Our Measurement model and Dynamic model are given as:

- Dynamic model:

$$z_t = g(z_{t-1}, u_t) + v$$

- Measurement model:

$$x_t = h(z_t) + w$$

where $v \sim \mathcal{N}(0, \mathbf{Q})$ and $w \sim \mathcal{N}(0, \mathbf{R})$ are the dynamic noise and the measurement noise. Since the PDFs of both Dynamic model and Measurement model follow Gaussian, we have:

- Logarithm of Dynamic model:

$$\begin{aligned} \log p(z_t|z_{t-1}, u_{t-1}) &= \text{const.} - \frac{1}{2} (z_t - g(z_{t-1}, u_t))^T \mathbf{Q}^{-1} (z_t - g(z_{t-1}, u_t)) \\ &\propto - \underbrace{(z_t - g(z_{t-1}, u_t))^T}_{\text{error } e} \underbrace{\mathbf{Q}^{-1}}_{\mathbf{\Omega}} \underbrace{(z_t - g(z_{t-1}, u_t))}_{\text{error } e} \\ &\stackrel{\text{def}}{=} -\mathbf{e}_D^T(z_t) \mathbf{\Omega}_D \mathbf{e}_D(z_t) \\ &\stackrel{\text{def}}{=} -e_D(z_t) \end{aligned} \quad (7.5)$$

- Logarithm of Measurement model:

$$\begin{aligned} \log p(x_t|z_t) &= \text{const.} - \frac{1}{2} (x_t - h(z_t))^T \mathbf{R}^{-1} (x_t - h(z_t)) \\ &\propto - \underbrace{(x_t - h(z_t))^T}_{\text{error } e} \underbrace{\mathbf{R}^{-1}}_{\mathbf{\Omega}} \underbrace{(x_t - h(z_t))}_{\text{error } e} \\ &\stackrel{\text{def}}{=} -\mathbf{e}_M^T(z_t) \mathbf{\Omega}_M \mathbf{e}_M(z_t) \\ &\stackrel{\text{def}}{=} -e_M(z_t) \end{aligned} \quad (7.6)$$

where $\mathbf{e}$ denotes an error (vector), and e denotes an error function (scalar). $\mathbf{\Omega}$ denotes the *Information* matrix, which is defined as the inverse of the

covariance matrix. Substituting Eqn. (7.5) and Eqn. (7.6) into Eqn. (7.4), we have:

$$\log p(z_{0:t}|x_{0:t}, u_{0:t-1}) \propto - \sum_t \left[\mathbf{e}_D^T(z_t) \boldsymbol{\Omega}_D \mathbf{e}_D(z_t) + \mathbf{e}_M^T(z_t) \boldsymbol{\Omega}_M \mathbf{e}_M(z_t) \right]. \quad (7.7)$$

Consequently, the optimal solution $\hat{\mathbf{z}} = \{\hat{z}_0, \dots, \hat{z}_T\}$ by MAP is given as:

$$\begin{aligned} \hat{\mathbf{z}} &= \underset{\mathbf{z}}{\operatorname{argmax}} \log p(z_{0:T}|x_{0:T}, u_{0:T-1}) \\ &= \underset{\mathbf{z}}{\operatorname{argmin}} \sum_t^T \left[\underbrace{\mathbf{e}_D^T(z_t) \boldsymbol{\Omega}_D \mathbf{e}_D(z_t)}_{\text{Dynamic error}} + \underbrace{\mathbf{e}_M^T(z_t) \boldsymbol{\Omega}_M \mathbf{e}_M(z_t)}_{\text{Measurement error}} \right]. \end{aligned} \quad (7.8)$$

Note that the probabilistic estimation problem is now converted to the optimization problem of quadratic error function $e = \mathbf{e}^T \boldsymbol{\Omega} \mathbf{e}$. In Eqn. (7.8) the optimal solution is obtained by directly considering the errors of the Measurement model and the Dynamic model, different from the previous filter-based approach. Now let us discuss how we achieve optimization in the next section.

7.2 Gauss-Newton Method and Graph Optimization

The error function $e = \mathbf{e}^T \boldsymbol{\Omega} \mathbf{e}$ in Eqn. (7.8) is a quadratic function according to $\mathbf{e}$. Therefore, the optimal solution $\hat{\mathbf{z}}$ can be obtained by simply taking the derivative, if the error $\mathbf{e}$ is expressed as a linear combination of $\mathbf{z}$. Unfortunately, this condition does not hold because the error $\mathbf{e}$ involves the nonlinear function of $\mathbf{z}$. To make our discussion simple, suppose we only have Measurement model for our Graph model. As defined in Eqn. (7.6) the error $\mathbf{e}$ is given as:

$$\mathbf{e}_M(z_t) = x_t - h(z_t). \quad (7.9)$$

In general, the measurement function h is nonlinear so that it is challenging to directly handle the error function. Therefore, to obtain the optimal solution $\hat{z}_t$ approximately, we adopt Gauss-Newton method.

Example: Gauss-Newton Method for Single Node

Suppose we have an appropriate initial point z_t . Similar to EKF, near this initial point, we can perform linearization by adopting Taylor expansion to the error function e as the following:

$$\begin{aligned} \mathbf{e}_M(z_t + \Delta z) &\simeq \mathbf{e}_M(z_t) + \mathbf{H}_t((z_t + \Delta z) - z_t) \\ &= \mathbf{e}_M(z_t) + \mathbf{H}_t \Delta z \end{aligned} \quad (7.10)$$

Algorithm 6 Gauss-Newton Method for Single Node**Require:** initial point z_t

```

1: while  $e < \tau$  do
2:   compute  $\mathbf{A}_{M,t}$ ,  $\mathbf{b}_{M,t}$  and  $c_{M,t}$ 
3:    $z_t = z_t - \mathbf{A}_{M,t}^{-1} \mathbf{b}_{M,t}$ 
4:    $e = e_M(z_t)$ 
5: end while
6: return  $z_t$ 

```

where $\Delta z \ll \epsilon$. $\mathcal{H}$ is the Jacobian which is given as

$$\begin{aligned} \mathcal{H}_t &= \left. \frac{\partial \mathbf{e}_M(z)}{\partial z} \right|_{z=z_t} \\ &= - \left. \frac{\partial h(z)}{\partial z} \right|_{z=z_t}. \end{aligned} \quad (7.11)$$

Using Eqn. (7.10), the error function e for $z_t + \Delta z$ can be expressed as:

$$\begin{aligned} e_M(z_t + \Delta z) &= \mathbf{e}_M^T(z_t + \Delta z) \mathbf{\Omega}_M \mathbf{e}_M(z_t + \Delta z) \\ &\simeq (\mathbf{e}_M(z_t) + \mathcal{H}_t \Delta z)^T \mathbf{\Omega}_M (\mathbf{e}_M(z_t) + \mathcal{H}_t \Delta z) \\ &= \Delta z^T \mathcal{H}_t^T \mathbf{\Omega}_M \mathcal{H}_t \Delta z + 2 \mathbf{e}_M^T(z_t) \mathbf{\Omega}_M \mathcal{H}_t \Delta z + \mathbf{e}_M^T(z_t) \mathbf{\Omega}_M \mathbf{e}_M(z_t) \\ &= \Delta z^T \mathbf{A}_{M,t} \Delta z + 2 \mathbf{b}_{M,t} \Delta z + c_{M,t} \end{aligned} \quad (7.12)$$

where

$$\begin{aligned} \mathbf{A}_{M,t} &= \mathcal{H}_t^T \mathbf{\Omega}_M \mathcal{H}_t \\ \mathbf{b}_{M,t} &= \mathbf{e}_M^T(z_t) \mathbf{\Omega}_M \mathcal{H}_t \\ c_{M,t} &= \mathbf{e}_M^T(z_t) \mathbf{\Omega}_M \mathbf{e}_M(z_t). \end{aligned}$$

Since Eqn. (7.12) is now a quadratic function according to Δz , we can simply take the derivative in order to find the local minimum as:

$$\frac{\partial e_M(z_t + \Delta z)}{\partial \Delta z} = 2 \mathbf{A}_{M,t} \Delta z + 2 \mathbf{b}_{M,t}. \quad (7.13)$$

Set to zero we finally have:

$$\Delta \hat{z} = -\mathbf{A}_{M,t}^{-1} \mathbf{b}_{M,t}. \quad (7.14)$$

Using Eqn. (7.14) the optimal solution $\hat{z}_t$ is approximately represented as:

$$\hat{z}_t \simeq z_t + \Delta \hat{z}. \quad (7.15)$$

By setting this approximated solution as a new initial point, we can iteratively perform the same process while the value of the error function converges to the certain threshold. This is the case of Gauss-Newton method for time step t (single node), and is shown in Algorithm. 6.

Graph SLAM

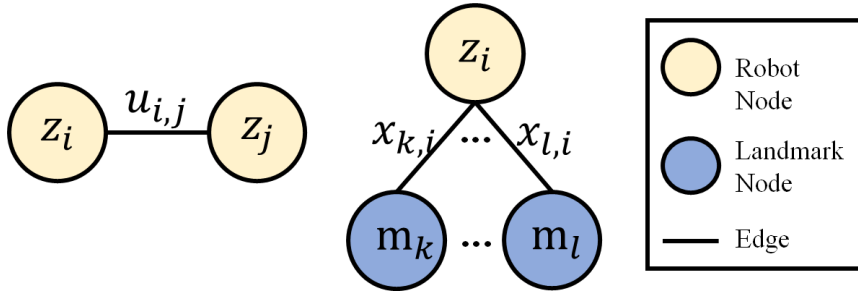


Figure 8.1: Example of the pose graph

8.1 Graph Optimization and Gauss-Newton Method

By using MAP formulation in Eqn. (7.2), we can consider the relations between robot poses, and the relations between the robot poses and the landmarks, as shown in Fig. 8.1. In cases like filter-based SLAM, we also assume that we know the data association of observed landmarks this time. In this representation, a set of robot poses $\mathcal{Z}$ and a set of landmarks $\mathcal{M}$ can be seen as the nodes, and a set of odometries $\mathcal{U}$ and a set of measurements $\mathcal{X}$ can be seen as the edges between the nodes. Then our graphical model $\mathcal{G}$ is defined with a set of vertices $\mathcal{V}$ and a set of edges $\mathcal{E}$ as:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}) \quad (8.1)$$

where

$$\begin{aligned}\mathbf{v} &= \{\mathbf{v}_i^T\}_i^T \\ &= \{z_0^T, \dots, z_T^T, \mathbf{m}_0^T, \dots, \mathbf{m}_M^T\}^T \\ &= \mathcal{Z} \cup \mathcal{M}\end{aligned}\tag{8.2}$$

$$\begin{aligned}\mathcal{E} &= \{\mathcal{E}_k^T\}_k^T \\ &= \{u_{i,j}^T\}^T \cup \{x_{i',j'}^T\}^T \quad \forall i, j, i', j' \\ &= \mathcal{U} \cup \mathcal{X}.\end{aligned}\tag{8.3}$$

Meanwhile, our Dynamic Model and Measurement Model can also be rewritten as follows:

- Dynamic model:

$$z_j = g(z_i, u_{i,j}) + v\tag{8.4}$$

- Measurement model:

$$x_{k,i} = h(\mathbf{m}_k, z_i) + w\tag{8.5}$$

where $\mathbf{m}_k$ is the k^{th} landmark, and $v \sim \mathcal{N}(0, \mathbf{\Omega}_D^{-1})$ and $w \sim \mathcal{N}(0, \mathbf{\Omega}_M^{-1})$ are the dynamic noise and the measurement noise. In order to represent the relation between the nodes, without loss of generality we can redefine our Dynamic Model as:

$$u_{i,j} = g'(z_i, z_j) + v'.\tag{8.6}$$

For simplicity, we omit the superscript in Eqn. (8.6). The error e_D of this Dynamic Model for z_i and z_j is given as:

$$\begin{aligned}e_D(z_i, z_j) &= (u_{i,j} - g(z_i, z_j))^T \mathbf{\Omega}_D (u_{i,j} - g(z_i, z_j)) \\ &= \mathbf{e}_D^T(z_i, z_j) \mathbf{\Omega}_D \mathbf{e}_D(z_i, z_j).\end{aligned}\tag{8.7}$$

In order to extend our definition, we can rewrite our Dynamic error $e_D(z_i, z_j)$ as:

$$e_{D,ij}(\mathbf{v}) = \mathbf{e}_{D,ij}^T(\mathbf{v}) \mathbf{\Omega}_D \mathbf{e}_{D,ij}(\mathbf{v}).$$

Similarly, we can define the error e_M of the Measurement Model for $\mathbf{m}_i$ and z_j as:

$$\begin{aligned}e_{M,i'j'}(\mathbf{v}) &= (x_{j,i} - h(\mathbf{m}_j, z_i))^T \mathbf{\Omega}_M (x_{j,i} - h(\mathbf{m}_j, z_i)) \\ &= \mathbf{e}_{M,i'j'}^T(\mathbf{v}) \mathbf{\Omega}_M \mathbf{e}_{M,i'j'}(\mathbf{v})\end{aligned}\tag{8.8}$$

Then the total error e is defined as:

$$\begin{aligned} e(\mathbf{v}) &= \sum_{u_{i,j} \in \mathcal{U}} e_{D,ij}(\mathbf{v}) + \sum_{x_{i,j} \in \mathcal{X}} e_{M,ij}(\mathbf{v}) \\ &= \sum_{u_{i,j} \in \mathcal{U}} \mathbf{e}_{D,ij}^T(\mathbf{v}) \boldsymbol{\Omega}_D \mathbf{e}_{D,ij}(\mathbf{v}) + \sum_{x_{i,j} \in \mathcal{X}} \mathbf{e}_{M,ij}^T(\mathbf{v}) \boldsymbol{\Omega}_M \mathbf{e}_{M,ij}(\mathbf{v}). \end{aligned} \quad (8.9)$$

By adopting Taylor expansion, with the initial pose $\mathbf{v}$ we have the following linearizations:

$$\mathbf{e}_{D,ij}(\mathbf{v} + \Delta \mathbf{v}) \simeq \mathbf{e}_{D,ij}(\mathbf{v}) + \mathcal{G}_{ij} \Delta \mathbf{v} \quad (8.10)$$

$$\mathbf{e}_{M,ij}(\mathbf{v} + \Delta \mathbf{v}) \simeq \mathbf{e}_{M,ij}(\mathbf{v}) + \mathcal{H}_{ij} \Delta \mathbf{v} \quad (8.11)$$

where

$$\begin{aligned} \mathcal{G}_{ij} &= \begin{bmatrix} \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial v_0} & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial v_1} & \cdots & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial v_{|\mathbf{v}|-1}} & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial v_{|\mathbf{v}|}} \\ \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial z_0} & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial z_1} & \cdots & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial m_{M-1}} & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial m_M} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial z_0} & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial z_1} & \cdots & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial m_{M-1}} & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial m_M} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{0} & \cdots & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial z_i} & \cdots & \frac{\partial \mathbf{e}_{D,ij}(\mathbf{v})}{\partial z_j} & \cdots & \mathbf{0} \end{bmatrix} \end{aligned} \quad (8.12)$$

$$\begin{aligned} \mathcal{H}_{ij} &= \begin{bmatrix} \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial v_0} & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial v_1} & \cdots & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial v_{|\mathbf{v}|-1}} & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial v_{|\mathbf{v}|}} \\ \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial z_0} & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial z_1} & \cdots & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial m_{M-1}} & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial m_M} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial z_0} & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial z_1} & \cdots & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial m_{M-1}} & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial m_M} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{0} & \cdots & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial z_i} & \cdots & \frac{\partial \mathbf{e}_{M,ij}(\mathbf{v})}{\partial m_j} & \cdots & \mathbf{0} \end{bmatrix}. \end{aligned} \quad (8.13)$$

Similar to Eqn. (7.12), using Eqn. (8.12) and Eqn. (8.13) we can represent Eqn. (8.9) as the following:

$$e(\mathbf{v} + \Delta \mathbf{v}) \simeq \Delta \mathbf{v}^T \mathbf{A} \Delta \mathbf{v} + 2\mathbf{b} \Delta \mathbf{v} + c \quad (8.14)$$

where

$$\begin{aligned} \mathbf{A} &= \mathbf{A}_D + \mathbf{A}_M \\ &= \underbrace{\sum_{u_{i,j} \in \mathcal{U}} \mathcal{G}_{ij}^T \boldsymbol{\Omega}_D \mathcal{G}_{ij}}_{\mathbf{A}_D} + \underbrace{\sum_{x_{i,j} \in \mathcal{X}} \mathcal{H}_{ij}^T \boldsymbol{\Omega}_M \mathcal{H}_{ij}}_{\mathbf{A}_M} \end{aligned} \quad (8.15)$$

$$\begin{aligned} \mathbf{b} &= \mathbf{b}_D + \mathbf{b}_M \\ &= \underbrace{\sum_{u_{i,j} \in \mathcal{U}} \mathbf{e}_{D,ij}^T \boldsymbol{\Omega}_D \mathcal{G}_{ij}}_{\mathbf{b}_D} + \underbrace{\sum_{x_{i,j} \in \mathcal{X}} \mathbf{e}_{M,ij}^T \boldsymbol{\Omega}_M \mathcal{H}_{ij}}_{\mathbf{b}_M} \end{aligned} \quad (8.16)$$

$$\begin{aligned} c &= c_D + c_M \\ &= \underbrace{\sum_{u_{i,j} \in \mathcal{U}} \mathbf{e}_{D,ij}^T \boldsymbol{\Omega}_D \mathbf{e}_{D,ij}^T}_{c_D} + \underbrace{\sum_{x_{i,j} \in \mathcal{X}} \mathbf{e}_{M,ij}^T \boldsymbol{\Omega}_M \mathbf{e}_{M,ij}^T}_{c_M}. \end{aligned} \quad (8.17)$$

Algorithm 7 Graph SLAM with Gauss-Newton Method**Require:** initial graph $G = \{\mathcal{V}, \mathcal{E}\}$, update step λ and error threshold τ

```

1: while  $e(\mathcal{V}) < \tau$  do
2:   compute  $\mathbf{A}$ ,  $\mathbf{b}$  and  $c$ 
3:    $\Delta \mathcal{V} = -\mathbf{A}^{-1} \mathbf{b}$ 
4:    $\mathcal{V} = \mathcal{V} + \lambda \Delta \mathcal{V}$ 
5:    $e(\mathcal{V}) = \sum_{u_{i,j} \in \mathcal{U}} e_{D,ij}(\mathcal{V}) + \sum_{x_{i,j} \in \mathcal{X}} e_{M,ij}(\mathcal{V})$ 
6: end while
7: return  $\mathcal{V}$ 

```

Note that for simplicity we omit $\mathcal{V}$ for $e_{ij}(\mathcal{V})$ in Eqn. (8.16) and Eqn. (8.17). Differentiating Eqn. (8.14) and set it to zero, we finally have:

$$\Delta \mathcal{V} = -\mathbf{A}^{-1} \mathbf{b}. \quad (8.18)$$

Consequently, by using Eqn. (8.18) we can update the nodes $\mathcal{V}$. By iteratively performing this update process, we have the approximated optimal solution. The whole process for this pose graph optimization is represented as in Algorithm. 7.

8.2 Graph Optimization and Levenberg-Marquardt

The most popular issue of the Graph SLAM is the sparsity of the Jacobian, which makes it challenge to compute the solution in Eqn. (8.18). This solution should be iteratively calculated for the Gauss-Newton method in Algorithm. 7. However, $\mathbf{A}$ in Eqn. (8.18) has large dimension and also sparse in usual, it is hard to achieve real-time performance with simple matrix inversion or sometimes it faces singularity problem. For example, $\mathbf{A}_D$ in Eqn. (8.16) can be expressed as:

$$\mathbf{A}_D = \sum_{u_{i,j} \in \mathcal{U}} \mathcal{G}_{ij}^T \Omega_D \mathcal{G}_{ij} \quad (8.19)$$

where

$$\begin{aligned}
& \mathcal{G}_{ij}^T \Omega_D \mathcal{G}_{ij} = \\
& \begin{bmatrix} \mathbf{0} & \dots & \frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_i} & \dots & \frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_j} & \dots & \mathbf{0} \end{bmatrix}^T \Omega_D \begin{bmatrix} \mathbf{0} & \dots & \frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_i} & \dots & \frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_j} & \dots & \mathbf{0} \end{bmatrix} \\
& = \begin{bmatrix} \mathbf{0} & \dots & \left(\frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_i} \right)^T \Omega_D \frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_i} & \dots & \left(\frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_i} \right)^T \Omega_D \frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_j} & \dots & \mathbf{0} \\ \mathbf{0} & \dots & \dots & \dots & \dots & \dots & \mathbf{0} \\ \mathbf{0} & \dots & \left(\frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_j} \right)^T \Omega_D \frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_i} & \dots & \left(\frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_j} \right)^T \Omega_D \frac{\partial e_{D,ij}(\mathcal{V})}{\partial z_j} & \dots & \mathbf{0} \\ \mathbf{0} & \dots & \dots & \dots & \dots & \dots & \mathbf{0} \end{bmatrix}. \quad (8.20)
\end{aligned}$$

Algorithm 8 Graph SLAM with Levenberg Method**Require:** initial graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, update step λ and error threshold τ

```

1: while  $e(\mathcal{V}) < \tau$  do
2:    $e_{pre}(\mathcal{V}) = \sum_{u_{i,j} \in \mathcal{U}} e_{D,ij}(\mathcal{V}) + \sum_{x_{i,j} \in \mathcal{X}} e_{M,ij}(\mathcal{V})$ 
3:   compute  $\mathbf{A}$ ,  $\mathbf{b}$  and  $c$ 
4:    $\Delta \mathcal{V} = -(\mathbf{A} + \lambda \cdot \mathbf{I})^{-1} \mathbf{b}$ 
5:    $\mathcal{V} = \mathcal{V} + \Delta \mathcal{V}$ 
6:    $e(\mathcal{V}) = \sum_{u_{i,j} \in \mathcal{U}} e_{D,ij}(\mathcal{V}) + \sum_{x_{i,j} \in \mathcal{X}} e_{M,ij}(\mathcal{V})$ 
7:   if  $e_{pre}(\mathcal{V}) < e(\mathcal{V})$  then
8:      $\lambda = 2 \cdot \lambda$ 
9:   else
10:     $\lambda = 1/2 \cdot \lambda$ 
11:   end if
12: end while
13: return  $\mathcal{V}$ 

```

In Eqn. (8.20), the result is sparse because it is often the case to only obtain observations for some of the landmarks. To relax the problem and accelerate the computation process of the large size matrix, matrix decomposition methods such as Cholesky factorization or QR decomposition are widely used. Also, because of the similar reason the result from Eqn. (8.20) sometimes not full rank thus $\mathbf{A}$ becomes non-invertible. To handle this issue, we can adopt the damping factor λ . Suppose we have not full rank matrix $\mathbf{A}$, thus it is hard to find the inverse matrix of it for Eqn. (8.18). Then we can have the alternative solution by modifying $\mathbf{A}$ as:

$$\Delta \mathcal{V} = -(\mathbf{A} + \lambda \cdot \mathbf{I})^{-1} \mathbf{b}. \quad (\text{Levenberg}) \quad (8.21)$$

The algorithm table of the graph optimization with Levenberg Method is displayed in Algorithm. 8. Levenberg method incorporates a damping parameter that helps prevent the optimization process from diverging, especially in cases where the Gauss-Newton method might encounter singularities or ill-conditioned problems. Also, While the Gauss-Newton method relies solely on the curvature information of the objective function, the Levenberg method combines this curvature information with a scaled identity matrix. This combination allows Levenberg's method to have smoother convergence behavior, making it more stable and less prone to oscillations during the optimization process.

Since the Levenberg method improves upon the Gauss-Newton method by reducing the risk of divergence and enabling stable convergence, it further enhances optimization effectiveness by considering the curvature of the target nonlinear function provided by Marquardt's method. Marquardt's method, also known as the damped least squares method, adjusts the curvature matrix to ensure stability and convergence in nonlinear least squares

Algorithm 9 Graph SLAM with Levenberg-Marquadt Method**Require:** initial graph $G = \{\mathcal{V}, \mathcal{E}\}$, update step λ and error threshold τ

```

1: while  $e(\mathcal{V}) < \tau$  do
2:    $e_{pre}(\mathcal{V}) = \sum_{u_{i,j} \in \mathcal{U}} e_{D,ij}(\mathcal{V}) + \sum_{x_{i,j} \in \mathcal{X}} e_{M,ij}(\mathcal{V})$ 
3:   compute  $\mathbf{A}$ ,  $\mathbf{b}$  and  $c$ 
4:    $\Delta \mathcal{V} = -(\mathbf{A} + \lambda \cdot \text{diag}(\mathbf{A}))^{-1} \mathbf{b}$ 
5:    $\mathcal{V} = \mathcal{V} + \Delta \mathcal{V}$ 
6:    $e(\mathcal{V}) = \sum_{u_{i,j} \in \mathcal{U}} e_{D,ij}(\mathcal{V}) + \sum_{x_{i,j} \in \mathcal{X}} e_{M,ij}(\mathcal{V})$ 
7:   if  $e_{pre}(\mathcal{V}) < e(\mathcal{V})$  then
8:      $\lambda = 2 \cdot \lambda$ 
9:   else
10:     $\lambda = 1/2 \cdot \lambda$ 
11:   end if
12: end while
13: return  $\mathcal{V}$ 

```

problems. To incorporate the curvature information of $\mathbf{A}$, we can rewrite Eqn. (8.21) as the following:

$$\Delta \mathcal{V} = -(\mathbf{A} + \lambda \cdot \text{diag}(\mathbf{A}))^{-1} \mathbf{b}. \quad (\text{Levenberg-Marquardt}) \quad (8.22)$$

By incorporating Marquardt's curvature information, the Levenberg-Marquardt algorithm typically exhibits smoother convergence behavior and enhanced stability compared to the original Levenberg method. This improvement is particularly beneficial for optimization problems with strong nonlinearities or ill-conditioned objective functions. The algorithm table of this Levenberg-Marquadt Method is displayed in Algorithm. 9.

8.3 Jacobians for $SE2$

Jacobians for $SE2$ in Global Frame

To obtain the linearized form of the error as in Eqn. (8.10) and Eqn. (8.11), we need to compute Jacobians expressed in Eqn. (8.12) and Eqn. (8.13). Then what exactly we need to compute for Jacobians?

To have a intuition of our problem, let us consider a 2D graph optimization. To generalize our discussion, let

$$\begin{aligned} z_i &= \begin{bmatrix} z_i^x & z_i^y & z_i^\theta \end{bmatrix}^T \\ &= \begin{bmatrix} \mathbf{t}_i^T & \theta_i \end{bmatrix}^T \end{aligned} \quad (8.23)$$

be the pose of the i^{th} node of our graph, where

$$\mathbf{t}_i = \begin{bmatrix} z_i^x & z_i^y \end{bmatrix}^T, \quad \theta_i = z_i^\theta. \quad (8.24)$$

Also, let

$$x_{ij} = f(z_i, z_j) + \epsilon \quad (8.25)$$

where x_{ij} be the relation (edge) between z_i and z_j . ϵ is a white Gaussian noise. Here f can denote Dynamic Model or Measurement Model. The pose z_i can also be represented as a transform matrix $\mathbf{T}_i$ as:

$$\mathbf{T}_i = \begin{bmatrix} \mathbf{R}_i & \mathbf{t}_i \\ 0 & 1 \end{bmatrix} \quad (8.26)$$

where

$$\mathbf{R}_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \\ \sin \theta_i & \cos \theta_i \end{bmatrix}. \quad (8.27)$$

Then the relative pose $\mathbf{T}_{ij}$ between the nodes z_i and z_j can be represented by transformation matrices as:

$$\begin{aligned} \mathbf{T}_{ij} &= (\mathbf{T}_i)^{-1} \mathbf{T}_j \\ &= \begin{bmatrix} \mathbf{R}_i & \mathbf{t}_i \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{R}_j & \mathbf{t}_j \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}_i^T & -\mathbf{R}_i^T \mathbf{t}_i \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_j & \mathbf{t}_j \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{R}_i^T \mathbf{R}_j & \mathbf{R}_i^T (\mathbf{t}_j - \mathbf{t}_i) \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} R(\theta_j - \theta_i) & R(-\theta_i)(\mathbf{t}_j - \mathbf{t}_i) \\ 0 & 1 \end{bmatrix} \end{aligned} \quad (8.28)$$

where

$$R(\theta) \stackrel{\text{def}}{=} \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (8.29)$$

From Eqn. (8.28), We can rewrite Eqn. (8.25) as the following:

$$f(z_i, z_j) = \begin{bmatrix} R(-\theta_i)(\mathbf{t}_j - \mathbf{t}_i) \\ \theta_j - \theta_i \end{bmatrix}. \quad (8.30)$$

Since the error $\mathbf{e}$ with f can be defined as

$$\mathbf{e}_{ij}(z_i, z_j) = f(z_i, z_j) - x_{ij}, \quad (8.31)$$

similar to Eqn. (8.12) and Eqn. (8.13) we can compute the Jacobian $\mathcal{F}_{ij}$ as:

$$\begin{aligned} \mathcal{F}_{ij} &= \begin{bmatrix} \mathbf{0} & \dots & \frac{\partial \mathbf{e}_{ij}}{\partial z_i} & \dots & \mathbf{0} & \dots & \frac{\partial \mathbf{e}_{ij}}{\partial z_j} & \dots & \mathbf{0} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{0} & \dots & \mathcal{F}_i & \dots & \mathbf{0} & \dots & \mathcal{F}_j & \dots & \mathbf{0} \end{bmatrix}. \end{aligned} \quad (8.32)$$

where

$$\begin{aligned}
\mathcal{F}_i &= \frac{\partial \mathbf{e}_{ij}}{\partial z_i} = \begin{bmatrix} \frac{\partial \mathbf{e}_{ij}}{\partial \mathbf{t}_i} & \frac{\partial \mathbf{e}_{ij}}{\partial \theta_i} \end{bmatrix} \\
&= \begin{bmatrix} \frac{\partial f(z_i, z_j)}{\partial \mathbf{t}_i} & \frac{\partial f(z_i, z_j)}{\partial \theta_i} \end{bmatrix} \\
&= - \begin{bmatrix} R(-\theta_i) & R(-\pi/2) R(-\theta_i) (\mathbf{t}_j - \mathbf{t}_i) \\ 0 & 1 \end{bmatrix} \quad (8.33)
\end{aligned}$$

$$\begin{aligned}
\mathcal{F}_j &= \frac{\partial \mathbf{e}_{ij}}{\partial z_j} = \begin{bmatrix} \frac{\partial \mathbf{e}_{ij}}{\partial \mathbf{t}_j} & \frac{\partial \mathbf{e}_{ij}}{\partial \theta_j} \end{bmatrix} \\
&= \begin{bmatrix} \frac{\partial f(z_i, z_j)}{\partial \mathbf{t}_j} & \frac{\partial f(z_i, z_j)}{\partial \theta_j} \end{bmatrix} \\
&= \begin{bmatrix} R(-\theta_i) & 0 \\ 0 & 1 \end{bmatrix}. \quad (8.34)
\end{aligned}$$

Therefore, we can approximately represent $\mathbf{e}_{ij}(z_i + \Delta z_i, z_j + \Delta z_j)$ as:

$$\mathbf{e}_{ij}(z_i + \Delta z_i, z_j + \Delta z_j) \simeq \mathbf{e}_{ij}(z_i, z_j) + \mathcal{F}_i \Delta z_i + \mathcal{F}_j \Delta z_j \quad (8.35)$$

where

$$\Delta z = \begin{bmatrix} \Delta \mathbf{t} & \Delta \theta \end{bmatrix}^T \quad (8.36)$$

is a pose update according to the global frame.

Jacobians for *SE2* in Node Frame

Until now, we consider the Jacobian and the pose update according to the global frame, thus Δz denotes the update (or increment) based on the global frame. However, in most cases for SLAM, these increments occur according to the robot's movement, so most increments occur in the coordinate frame based on the robot's heading angle. Therefore, it is advantageous for optimization to define Δz as the increment in the robot frame rather than the increment in the global frame.

Then, from now on, let Δz be the increment in the frame of node z . Then, the node z updated with this increment can be expressed as follows:

$$\begin{aligned}
z \boxplus \Delta z &= \begin{bmatrix} \mathbf{t} & \theta \end{bmatrix}^T \boxplus \begin{bmatrix} \Delta \mathbf{t} & \Delta \theta \end{bmatrix}^T \\
&= \begin{bmatrix} \mathbf{t} + \underbrace{R(\theta) \Delta \mathbf{t}}_{\text{heading}} & \theta + \Delta \theta \end{bmatrix}^T. \quad (8.37)
\end{aligned}$$

Here $\boxplus$ operation is used because we redefine the increment of z , which cannot be further expressed with traditional vector operation.

According to our newly defined increment in Eqn. (8.37), let us calculate the Jacobians. From Eqn. (8.30) we have:

$$\begin{aligned}
 f(z_i \boxplus \Delta z_i, z_j \boxplus \Delta z_j) &= \begin{bmatrix} R(-\theta_i - \Delta\theta_i)(t_j + \mathbf{R}_j \Delta t_j - t_i - \mathbf{R}_i \Delta t_i) \\ \theta_j + \Delta\theta_j - \theta_i - \Delta\theta_i \end{bmatrix} \\
 &= \begin{bmatrix} \mathbf{R}_i^T R(-\Delta\theta_i)(t_j + \mathbf{R}_j \Delta t_j - t_i - \mathbf{R}_i \Delta t_i) \\ \theta_j + \Delta\theta_j - \theta_i - \Delta\theta_i \end{bmatrix} \\
 &\simeq \begin{bmatrix} \mathbf{R}_i^T (\mathbf{I} + R(-\frac{\pi}{2}) \Delta\theta_i)(t_j + \mathbf{R}_j \Delta t_j - t_i - \mathbf{R}_i \Delta t_i) \\ \theta_j + \Delta\theta_j - \theta_i - \Delta\theta_i \end{bmatrix}
 \end{aligned} \tag{8.38}$$

where

$$R(-\Delta\theta_i) = \begin{bmatrix} \cos(-\Delta\theta_i) & -\sin(-\Delta\theta_i) \\ \sin(-\Delta\theta_i) & \cos(-\Delta\theta_i) \end{bmatrix} \simeq \begin{bmatrix} 1 & \Delta\theta_i \\ -\Delta\theta_i & 1 \end{bmatrix} = \mathbf{I} + R\left(-\frac{\pi}{2}\right) \Delta\theta_i. \tag{8.39}$$

For the linear approximation, Eqn. (8.38) should be equal to

$$f(z_i, z_j) \boxplus [\mathcal{F}_i \Delta z_i + \mathcal{F}_j \Delta z_j]. \tag{8.40}$$

We want to know $\mathcal{F}_i$ and $\mathcal{F}_j$ For now, for simplicity let

$$\mathcal{F}_i \Delta z_i + \mathcal{F}_j \Delta z_j = \Delta z. \tag{8.41}$$

Then we have:

$$f(z_i, z_j) \boxplus \Delta z = \begin{bmatrix} \mathbf{R}_i^T (t_j - t_i) + \mathbf{R}_i^T \mathbf{R}_j \Delta t \\ \theta_j - \theta_i + \Delta\theta \end{bmatrix}. \tag{8.42}$$

By equating Eqn. (8.38) and Eqn. (8.42), we have:

$$\begin{aligned}
 \Delta z &= \begin{bmatrix} \Delta t \\ \Delta\theta \end{bmatrix} \\
 &= - \underbrace{\begin{bmatrix} \mathbf{R}_j^T \mathbf{R}_i & R(-\frac{\pi}{2}) \mathbf{R}_j^T (t_i - t_j) \\ 0 & 1 \end{bmatrix}}_{\mathcal{F}_i} \underbrace{\begin{bmatrix} \Delta t_i \\ \Delta\theta_i \end{bmatrix}}_{\Delta z_i} + \underbrace{\begin{bmatrix} \mathbf{I} & \mathbf{0} \\ 0 & 1 \end{bmatrix}}_{\mathcal{F}_j} \underbrace{\begin{bmatrix} \Delta t_j \\ \Delta\theta_j \end{bmatrix}}_{\Delta z_j}.
 \end{aligned} \tag{8.43}$$

Consequently, the Jacobians can be given as:

$$\mathcal{F}_i = - \begin{bmatrix} \mathbf{R}_j^T \mathbf{R}_i & R(-\frac{\pi}{2}) \mathbf{R}_j^T (t_i - t_j) \\ 0 & 1 \end{bmatrix} \tag{8.44}$$

$$\mathcal{F}_j = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ 0 & 1 \end{bmatrix}. \tag{8.45}$$

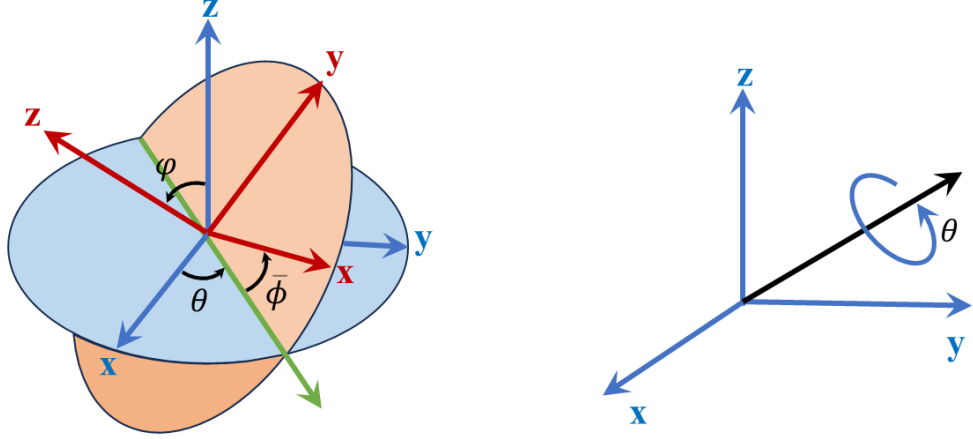


Figure 8.2: Example of the expressions of rotation. Left: Euler angles for 3D rotation. Right: Quaternion rotations.

8.4 Jacobians for $SE3$

Up to this point, we have focused on a 2D graph optimization problem. In this scenario, a pose can be represented by a 3-dimensional vector, consisting of 2 variables for Cartesian coordinates and 1 variable for rotational angle. However, in a 3D graph optimization problem, poses have six degrees of freedom, and rotations are typically represented using Euler angles, which involve three variables as shown in Fig. 8.2. Yet, optimizing poses with Euler Angles often leads to singularities, such as the Gimbal-lock phenomenon, during the optimization process. Conversely, Quaternion representations offer resilience and stability, although they may encounter manifold constraint violations during optimization.

Hence, Quaternions are preferred over Euler angles in 3D Graph SLAM for several reasons. Firstly, they mitigate Gimbal Lock, a situation where two rotation axes align, constraining three-dimensional rotation. Represented by 4D complex numbers, Quaternions ensure stable rotation representation across all states, effectively preventing Gimbal Lock. Moreover, Quaternions offer operational efficiency owing to their compact representation with only four elements. This enhances the efficiency of rotation synthesis and interpolation operations. Spherical Linear Interpolation (Slerp) using Quaternions facilitates smooth interpolation between two rotation states, crucial for tasks such as path planning and optimization. Quaternions also exhibit numerical stability, unlike Euler angles, which tend to accumulate errors in rotation operations, potentially leading to numerical instability. By minimizing cumulative errors, Quaternions provide a more stable rotation representation.

Additionally, Quaternions streamline rotation operations by simplifying rotation matrix computations and ensuring consistent and efficient rotation operations compared to Euler angles. Quaternion multiplication allows for straightforward composition of two rotations and ensures uniformity across various rotation operations.

Rotation and Exponential Map

Suppose z_i is the pose of i^{th} node. Different from the 2D case, let us represent the pose z_i as $\mathbf{T}_i$ where

$$z = \mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix}. \quad (8.46)$$

Since $\mathbf{T} \in SE(3)$, we need to define operations as follows:

$$\mathbf{T} \boxplus \Delta \mathbf{T} \stackrel{\text{def}}{=} \Delta \mathbf{T} \cdot \mathbf{T} \quad (8.47)$$

$$\mathbf{T}_j \boxminus \mathbf{T}_i \stackrel{\text{def}}{=} \mathbf{T}_i^{-1} \mathbf{T}_j. \quad (8.48)$$

Here the function of the relative pose can be defined as:

$$f(z_i, z_j) = \mathbf{T}_j \boxminus \mathbf{T}_i = \mathbf{T}_i^{-1} \mathbf{T}_j$$

Rather using Euler angle representation, we can adopt axis-angle representation. According to Euler's rotation theorem, any rotation or sequence of rotations of a rigid body or coordinate system about a fixed point is equivalent to a single rotation by a given angle θ about a unit vector $\mathbf{u} = [u^x, u^y, u^z]^T$. Here we define a rotation vector $\boldsymbol{\omega}$ as

$$\boldsymbol{\omega} \stackrel{\text{def}}{=} \theta \mathbf{u}. \quad (8.49)$$

The skew matrix of $\boldsymbol{\omega}$ is given as

$$[\boldsymbol{\omega}]_{\times} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}. \quad (8.50)$$

Using $\boldsymbol{\omega}_{\times}$, we can obtain $SO(3)$ representation from the exponential map as the following:

$$\mathbf{R} = \exp([\boldsymbol{\omega}]_{\times}) \quad (8.51)$$

where

$$\begin{aligned}
\exp([\boldsymbol{\omega}]_{\times}) &= \exp\left(\begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}\right) \\
&= \mathbf{I} + [\boldsymbol{\omega}]_{\times} + \frac{1}{2!}([\boldsymbol{\omega}]_{\times})^2 + \frac{1}{3!}([\boldsymbol{\omega}]_{\times})^3 + \dots \\
&= \mathbf{I} + \left(\frac{\sin \theta}{\theta}\right) [\boldsymbol{\omega}]_{\times} + \left(\frac{1 - \cos \theta}{\theta^2}\right) ([\boldsymbol{\omega}]_{\times})^2 \quad (\text{Rodrigues Formula}).
\end{aligned} \tag{8.52}$$

We can extend our expression to $SE3$ as well. Let $\xi = [\boldsymbol{\omega} \quad \boldsymbol{\nu}]^T$, where $\boldsymbol{\nu} = [\nu^x \quad \nu^y \quad \nu^z]^T$. Then the skew matrix and the exponential map are given as:

$$[\xi]_{\times} = \begin{bmatrix} [\boldsymbol{\omega}]_{\times} & \boldsymbol{\nu} \\ \mathbf{0} & 0 \end{bmatrix} \tag{8.53}$$

$$\begin{aligned}
\exp([\xi]_{\times}) &= \exp\left(\begin{bmatrix} [\boldsymbol{\omega}]_{\times} & \boldsymbol{\nu} \\ \mathbf{0} & 0 \end{bmatrix}\right) \\
&= \begin{bmatrix} \exp([\boldsymbol{\omega}]_{\times}) & \mathbf{Q}\boldsymbol{\nu} \\ \mathbf{0} & 1 \end{bmatrix} \\
&= \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix} = \mathbf{T} = z
\end{aligned} \tag{8.54}$$

where

$$\mathbf{Q} = \mathbf{I} + \left(\frac{1 - \cos \theta}{\theta^2}\right) [\boldsymbol{\omega}]_{\times} + \left(\frac{\theta - \sin \theta}{\theta^3}\right) ([\boldsymbol{\omega}]_{\times})^2. \tag{8.55}$$

Therefore, from Eqn. (8.54) we can define the exponential mapping $\exp(\xi)$ from z to ξ as the following:

$$\exp(\cdot) : \xi = [\boldsymbol{\omega} \quad \boldsymbol{\nu}]^T \rightarrow \mathbf{T} = z \tag{8.56}$$

where

$$\mathbf{R} = \exp([\boldsymbol{\omega}]_{\times}) \tag{8.57}$$

$$\mathbf{t} = \mathbf{Q}\boldsymbol{\nu}. \tag{8.58}$$

Similarly, by the definition of logarithm we can also define $\log(\mathbf{T})$ as:

$$\log(\cdot) : z = \mathbf{T} \rightarrow \xi = [\boldsymbol{\omega} \quad \boldsymbol{\nu}]^T \tag{8.59}$$

where

$$[\boldsymbol{\omega}]_{\times} = \log(\mathbf{R}) \quad (8.60)$$

$$\boldsymbol{\nu} = \mathbf{Q}^{-1} \mathbf{t} \quad (8.61)$$

$$\mathbf{Q}^{-1} = \mathbf{I} - \frac{1}{2!} [\boldsymbol{\omega}]_{\times} + \frac{1}{\theta^2} \left(1 - \frac{\theta \sin \theta}{2(1 - \cos \theta)} \right) ([\boldsymbol{\omega}]_{\times})^2. \quad (8.62)$$

Now we have mappings from z to ξ , vice versa. For 3D graph optimization problem, pose z is mapped to ξ thus our error function and the Jacobian are redefined from

$$\begin{aligned} \mathbf{e}_{ij} &= f(z_i, z_j) \boxminus x_{ij} \\ &= \mathbf{T}_{x_{ij}}^{-1} \left(\mathbf{T}_i^{-1} \mathbf{T}_j \right) \end{aligned} \quad (8.63)$$

$$\mathcal{F}_i = \frac{\partial \mathbf{e}_{ij}}{\partial z_i} \quad (8.64)$$

to

$$\begin{aligned} \mathbf{e}_{ij} &= f(\xi_i, \xi_j) \boxminus \xi_{x_{ij}} \\ &= \log \left(\mathbf{T}_{x_{ij}}^{-1} \left(\mathbf{T}_i^{-1} \mathbf{T}_j \right) \right) \end{aligned} \quad (8.65)$$

$$\mathcal{F}_i = \frac{\partial \mathbf{e}_{ij}}{\partial \xi_i} \quad (8.66)$$

Consequently, when we have the one-step increment $\Delta \xi = \begin{bmatrix} \Delta \boldsymbol{\omega} & \Delta \boldsymbol{\nu} \end{bmatrix}$ by substituting Eqn. (8.65) and Eqn. (8.66) to Eqn. (8.18), Δz is given as:

$$\exp([\Delta \xi]_{\times}) = \Delta z = \Delta \mathbf{T}, \quad (8.67)$$

then the pose $\mathbf{T}$ is finally updated as:

$$\mathbf{T} \leftarrow \mathbf{T} \boxplus \Delta \mathbf{T} = \Delta \mathbf{T} \cdot \mathbf{T}. \quad (8.68)$$

Conclusion

In the expansive realm of Simultaneous Localization and Mapping (SLAM), our exploration reaches a pivotal crossroads, where the intricate integration of perception, Bayesian Estimation, and autonomous navigation unfolds into a narrative of profound consequence. This synthesis not only signifies a technological milestone but heralds a transformative paradigm shift, fundamentally reshaping the landscape of robotics and spatial comprehension.

At the core of this culmination is the essence of perception—a cornerstone of SLAM that metamorphoses raw sensor data into a nuanced tapestry of spatial awareness. Mobile robots, equipped with an arsenal of sensors such as LiDAR, vision systems, and depth sensors, engage in an elaborate dance with their surroundings. It surpasses mere object recognition; it involves the discernment of spatial intricacies, decoding environmental dynamics, and interpreting the intricate interplay of data points. This heightened perceptual acuity lays the foundation for SLAM's remarkable capability to navigate and map the continually evolving world.

Bayesian Estimation, standing as a virtuoso in the realm of probabilistic reasoning, takes center stage. Filters like the Extended Kalman Filter, Particle Filter, and Unscented Kalman Filter emerge as masterful orchestrators, harmonizing uncertainties and nonlinearities within dynamic systems. This fusion of probabilistic reasoning and real-time adaptability empowers robots with a sophisticated capability to estimate, predict, and refine their understanding of the environment—a milestone that propels the field toward the realization of intelligent, context-aware machines.

Simultaneous Localization and Mapping (SLAM) emerges as the tacit cartographer, adept at mapping uncharted territories and anchoring robots in dynamic realities. As mobile robots traverse ever-changing landscapes, SLAM serves as the unwavering compass, facilitating simultaneous navigation, localization, and mapping of the surroundings—a feat that bestows robotic entities with an enhanced spatial awareness and an ability to dynamically adapt to their surroundings.

The zenith of this convergence materializes in autonomous navigation, where mobile robots transcend interactive capabilities to autonomous decision-making. The orchestration of path planning, obstacle avoidance, and dynamic route reconfiguration becomes the hallmark of robots navigating complex, real-world scenarios. The promise of autonomy extends beyond mere movement; it encapsulates intelligent adaptability and an innate understanding of the ever-shifting environment—a testament to the maturity of embodied AI.

In conclusion, the intricate synergy of perception, Bayesian Estimation, SLAM, and autonomous navigation unfolds a tapestry of intelligent machines seamlessly interacting with and navigating through the complexities of our world. From bustling city streets to unexplored terrains, the intelligent, embodied robot emerges as a companion, collaborator, and navigator in the journey toward a new era of AI-driven innovation. The convergence of these diverse elements is not merely a technological feat; it is a testament to the evolution of robotics as a field breathing life into machines. As we stand at this juncture, the journey into the frontiers of embodied AI is only beginning, characterized by anticipation, exploration, and the relentless pursuit of intelligent autonomy—an epoch where machines navigate and map the uncharted territories of our collective technological future.